ACADEMY OF TECHNOLOGY

Aedconagar, Hooghly – 712121

Department of Computer Science & Engineering

Object Oriented Programming

Semester Answer Book

• • •

Complete, elaborated answers with worked examples
to the Question Banks for CA1 and CA2

PCC-CS503 · 5th Semester

Academic Year 2026–27

CA1: Abstract Data Types, Classes & Objects, Constructors, Static Members, Encapsulation and Object Relationships.
CA2: Inheritance, Abstract Classes & Interfaces, Packages, Inner Classes, String, StringBuffer and StringBuilder.

How to Use This Book

This book answers every question of the PCC-CS503 Question Banks for CA1 and CA2, in the same order, under the same module and Bloom's-level headings. The marks printed under each question decide the depth of the answer: a one-mark question gets a one- or two-line answer, a two- or three-mark question gets a short paragraph with an example, and a four- or five-mark question gets a full explanation with a worked example, a complete program, or a table, as an examiner expects.

Every answer carries at least one example, because examples are what convince an examiner that a concept has been understood and not merely memorised. Programs are written in Java and follow the usual convention of one public class per file; where an answer needs several classes they are shown together so that they can be pasted into one file and run. The expected output is given beneath the program.

When writing in the examination, reproduce the structure and not the exact words: a definition first, then the explanation, then the example, then (if marks permit) a diagram, comparison table or output. Answers to the same concept asked at different levels deliberately overlap, so that each answer can be read on its own on the night before the examination.

Contents

Module 1: Abstract Data Types (ADTs) 6

Bloom's Level: Remember (L1) 7

Bloom's Level: Understand (L2) 10

Bloom's Level: Apply (L3) 16

Module 2: Features of Object-Oriented Programming 21

Bloom's Level: Remember (L1) 22

Bloom's Level: Understand (L2) 23

Bloom's Level: Apply (L3) 28

Bloom's Level: Analyze (L4) 38

Module 3: Constructors, Overloading and Passing Objects 43

Bloom's Level: Remember (L1) 44

Bloom's Level: Understand (L2) 46

Bloom's Level: Apply (L3) 50

Bloom's Level: Analyze (L4) 59

Module 4: Static Members, Encapsulation and Object Relationships 65

Bloom's Level: Remember (L1) 66

Bloom's Level: Understand (L2) 67

Bloom's Level: Apply (L3) 71

Bloom's Level: Analyze (L4) 82

Module 5: Inheritance 88

Bloom's Level: Remember (L1) 89

Bloom's Level: Understand (L2) 92

Bloom's Level: Apply (L3) 99

Bloom's Level: Analyze (L4) 110

Module 6: Abstract Classes and Interfaces 117

Bloom's Level: Remember (L1) 118

Bloom's Level: Understand (L2) 122

Bloom's Level: Apply (L3) 131

Bloom's Level: Analyze (L4) 142

Module 7: Packages and Inner Classes 150

Bloom's Level: Remember (L1) 151

Bloom's Level: Understand (L2) 154

Bloom's Level: Apply (L3) 160

Bloom's Level: Analyze (L4) 172

Module 8: String, StringBuffer and StringBuilder 183

Bloom's Level: Remember (L1) 184

Bloom's Level: Understand (L2) 185

Bloom's Level: Apply (L3) 188

Bloom's Level: Analyze (L4) 192

MODULE

1

Abstract Data Types (ADTs)

Specification, concrete state space, concrete invariant and abstraction function

Bloom's Level: Remember (L1)

Q1. What does ADT stand for?

1 Mark · CO1

ADT stands for **Abstract Data Type**. It is a mathematical model of a data type that is defined by the values it can hold and the operations that may be performed on them, without saying anything about how those values are stored in memory. For example, a Stack is an ADT: it is known by `push`, `pop` and `isEmpty`, not by whether it is stored in an array or a linked list.

Q2. Define an Abstract Data Type.

2 Marks · CO1

An Abstract Data Type is a user-defined type that is specified completely by (i) a set of abstract values it can take and (ii) a set of operations that act on those values, while the internal representation and the algorithms that implement the operations remain hidden from the user. The user works only with the specification, so the implementation can be changed without affecting any client program.

> **Example.** A Queue ADT has the values "sequences of items" and the operations `enqueue`, `dequeue` and `isEmpty`. Whether the queue is held in a circular array or a linked list is invisible to the user, who only knows that the item enqueued first is dequeued first.

Q3. What is meant by ADT specification?

2 Marks · CO1

An ADT specification is the precise, implementation-independent description of an ADT. It states the abstract values (the data), the name and signature of every operation, and for each operation its **precondition** (what must be true before it is called) and **postcondition** (what is guaranteed to be true afterwards). It is the contract between the person who implements the type and the person who uses it.

> **Example.** Specification of `pop()` on a Stack: precondition — the stack is not empty; postcondition — the top item is removed and returned, and the size reduces by one.

Q4. What is a concrete state space?

2 Marks · CO1

The concrete state space is the set of all possible states of the chosen implementation, that is, every combination of values that the implementation's variables can hold. It is the "raw" space in which the data lives in memory, and it is usually larger than the set of states that are actually meaningful.

> **Example.** If a Text is stored as `char[] data` (capacity 10) together with `int length`, the concrete state space is the set of all pairs (data, length), including meaningless ones such as length = 99 or length = -4.

Q5. What is a concrete invariant?

2 Marks · CO1

A concrete invariant is a condition over the concrete variables that must be true for every valid concrete state, before and after every operation. It separates the legal concrete states from the nonsensical ones in the concrete state space. Every operation may assume it on entry and must restore it on exit.

> **Example.** For the Text representation (`data`, `length`) the invariant is `0 <= length <= data.length`. A state with `length = 99` and an array of size 10 violates it and is therefore not a valid Text.

Q6. What is an abstraction function?

2 Marks · CO1

An abstraction function is a mapping from the concrete states that satisfy the invariant to the abstract values of the ADT. It tells us, for each valid concrete state, which abstract value that state represents. It is normally many-to-one, because several different concrete states may represent the same abstract value.

> **Example.** For Text, AF(data, length) = the sequence data[0], data[1], ..., data[length - 1]. So AF([‘H’,’I’,’x’,’x’], 2) = “HI”.

Q7. What is the purpose of an abstraction function?

2 Marks · CO1

The purpose of the abstraction function is to give the concrete representation its meaning. Without it, the array and the integer in an implementation are only bits; with it, they become a Text, a Stack or a Set. It is also the yardstick for correctness: an operation is correct if, after it runs, the abstraction function applied to the new concrete state gives exactly the abstract value the specification promises.

> **Example.** After `concat("AB","CD")` the concrete array may be [‘A’,’B’,’C’,’D’] with length 4; AF of this state is “ABCD”, which matches the specification.

Q8. What is meant by an ADT operation?

2 Marks · CO1

An ADT operation is a named action defined on the ADT that creates, observes or changes its values. Operations are the only interface through which a client may touch the data. They are usually grouped as **constructors** (create a value), **mutators** (change a value), and **observers** (report information without changing it).

> **Example.** In a Stack: `new Stack()` is a constructor, `push(x)` and `pop()` are mutators, and `isEmpty()` and `size()` are observers.

Q9. Give one example of an ADT.

2 Marks · CO1

A **Stack** is a standard ADT. Its abstract value is a sequence of items in which insertions and deletions take place at one end only, the top. Its operations are `push(item)`, `pop()`, `peek()` and `isEmpty()`. Other common ADTs are Queue, List, Set, Map and Text.

> **Example.** Pushing 5, 8, 2 and then calling `pop()` returns 2, because a Stack follows the Last-In-First-Out rule; the user never needs to know whether an array or a linked list is used underneath.

Q10. What is the difference between an abstract state and a concrete state?

2 Marks · CO1

An **abstract state** is a value as the user of the ADT sees it, for example the sequence “HELLO”. A **concrete state** is the actual content of the implementation’s variables that represents it, for example the array [‘H’,’E’,’L’,’L’,’O’,’?’,’?’] and the integer 5. One abstract state can be represented by many concrete states (the leftover cells ‘?’ may hold anything), but each valid concrete state represents exactly one abstract state.

Q11. What is the role of an invariant in ADT implementation?

2 Marks · CO1

The invariant defines which concrete states are valid. It lets every operation be written on the safe assumption that the data is well formed, and it obliges every operation to leave the data well formed. Thus the invariant acts as a guard against corruption of the representation and is the basis for proving the implementation correct.

> **Example.** In an array-based Stack with `top` as the index of the next free cell, the invariant `0 <= top <= capacity` guarantees that `push` and `pop` never read outside the array when the preconditions hold.

Q12. What is meant by implementation of an ADT?

2 Marks · CO1

The implementation of an ADT is the concrete realisation of its specification in a programming language: it chooses a data structure (the concrete representation), writes the code of every operation, and ensures that these operations obey the specification. In Java this is normally a class whose fields are private and whose public methods are the ADT operations.

> **Example.** The Stack ADT may be implemented as a class `ArrayStack` with a private `int[] items` and `int top`, or as `LinkedStack` with a chain of nodes. Both are different implementations of one specification.

Q13. What does the Text example illustrate?

2 Marks · CO1

The Text example illustrates, in a simple setting, how an abstract value (a sequence of characters) is related to a concrete representation (a character array plus a length). It shows the three ideas that connect any implementation to its ADT: the concrete state space (all arrays and lengths), the concrete invariant (`0 <= length <= data.length`), and the abstraction function (the first `length` characters). It also shows that the unused cells of the array do not belong to the abstract value.

> **Example.** `data = {'J','A','V','A','X','X'}` with `length = 4` represents the Text “JAVA”; the two X’s are ignored.

Q14. What is a concrete representation?

2 Marks · CO1

A concrete representation is the way an abstract value is actually stored using the data structures of a programming language: arrays, linked nodes, primitive variables and so on. It is the “inside” of the ADT, hidden from clients.

> **Example.** A Stack can be represented concretely by an `int[]` array and an integer `top`; a Text can be represented by a `char[]` array and an integer `length`.

Q15. Name the three important concepts used to relate an implementation to an ADT.

2 Marks · CO1

The three concepts are:

- **Concrete state space** — the set of all states the implementation's variables can take.
- **Concrete invariant** — the condition that selects the valid states from that space.
- **Abstraction function** — the mapping from valid concrete states to the abstract values of the ADT.

> **Example.** For Text: the space is all pairs (char[], int); the invariant is `0 <= length <= data.length`; the abstraction function returns the first `length` characters.

Bloom's Level: Understand (L2)

Q1. Explain the concept of an Abstract Data Type with a suitable example.

3 Marks · CO1

An Abstract Data Type separates **what** a data type does from **how** it does it. The type is described only by the set of values it can take and by the operations that act on those values. The programmer who uses the type sees a clean interface; the programmer who implements it is free to choose any data structure, provided the specification is satisfied. This separation gives three benefits: abstraction (the user need not learn the internals), modularity (the implementation may be replaced without changing client code), and reliability (the data can only be changed through controlled operations).

The word "abstract" means that the type is defined mathematically and not by a particular memory layout. In Java, the abstraction is realised with a class that keeps its fields `private` and exposes only public methods, or with an `interface` that several classes implement.

> **Example. Stack ADT.** Values: finite sequences of items. Operations: `push(x)`, `pop()`, `peek()`, `isEmpty()`. A client can write `s.push(10); s.push(20); int x = s.pop();` and be sure that `x` is 20. Whether `s` is built on an array or a linked list is irrelevant to the client. If the array is later replaced by a linked list for better growth, not a single line of client code changes.

Q2. Explain the difference between ADT specification and ADT implementation.

3 Marks · CO1

The **specification** says what the ADT does; the **implementation** says how it does it. The specification is written for the users of the type and is independent of any programming language or data structure. The implementation is written for the machine and consists of a concrete representation and the code of the operations. One specification can have many implementations, but every implementation must satisfy the one specification.

Aspect	ADT Specification	ADT Implementation
Answers the question	What does it do?	How does it do it?
Audience	Users (clients)	Implementers and maintainers

Aspect	ADT Specification	ADT Implementation
Contents	Values, operations, pre- and postconditions	Data structure, invariant, abstraction function, code
Visibility	Public interface	Hidden (private)
Changeability	Stable; changing it breaks clients	May change freely if the specification still holds

> **Example.** Specification of `Stack`: `pop()` returns the most recently pushed item. Implementation 1 stores items in an array; implementation 2 stores them in a linked list. Both satisfy the same specification.

Q3. Explain the purpose of a concrete state space in implementing an ADT.

3 Marks · CO1

When we implement an ADT we must decide how to store its values, and the concrete state space is the formal description of that decision: it lists every state that the chosen variables can possibly hold. Its purpose is threefold. First, it fixes the "universe" in which the implementation operates, so that the invariant and the abstraction function can be defined precisely over it. Second, it lets us see which states are legal and which are not, because the space is nearly always bigger than the set of meaningful states. Third, it makes clear that the same abstract value may correspond to many points in this space, which is why the abstraction function is many-to-one.

> **Example.** For a Text stored as `char[] data` of size 5 and `int length`, the space contains pairs such as (['H','I','A','B','C'], 2) and (['H','I','Z','Z','Z'], 2). Both lie in the space and both represent "HI". A pair such as (['H','I','A','B','C'], 8) is also in the space but is not valid, and the invariant is what rejects it.

Q4. What is a concrete invariant? Why is it necessary?

3 Marks · CO1

A concrete invariant is a logical condition on the concrete variables that is true of every legal state of the implementation. It is established by the constructor, assumed at the start of each operation, and re-established at the end of each operation.

It is necessary for four reasons:

- It identifies which concrete states are meaningful, filtering out garbage states from the concrete state space.
- It lets each operation be written simply, since the programmer may rely on well-formed data and need not re-check everything.
- It is the key to proving correctness: if every operation preserves the invariant, the invariant holds forever.
- It documents design assumptions, helping others to maintain the code.

> **Example.** In a Text, the invariant is `0 <= length <= data.length`. The method `charAt(i)` can simply return `data[i]` after checking `i < length`, because the invariant guarantees that cells 0 to length - 1 exist. If `length` were allowed to exceed the array size, `charAt` would throw an out-of-bounds exception.

Q5. Explain the role of an abstraction function in ADT implementation.

3 Marks · CO1

The abstraction function is the bridge between the implementer's world and the user's world. It interprets each valid concrete state as an abstract value. Its roles are:

- **Giving meaning**: it declares that a particular array and integer represent a particular Text or Stack.
- **Defining correctness**: an operation is correct when the abstract value obtained by applying the function to the resulting concrete state is the value that the specification demands.
- **Explaining many-to-one representation**: different concrete states may map to one abstract value, and the function makes this explicit.
- **Allowing change of representation**: a new implementation needs only a new abstraction function; the specification stays untouched.

> **Example.** For Text, AF(data, length) = data[0..length-1]. After `append('S')` on ("JAVA", array ['J','A','V','A','?'], length 4) the concrete state becomes (['J','A','V','A','S'], 5) and AF gives "JAVAS", exactly the value required by the specification.

Q6. Differentiate between: Abstract state and Concrete state

3 Marks · CO1

Basis	Abstract State	Concrete State
Meaning	Value as seen by the user of the ADT	Actual content of the implementation's variables
Level	Conceptual, mathematical	Physical, programming-language level
Example (Text)	The sequence "HELLO"	data = ['H','E','L','L','O','?','?'], length = 5
Uniqueness	Each abstract state is a single value	An abstract state can have many concrete states
Visibility	Visible to clients	Hidden inside the class
Link	Obtained from the concrete state by the abstraction function	Must satisfy the concrete invariant

An abstract state is what we **mean**; a concrete state is what we **store**. The same abstract state "HELLO" is equally represented by (['H','E','L','L','O'], 5) and by (['H','E','L','L','O','Z','Z'], 5), so two distinct concrete states correspond to one abstract state. The reverse never happens: a valid concrete state cannot represent two different abstract values.

Q7. Explain how an abstraction function connects a concrete representation with an abstract data type.

3 Marks · CO1

An ADT is defined at the abstract level (values and operations) and is implemented at the concrete level (variables and code). The two levels are otherwise unrelated, and the abstraction function is the formal link between them. Given a concrete state that satisfies the invariant, the function returns the abstract value that this state stands for. Every operation of the implementation is then judged by the following rule: if the operation takes concrete state C to concrete state C', then the abstract operation applied to AF(C) must equal AF(C').

In this way the function lets us reason about the program without looking at the array indices: we think: the Text was HELLO and is now HELLO WORLD, while the code only moves characters in an array.

> **Example.** Concrete state C = (['A','B','?','?'], 2); AF(C) = "AB". After `append('C')`, C' = (['A','B','C','?'], 3) and AF(C') = "ABC", which is "AB" followed by 'C' as the abstract specification requires.

Q8. Explain the relationship among: a) Concrete state space, b) Concrete invariant, c) Abstraction function

3 Marks · CO1

The three notions are layered. The **concrete state space** is the largest set: all values that the representation variables could hold. The **concrete invariant** selects from it a smaller subset, the **valid** concrete states. The **abstraction function** is defined only on this valid subset and maps each of its members to an abstract value of the ADT.

Concept	Role	Text example
Concrete state space	Everything the variables can hold	All pairs (char[5], int)
Concrete invariant	Picks out the valid states	0 <= length <= 5
Abstraction function	Maps valid states to abstract values	First length characters

The relationship can be shown as: state space (all) contains valid states (invariant) which map onto abstract values (abstraction function). Removing any one of the three leaves the implementation unexplained: without the invariant, the function would have to give meaning to garbage; without the function, the valid states would have no meaning.

> **Example.** The pair (['C','A','T','?','?'], 3) is in the space, satisfies the invariant, and is mapped to the abstract value "CAT". The pair (['C','A','T','?','?'], 9) is in the space but fails the invariant and is not mapped to anything.

Q9. Explain how an ADT can have different implementations while maintaining the same specification.

3 Marks · CO1

A specification mentions only abstract values and operations, so any concrete representation that can support these operations correctly is an acceptable implementation. Different implementations choose different concrete state spaces, invariants and abstraction functions, yet all of them make the operations behave as promised. The client, who has only the specification, cannot tell them apart except possibly by speed and memory use.

> **Example. Stack.** Implementation A uses `int[] items` and `int top`, with invariant 0 <= top <= items.length and AF = items[0..top-1]. Implementation B uses a linked list of nodes, with invariant "head is null or heads a finite chain" and AF = the nodes read from head to tail. Both satisfy "`pop()` returns the last pushed item", so a program `s.push(1); s.push(2); s.pop();` returns 2 in either. Choosing between them is a matter of efficiency (arrays are compact; lists grow without limit).

Q10. Explain the significance of specifying operations before implementing an ADT.

3 Marks · CO1

Writing the specification first is good engineering practice for the following reasons:

- **Clarity of purpose**: it forces the designer to decide precisely what the type must do before worrying about how.
- **Independence**: clients and implementers can work in parallel, because the specification is the agreed contract.
- **Freedom to optimise**: the implementer may choose or later change the data structure so long as the contract is kept.
- **Testability**: pre- and postconditions become the test cases of the operations.
- **Fewer defects**: ambiguities such as "what does `pop()` do on an empty stack?" are discovered early, when they are cheap to fix.

> **Example.** Before coding a Stack, we decide that `pop()` has the precondition "stack not empty". The implementer then knows the code need not handle emptiness silently, and the tester knows to include a test that calls `pop()` on an empty stack to check the stated behaviour.

Q11. Consider a Stack ADT with push(), pop() and isEmpty(). Explain what information should be included in its specification.

3 Marks · CO1

A complete specification of the Stack ADT should contain:

- **Name and nature of the data**: a finite sequence of items with one open end, the top.
- **Signature of each operation**: name, parameters and result type.
- **Preconditions**: what the caller must guarantee.
- **Postconditions**: what the operation guarantees on return, including effect on the state.
- **Special behaviour and errors**: for example what happens on pop from an empty stack.

Operation	Precondition	Postcondition
push(x)	None (stack not full, if bounded)	x is on top; size increases by 1

Operation	Precondition	Postcondition
pop()	Stack not empty	Top item removed and returned; size decreases by 1
isEmpty()	None	Returns true if and only if the stack has no items; state unchanged

> **Example.** Starting with an empty stack, `push(4); push(9); pop()` must return 9 and leave the stack holding only 4, and `isEmpty()` must then return false.

Q12. Suppose a Text ADT represents a sequence of characters. Explain the purpose of operations: a) length() b) charAt() c) concat()

3 Marks · CO1

- **length()** is an observer that returns the number of characters in the Text. It lets the client know the valid range of positions and loop safely over the Text. Example: `length("HELLO")` = 5.
- **charAt(i)** is an observer that returns the character at position `i` (0-based). Its precondition is `0 <= i < length()`. It lets the client read individual characters without access to the storage. Example: `charAt("HELLO", 1)` = 'E'.
- **concat(T1, T2)** is a constructor-like operation that produces a new Text consisting of all characters of T1 followed by all characters of T2, leaving both originals unchanged. Example: `concat("Object","Oriented")` = "ObjectOriented".

Together these three are enough to read, traverse and combine Texts, which is why they appear in the minimal specification of the ADT.

Q13. Explain how the Text example can be represented using a concrete data structure.

3 Marks · CO1

A Text is a sequence of characters, and the natural concrete structure for a sequence of known maximum size is an array. We therefore use two fields: a character array `data` to hold the characters and an integer `length` to record how many cells of the array are in use. The length field is needed because the array has a fixed capacity that may exceed the Text length.

- **Concrete state space**: all pairs (data, length).
- **Concrete invariant**: `0 <= length <= data.length`.
- **Abstraction function**: AF(data, length) = the sequence data[0], ..., data[length - 1].

```
class Text {
    private char[] data;
    private int length;
    Text(String s) { data = s.toCharArray(); length = data.length; }
    int length() { return length; }
    char charAt(int i) { return data[i]; }   // 0 <= i < length
}
```

> **Example.** `new Text("HELLO")` creates data = ['H','E','L','L','O'] and length = 5, which by the abstraction function is the Text "HELLO".

Bloom's Level: Apply (L3)

Q1. Design an ADT specification for a Stack. Include data, operations, purpose of each operation, preconditions and postconditions.

4 Marks · CO2

Data. A Stack of items of type T is a finite sequence s = (a1, a2, ..., an), n >= 0, in which all insertions and deletions take place at one end called the top (an). It follows the Last-In-First-Out (LIFO) discipline.

Operations, purpose, preconditions and postconditions.

Operation	Purpose	Precondition	Postcondition
create()	Make an empty stack	None	Stack is empty (n = 0)
push(x)	Add item x on the top	Stack not full (if bounded)	New top is x; n increases by 1
pop()	Remove and return the top item	n > 0	Returns old top; n decreases by 1
peek()	Read the top without removing	n > 0	Returns top; stack unchanged
isEmpty()	Test for emptiness	None	Returns true iff n = 0; unchanged
size()	Report number of items	None	Returns n; unchanged

> **Example.** Start with create(); push(10); push(20); push(30). Now size() = 3 and peek() = 30. pop() returns 30 and leaves (10, 20); a second pop() returns 20. After the third pop(), isEmpty() returns true and any further pop() violates its precondition.

Java interface form of the specification:

```
interface Stack<T> {
    void push(T x);   // post: x is the new top
    T pop();          // pre : !isEmpty()
    T peek();         // pre : !isEmpty()
    boolean isEmpty();
    int size();
}
```

Q2. Consider a Text ADT represented using a character array and an integer length. For the text HELLO identify the concrete state, concrete invariant, abstract value and abstraction function.

4 Marks · CO2

Let the representation be `char[] text` and `int length`, with the array capacity chosen as 8.

- **Concrete state:** text = ['H','E','L','L','O','?','?','?'] and length = 5. The cells marked '?' are unused and may hold any characters.
- **Concrete invariant:** `0 <= length <= text.length`, that is, 0 <= 5 <= 8, which holds.
- **Abstract value:** the sequence of characters "HELLO" (five characters).

- **Abstraction function:** AF(text, length) = text[0] text[1] ... text[length - 1]. Applying it: AF = ‘H’,’E’,’L’,’L’,’O’ = “HELLO”.

> **Example.** Another concrete state, ([‘H’,’E’,’L’,’L’,’O’,’Z’,’Q’,’X’], 5), is different from the first but has the same abstract value “HELLO”, because AF ignores everything at index 5 and beyond. This shows that the abstraction function is many-to-one.

Q3. Implement length() and charAt(position) for a simple Text ADT using a suitable concrete representation.

4 Marks · CO2

The representation is a `char[] data` with an `int length`; the invariant is `0 <= length <= data.length`.

```
public class Text {
    private char[] data;
    private int length;          // invariant: 0 <= length <= data.length

    public Text(String s) {
        data = s.toCharArray();
        length = data.length;
    }

    // observer: number of characters in the Text
    public int length() {
        return length;
    }

    // observer: pre-condition 0 <= position < length
    public char charAt(int position) {
        if (position < 0 || position >= length)
            throw new IndexOutOfBoundsException("Bad position: " + position);
        return data[position];
    }

    public static void main(String[] args) {
        Text t = new Text("HELLO");
        System.out.println(t.length());      // 5
        System.out.println(t.charAt(1));     // E
    }
}
```

Output

```
5
E
```

> **Example.** `charAt(5)` on “HELLO” violates the precondition (valid positions are 0 to 4), so the method throws `IndexOutOfBoundsException` instead of returning an unused array cell.

Q4. For text = [‘C’,’O’,’M’,’P’,’U’,’T’,’E’,’R’] and length = 8, answer: a) concrete state b) abstract Text value c) concrete invariant d) abstraction function output.

4 Marks · CO2

a) **Concrete state:** the pair (text, length) = ([‘C’,’O’,’M’,’P’,’U’,’T’,’E’,’R’], 8). The array is completely filled, because the length equals the array size.

b) **Abstract Text value:** the sequence of eight characters “COMPUTER”.

c) **Concrete invariant:** `0 <= length <= text.length`. Here 0 <= 8 <= 8, which is true (the upper limit is reached, which is still legal).

d) **Abstraction function output:** AF(text, length) = text[0] ... text[length - 1] = text[0] ... text[7], which produces “COMPUTER”.

> **Example.** If the same array had length = 3, the invariant would still hold, but AF would produce only “COM”. The array is the same, yet the abstract value differs because the abstraction function is applied to both fields together.

Q5. Develop an ADT specification for a simple List containing integers. Specify insertion, deletion, searching and size operations.

4 Marks · CO2

Data. A List of integers is a finite ordered sequence L = (x0, x1, ..., x(n-1)), n >= 0, in which duplicates are allowed and positions are numbered from 0.

Operation	Purpose	Precondition	Postcondition
insert(i, x)	Insert x at position i, shifting later items right	0 <= i <= n	Size is n + 1; element at i is x
delete(i)	Remove and return the element at position i	0 <= i < n	Size is n - 1; later items shifted left
search(x)	Find the first position of x	None	Returns index of first x, or -1 if absent; list unchanged
size()	Number of elements	None	Returns n; list unchanged

> **Example.** L = (4, 9, 7). `insert(1, 5)` gives (4, 5, 9, 7) and size() = 4. `search(9)` returns 2. `delete(0)` returns 4 and leaves (5, 9, 7). `search(100)` returns -1 because 100 is not in the list.

```
interface IntList {
    void insert(int i, int x);   // 0 <= i <= size()
    int  delete(int i);          // 0 <= i <  size()
    int  search(int x);          // index or -1
    int  size();
}
```

Q6. A Text ADT supports concat(T1,T2). Given T1 = “Object”, T2 = “Oriented”: a) expected abstract result b) specify the operation c) describe one concrete implementation.

4 Marks · CO2

a) **Expected abstract result:** concat(“Object”, “Oriented”) = “ObjectOriented”, of length 6 + 8 = 14.

b) **Specification.**

- Signature: `Text concat(Text t1, Text t2)`.
- Precondition: none (either Text may be empty).

- Postcondition: the result R satisfies length(R) = length(t1) + length(t2); for 0 <= i < length(t1), charAt(R, i) = charAt(t1, i); and for 0 <= j < length(t2), charAt(R, length(t1) + j) = charAt(t2, j). The arguments t1 and t2 are not modified.

c) **Concrete implementation.** Allocate a new array of size length(t1) + length(t2), copy the characters of t1 and then those of t2 into it, and set length accordingly.

```
static Text concat(Text a, Text b) {
    char[] d = new char[a.length + b.length];
    System.arraycopy(a.data, 0, d, 0, a.length);
    System.arraycopy(b.data, 0, d, a.length, b.length);
    Text r = new Text(d);          // sets data = d, length = d.length
    return r;
}
```

> **Example.** The new array is [‘O’,’b’,’j’,’e’,’c’,’t’,’O’,’r’,’i’,’e’,’n’,’t’,’e’,’d’] with length 14, and AF of this state is “ObjectOriented”, matching the abstract result.

Q7. Implement a simple Text ADT using a character array. Provide create(), length(), charAt(). Identify the concrete invariant.

4 Marks · CO2

```
public class Text {
    private char[] data;    // storage
    private int length;     // number of valid characters

    // Concrete invariant: data != null && 0 <= length <= data.length

    // create(): constructors
    public Text(int capacity) {          // creates an empty Text
        data = new char[capacity];
        length = 0;
    }
    public Text(String s) {              // creates Text from a String
        data = s.toCharArray();
        length = data.length;
    }

    public int length() { return length; }

    public char charAt(int i) {          // pre: 0 <= i < length
        if (i < 0 || i >= length) throw new IndexOutOfBoundsException();
        return data[i];
    }

    public static void main(String[] a) {
        Text t = new Text("JAVA");
        System.out.println(t.length() + " " + t.charAt(2));
        Text e = new Text(10);
        System.out.println(e.length());
    }
}
```

Output

```
4 V
0
```

Concrete invariant: `data != null` and `0 <= length <= data.length`. Both constructors establish it (for the empty Text, 0 <= 0 <= capacity; for the String Text, length equals the array size), and neither `length()` nor `charAt()` modifies the state, so it is preserved.

> **Example.** `new Text(10)` represents the empty Text (length 0) even though ten cells have been reserved; the abstraction function returns the empty sequence.

Q8. Consider char[] data = {'J','A','V','A','X','X','X'}; int length = 4; a) abstract Text b) unused portion c) abstraction function d) concrete invariant.

4 Marks · CO2

a) **Abstract Text:** only the first `length` = 4 characters are meaningful, so the Text is "JAVA".

b) **Unused portion:** the cells data[4], data[5] and data[6], holding 'X','X','X'. They are spare capacity and are not part of the abstract value; they could hold any characters without changing the Text.

c) **Abstraction function:** AF(data, length) = data[0] data[1] ... data[length - 1]. Here AF = 'J','A','V','A' = "JAVA". In code: `new String(data, 0, length)`.

d) **Concrete invariant:** `0 <= length <= data.length`, that is, 0 <= 4 <= 7, true.

> **Example.** If `append('S')` is performed, the concrete state becomes ({'J','A','V','A','S','X','X'}, 5); the invariant still holds (5 <= 7) and AF now returns "JAVAS". The 'X' at index 4 was overwritten, but that cell had never been part of the Text.

MODULE

2

Features of Object-Oriented Programming

Java basics, classes and objects, object identity, encapsulation and polymorphism (Unit 2)

Bloom's Level: Remember (L1)

Q1. What is Java?

1 Mark · CO1

Java is a high-level, object-oriented, platform-independent programming language developed by James Gosling at Sun Microsystems (1995) and now owned by Oracle. Java source code is compiled into bytecode that runs on the Java Virtual Machine, so the same program runs on any operating system that has a JVM.

Q2. What is a class?

2 Marks · CO1

A class is a user-defined blueprint or template that describes the data (fields) and behaviour (methods) common to a group of similar objects. It does not occupy memory for data until objects are created from it.

> **Example.** `class Car { String colour; void drive() { ... } }` describes what every car has (a colour) and can do (drive); each actual car is an object created from this class.

Q3. What is an object?

2 Marks · CO1

An object is a run-time instance of a class. It has **state** (the values held in its instance variables), **behaviour** (the methods that can be called on it) and **identity** (it is distinct from every other object, even one with the same state). Memory for an object is allocated on the heap when it is created with `new`.

> **Example.** `Car c1 = new Car();` creates an object `c1` of class `Car`; `Car c2 = new Car();` creates another, separate object.

Q4. Which keyword is used to create an object?

1 Mark · CO1

The keyword **`new`**. It allocates memory for the object on the heap, invokes the constructor and returns a reference to the object. Example: `Student s = new Student();`.

Q5. What is JVM?

2 Marks · CO1

The JVM (Java Virtual Machine) is the abstract machine that loads, verifies and executes Java bytecode. It also manages memory (heap, stack) and performs garbage collection. Because each operating system has its own JVM, the same `.class` file runs everywhere, which gives Java its "write once, run anywhere" nature.

> **Example.** `javac Hello.java` produces `Hello.class` (bytecode); `java Hello` starts the JVM, which executes that bytecode.

Q6. What is the extension of a Java source file?

1 Mark · CO1

The extension is **`.java`** (for example `Student.java`). After compilation the bytecode file has the extension `.class`.

Q7. What is the entry point of a Java application?

2 Marks · CO1

The entry point is the `main` method, whose standard signature is `public static void main(String[] args)`. The JVM calls it to start the program, and execution begins from its first statement.

> **Example.** `public class Hello { public static void main(String[] args) { System.out.println("Hi"); } }` prints Hi.

Q8. What is a method?

1 Mark · CO1

A method is a named block of statements inside a class that performs a specific task and may take parameters and return a value. It represents the behaviour of an object. Example: `int add(int a, int b) { return a + b; }`.

Q9. What is an instance variable?

1 Mark · CO1

An instance variable is a variable declared inside a class but outside any method, without the `static` keyword. Every object has its own separate copy of it, so it stores the state of that object. Example: `int roll;` in class `Student`.

Q10. What is an object reference?

2 Marks · CO1

An object reference is a variable that holds the address (handle) of an object in the heap and through which the object is accessed. The reference is not the object itself; it is only a way to reach it. Several references can refer to one object.

> **Example.** In `Student s = new Student();`, `s` is the reference and the object lies on the heap; `Student t = s;` makes `t` a second reference to the same object.

Bloom's Level: Understand (L2)

Q1. Differentiate between a class and an object.

2 Marks · CO1

Basis	Class	Object
Nature	Blueprint or template	Real instance of the class
Memory	No memory for data until instantiated	Occupies memory on the heap
Creation	Declared once with `class`	Created many times with `new`
Existence	Logical entity	Physical entity at run time
Example	`class Dog`	`Dog d = new Dog();`

A class is like an architect's plan of a house; objects are the houses built from it. One class can produce any number of objects, each with its own state.

Q2. Why is Java called platform independent?

2 Marks · CO1

A Java compiler does not produce machine code for a particular processor; it produces **bytecode**, an intermediate, processor-neutral code. This bytecode is executed by the JVM, which is available for Windows, Linux, macOS and so on. Therefore a program compiled on one platform runs without recompilation on any other platform having a JVM.

> **Example.** A `Hello.class` file compiled on Windows can be copied to a Linux machine and run with `java Hello` without any change.

Q3. Explain the purpose of the new keyword.

2 Marks · CO1

The `new` operator creates an object. It performs three actions: it allocates memory for the object on the heap, it calls the constructor to initialise the instance variables, and it returns the reference of the new object, which is normally stored in a reference variable. Without `new`, a reference variable holds only `null`.

> **Example.** `Rectangle r = new Rectangle(4, 5);` creates a Rectangle object with length 4 and breadth 5 and stores its reference in `r`.

Q4. What is an object reference?

2 Marks · CO1

An object reference is a variable of class type that stores the location of an object on the heap. All access to the object, such as reading a field or calling a method, takes place through the reference using the dot operator. A reference may hold `null`, meaning that it refers to no object, and using it then causes a `NullPointerException`.

> **Example.** `Student s = new Student(); s.roll = 5;` the variable `s` refers to the object, and `s.roll` reaches its field. `Student p = null; p.roll = 1;` fails with a `NullPointerException`.

Q5. Explain the difference between an instance variable and a local variable.

2 Marks · CO1

Basis	Instance Variable	Local Variable
Declared	Inside the class, outside methods	Inside a method, constructor or block
Memory	Heap (inside the object)	Stack (of the method call)
Default value	Yes (0, false, null)	None; must be initialised before use
Lifetime	As long as the object exists	Only while the method runs
Scope	Whole class (through the object)	Only inside its block
Access modifiers	Allowed	Not allowed

> **Example.** In `class A { int x = 5; void f() { int y = 10; } }`, `x` is an instance variable (each object of A has one) and `y` is a local variable that disappears when `f()` ends.

Q6. Can two reference variables refer to the same object? Explain.

2 Marks · CO1

Yes. A reference variable stores only an address, so assigning one reference to another copies the address and both variables then point to the same object. No new object is created. A change made to the object through one reference is visible through the other, because there is only one object.

> **Example.** `Box a = new Box(); a.side = 4; Box b = a; b.side = 9; System.out.println(a.side);` prints 9, since `a` and `b` refer to one object.

Q7. Why are methods used inside a class?

2 Marks · CO1

Methods define the behaviour of the objects of a class. They operate on the instance variables, giving a controlled way of reading or changing the state (encapsulation). They also allow code to be written once and reused (reusability), break a large problem into small parts (modularity) and make the program easier to test and maintain.

> **Example.** A `BankAccount` class has `deposit()` and `withdraw()` methods; every account object can use them instead of each program repeating the same code.

Q8. Explain the relationship between a class, object and method.

3 Marks · CO1

A **class** is the blueprint that declares the data members and the methods. An **object** is an instance created from that blueprint; it holds actual values in the data members. A **method** is a function defined in the class that works on the data of the object through which it is called. Thus the class defines, the object stores, and the method acts.

> **Example.** Class `Circle` has data `radius` and method `area()`. The statements `Circle c1 = new Circle(); c1.radius = 2;` create an object, and `c1.area()` runs the method on that object's radius, giving 12.56. Another object `c2` with radius 5 gives 78.5 from the same method, because the method uses the data of whichever object calls it.

Q9. Why are data members generally declared inside a class?

2 Marks · CO1

Data members hold the state of an object. Declaring them inside the class means that every object created from the class gets its own copy of this state, and the methods of the class can use them directly. It also keeps related data and the operations on it together in one unit (encapsulation), and the data can be protected by making it `private`.

> **Example.** Putting `rollNo`, `name` and `marks` inside `Student` ensures that each student object carries its own details.

Q10. What is the difference between an object and an object reference?

2 Marks · CO1

The **object** is the actual entity in heap memory holding the data. The **reference** is a variable (on the stack) that holds the address of the object. One object may have many references, and a reference may be re-pointed to another object or to `null`, but the object remains until it is no longer reachable, when the garbage collector removes it.

> **Example.** In `Dog d = new Dog();` the right-hand side creates the object; `d` is only the reference. After `d = null;` the object still exists in the heap until garbage collected.

Q11. What happens when two references refer to the same object?

2 Marks · CO1

Both references give access to the same single copy of the state. A change made through either of them is seen through the other, and the object stays alive until **both** references stop pointing to it. This is called aliasing and is the cause of many subtle bugs if not understood.

> **Example.** `Employee e1 = new Employee(); Employee e2 = e1; e2.id = 202;` now `e1.id` is also 202.

Q12. Why is the main() method declared as static?

2 Marks · CO1

The JVM must call `main` before any object of the class exists. A static method belongs to the class and can be called without creating an object, so `main` is declared `static`. If it were an instance method, the JVM would need an object to call it, but there is no object until the program starts, which would be a circular problem.

> **Example.** The JVM effectively executes `Hello.main(args)` using the class name, not an object.

Q13. What is the purpose of the public keyword in the main() method?

2 Marks · CO1

`public` makes the method accessible from anywhere. The JVM, which is outside the class and outside its package, has to call `main`, and it can do so only if the method is public. With any other access level, the program compiles but at run time the JVM reports that it cannot find or access the `main` method.

Q14. What is the role of the JVM in executing a Java program?

2 Marks · CO1

The JVM is the run-time engine for Java. Its roles are:

- **Loading** the `.class` files with the class loader.
- **Verifying** the bytecode for safety and correctness.
- **Executing** the bytecode, by interpreting it or compiling hot parts to machine code with the JIT compiler.
- **Managing memory**, including heap, stack and automatic garbage collection.
- **Providing platform independence**, by hiding differences between operating systems.

> **Example.** `java Hello` starts the JVM, loads `Hello.class`, verifies it, finds `main` and executes it line by line.

Q15. Why is a class considered a blueprint for objects?

2 Marks · CO1

Just as a blueprint specifies the structure of a building without being a building, a class specifies the variables and methods that its objects will have without itself holding any object's data. Any number of objects can be built from one class, all with the same structure but possibly different values.

> **Example.** The class `Student { int roll; String name; }` is a blueprint; `new Student()` called twice builds two different student objects with their own `roll` and `name`.

Q16. Explain the significance of each token of "public static void main (String args[])".

5 Marks · CO1

This is the standard entry point of a Java program. Each token has a meaning:

Token	Meaning and significance
public	Access modifier. Makes the method visible to the JVM, which calls it from outside the class. If it is not public, the JVM cannot start the program.
static	The method belongs to the class and can be called without creating an object. The JVM needs this because no object exists at start-up.
void	Return type. `main` returns nothing to the JVM; the program ends when it finishes (use `System.exit(n)` for a status code).
main	The fixed name that the JVM looks for as the starting point. A different name will compile, but the JVM will not run it as the entry point.
String args[]	Parameter: an array of `String` that receives command-line arguments. `String[] args` is the equivalent, preferred form. The name `args` may be changed.

The parentheses enclose the parameter list and the braces enclose the body. The complete signature must match exactly for the JVM to recognise the method.

> **Example.** Running `java Greet Asha Ravi` gives `args[0] = "Asha"` and `args[1] = "Ravi"`; `args.length` is 2.

```
public class Greet {
    public static void main(String[] args) {
        System.out.println("Hello " + args[0]);
    }
}
```

Output

```
Hello Asha
```

Bloom's Level: Apply (L3)

Q1. Define a class Rectangle with length and breadth. Create an object and implement methods to calculate and display its area and perimeter.

5 Marks · CO2

Approach. The class holds `length` and `breadth` as data members. Method `area()` returns length x breadth and `perimeter()` returns 2(length + breadth). The `main` method creates the object and calls the methods.

```
class Rectangle {
    int length, breadth;

    int area()      { return length * breadth; }
    int perimeter() { return 2 * (length + breadth); }

    void display() {
        System.out.println("Length    = " + length);
        System.out.println("Breadth   = " + breadth);
        System.out.println("Area      = " + area());
        System.out.println("Perimeter = " + perimeter());
    }

    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = 8;
        r.breadth = 5;
        r.display();
    }
}
```

Output

```
Length    = 8
Breadth   = 5
Area      = 40
Perimeter = 26
```

> **Example.** For a 8 x 5 rectangle, area = 8 x 5 = 40 and perimeter = 2 x (8 + 5) = 26.

Q2. Define a class Employee with id, name and salary. Create two objects and display the details of the employee having the higher salary.

5 Marks · CO2

Approach. Create two Employee objects, compare their salaries with an `if` statement and display the one with the greater value.

```
class Employee {
    int id;
    String name;
    double salary;

    void display() {
        System.out.println("ID: " + id + ", Name: " + name
                           + ", Salary: " + salary);
    }
```

```
    public static void main(String[] args) {
        Employee e1 = new Employee();
        e1.id = 101; e1.name = "Amit";  e1.salary = 42000;
        Employee e2 = new Employee();
        e2.id = 102; e2.name = "Sneha"; e2.salary = 51000;

        System.out.println("Employee with higher salary:");
        if (e1.salary > e2.salary) e1.display();
        else                       e2.display();
    }
}
```

Output

```
Employee with higher salary:
ID: 102, Name: Sneha, Salary: 51000.0
```

Example. Salaries 42000 and 51000 are compared; 51000 is larger, so Sneha's details are printed.

Q3. Define a class Complex with real and imaginary parts. Create two objects and write a method to add them. Display the resulting complex number.

5 Marks · CO2

Approach. Addition of complex numbers is (a + bi) + (c + di) = (a + c) + (b + d)i. The method `add` takes another Complex object and returns a new one.

```
class Complex {
    double real, imag;

    Complex(double r, double i) { real = r; imag = i; }

    Complex add(Complex c) {
        return new Complex(real + c.real, imag + c.imag);
    }

    void display() {
        System.out.println(real + " + " + imag + "i");
    }

    public static void main(String[] args) {
        Complex c1 = new Complex(3, 4);
        Complex c2 = new Complex(1, 2);
        Complex c3 = c1.add(c2);
        c3.display();
    }
}
```

Output

```
4.0 + 6.0i
```

Example. (3 + 4i) + (1 + 2i) = (3 + 1) + (4 + 2)i = 4 + 6i.

Q4. Create a class Distance with feet and inches. Write methods to input, add two distance objects and display the result.

5 Marks · CO2

Approach. Since 12 inches make 1 foot, after adding the inches we carry any excess into the feet.

```
import java.util.Scanner;

class Distance {
    int feet, inches;

    void input(Scanner sc) {
        System.out.print("Enter feet and inches: ");
        feet = sc.nextInt();
        inches = sc.nextInt();
    }

    Distance add(Distance d) {
        Distance r = new Distance();
        r.inches = inches + d.inches;
        r.feet = feet + d.feet + r.inches / 12;   // carry
        r.inches = r.inches % 12;
        return r;
    }

    void display() { System.out.println(feet + " ft " + inches + " in"); }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Distance a = new Distance(), b = new Distance();
        a.input(sc);
        b.input(sc);
        a.add(b).display();
    }
}
```

Output

```
Enter feet and inches: 5 9
Enter feet and inches: 3 7
9 ft 4 in
```

> **Example.** 5 ft 9 in + 3 ft 7 in = 8 ft 16 in. Since 16 in = 1 ft 4 in, the result is 9 ft 4 in.

Q5. Define a class Time with hours and minutes. Create two objects and write a method to calculate and display their total duration.

5 Marks · CO2

Approach. Add hours and minutes separately; 60 minutes make one hour, so carry the excess minutes into hours.

```
class Time {
    int hours, minutes;

    Time(int h, int m) { hours = h; minutes = m; }

    Time add(Time t) {
        int m = minutes + t.minutes;
```

```
        int h = hours + t.hours + m / 60;
        return new Time(h, m % 60);
    }

    void display() { System.out.println(hours + " h " + minutes + " min"); }

    public static void main(String[] args) {
        Time t1 = new Time(2, 45);
        Time t2 = new Time(1, 30);
        System.out.print("Total duration: ");
        t1.add(t2).display();
    }
}
```

Output

```
Total duration: 4 h 15 min
```

> **Example.** 2 h 45 min + 1 h 30 min = 3 h 75 min = 4 h 15 min.

Q6. Create a class BankAccount with account number, holder name and balance. Implement deposit(), withdraw() and displayBalance(). Demonstrate the operations.

5 Marks · CO2

Approach. `deposit` adds to the balance; `withdraw` subtracts only if sufficient balance exists.

```
class BankAccount {
    int accNo;
    String holder;
    double balance;

    BankAccount(int n, String h, double b) { accNo = n; holder = h; balance = b; }

    void deposit(double amt) {
        balance += amt;
        System.out.println("Deposited " + amt);
    }

    void withdraw(double amt) {
        if (amt > balance) System.out.println("Insufficient balance");
        else { balance -= amt; System.out.println("Withdrawn " + amt); }
    }

    void displayBalance() {
        System.out.println(holder + " (A/c " + accNo + ") balance = " + balance);
    }

    public static void main(String[] args) {
        BankAccount a = new BankAccount(1001, "Riya", 5000);
        a.displayBalance();
        a.deposit(2500);
        a.withdraw(1200);
        a.withdraw(9000);
        a.displayBalance();
    }
}
```

Output

```
Riya (A/c 1001) balance = 5000.0
Deposited 2500.0
Withdrawn 1200.0
Insufficient balance
Riya (A/c 1001) balance = 6300.0
```

> **Example.** 5000 + 2500 - 1200 = 6300; the withdrawal of 9000 is refused as it exceeds the balance.

Q7. Define a class Triplet containing three integers. Write methods to accept three numbers and determine whether they form a Pythagorean triplet.

5 Marks · CO2

Approach. Three numbers form a Pythagorean triplet if the square of the largest equals the sum of the squares of the other two.

```
import java.util.Scanner;

class Triplet {
    int a, b, c;

    void accept(Scanner sc) {
        a = sc.nextInt(); b = sc.nextInt(); c = sc.nextInt();
    }

    boolean isPythagorean() {
        int max = Math.max(a, Math.max(b, c));
        int s1, s2;
        if (max == a)      { s1 = b; s2 = c; }
        else if (max == b) { s1 = a; s2 = c; }
        else               { s1 = a; s2 = b; }
        return max * max == s1 * s1 + s2 * s2;
    }

    public static void main(String[] args) {
        Triplet t = new Triplet();
        t.accept(new Scanner(System.in));
        System.out.println(t.isPythagorean() ? "Pythagorean triplet"
                                             : "Not a triplet");
    }
}
```

Output

```
(input 5 3 4)  ->  Pythagorean triplet
(input 2 3 4)  ->  Not a triplet
```

> **Example.** For 5, 3, 4: largest = 5, and 5 x 5 = 25 = 9 + 16. For 2, 3, 4: 16 is not equal to 4 + 9 = 13.

Q8. Create a class Book with bookId, title and price. Create three objects and write a method to display the details of the book having the highest price.

5 Marks · CO2

Approach. Keep the three books in an array and track the maximum price while scanning.

```
class Book {
    int bookId;
    String title;
    double price;

    Book(int id, String t, double p) { bookId = id; title = t; price = p; }

    void display() {
        System.out.println(bookId + " | " + title + " | Rs. " + price);
    }

    static void highest(Book[] b) {
        Book max = b[0];
        for (int i = 1; i < b.length; i++)
            if (b[i].price > max.price) max = b[i];
        System.out.println("Costliest book:");
        max.display();
    }

    public static void main(String[] args) {
        Book[] books = {
            new Book(1, "Java Basics", 450),
            new Book(2, "Data Structures", 620),
            new Book(3, "Networks", 580)
        };
        highest(books);
    }
}
```

Output

```
Costliest book:
2 | Data Structures | Rs. 620.0
```

Example. Prices 450, 620, 580: the maximum is 620, belonging to “Data Structures”.

Q9. Create two objects of a class Number and perform addition and subtraction of their data members using a member method.

5 Marks · CO2

Approach. A member method of `Number` receives the second object as an argument and combines it with the data of the calling object.

```
class Number {
    int value;

    Number(int v) { value = v; }

    int add(Number n)      { return value + n.value; }
    int subtract(Number n) { return value - n.value; }

    public static void main(String[] args) {
        Number x = new Number(50);
        Number y = new Number(20);
        System.out.println("Sum        = " + x.add(y));
        System.out.println("Difference = " + x.subtract(y));
    }
}
```

Output

```
Sum        = 70
Difference = 30
```

Example. With x = 50 and y = 20: x.add(y) = 50 + 20 = 70 and x.subtract(y) = 50 - 20 = 30. Calling `y.subtract(x)` would give -30, since the method uses the calling object as the first operand.

Q10. Define a class Product with product code, name and price. Create objects for three products and calculate and display the total price of all products.

5 Marks · CO2

```
class Product {
    int code;
    String name;
    double price;

    Product(int c, String n, double p) { code = c; name = n; price = p; }

    public static void main(String[] args) {
        Product p1 = new Product(101, "Pen", 10);
        Product p2 = new Product(102, "Notebook", 45.5);
        Product p3 = new Product(103, "Bag", 650);
        Product[] all = { p1, p2, p3 };

        double total = 0;
        for (Product p : all) {
            System.out.println(p.code + "  " + p.name + "  " + p.price);
            total += p.price;
        }
        System.out.println("Total price = " + total);
    }
}
```

Output

```
101  Pen  10.0
102  Notebook  45.5
103  Bag  650.0
Total price = 705.5
```

Example. 10 + 45.5 + 650 = 705.5.

Q11. Define a class Student with private rollNo, name and marks. Provide methods to accept and display the data. Create two objects and display the student with higher marks.

5 Marks · CO2

Approach. Private fields are set through an `accept` method (encapsulation); a getter lets `main` compare marks.

```
class Student {
    private int rollNo;
    private String name;
    private double marks;
```

```
    void accept(int r, String n, double m) { rollNo = r; name = n; marks = m; }
    double getMarks() { return marks; }

    void display() {
        System.out.println(rollNo + " " + name + " " + marks);
    }

    public static void main(String[] args) {
        Student s1 = new Student(), s2 = new Student();
        s1.accept(1, "Arjun", 78.5);
        s2.accept(2, "Meera", 86.0);
        System.out.println("Higher marks:");
        if (s1.getMarks() >= s2.getMarks()) s1.display(); else s2.display();
    }
}
```

Output

```
Higher marks:
2 Meera 86.0
```

> **Example.** 86.0 > 78.5, so Meera's record is displayed. Because the fields are private, outside code cannot write `s1.marks = -5`.

Q12. Create a class Employee with appropriate data members and methods. Write a Java program to calculate tax based on taxable income and display the employee details with the calculated tax.

5 Marks · CO2

Approach. A simple slab system is used: no tax up to 2,50,000; 5% on income between 2,50,000 and 5,00,000; 20% on income above 5,00,000 (plus the 5% slab amount); the slabs are illustrative and may be replaced with those given in the question.

```
class Employee {
    int id;
    String name;
    double income;

    Employee(int i, String n, double inc) { id = i; name = n; income = inc; }

    double calculateTax() {
        if (income <= 250000) return 0;
        if (income <= 500000) return (income - 250000) * 0.05;
        return 12500 + (income - 500000) * 0.20;
    }

    void display() {
        System.out.println("ID     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Income : " + income);
        System.out.println("Tax    : " + calculateTax());
    }

    public static void main(String[] args) {
        new Employee(7, "Kiran", 700000).display();
    }
}
```

Output

```
ID     : 7
Name   : Kiran
Income : 700000.0
Tax    : 52500.0
```

Example. Income 7,00,000: slab 2 tax = (5,00,000 - 2,50,000) x 5% = 12,500; slab 3 tax = (7,00,000 - 5,00,000) x 20% = 40,000. Total = 52,500.

Q13. Define a class ArrayOperation containing an integer array. Implement methods to calculate the maximum, minimum and average values. Create an object and demonstrate all operations.

5 Marks · CO2

```
class ArrayOperation {
    int[] a;

    ArrayOperation(int[] arr) { a = arr; }

    int max() {
        int m = a[0];
        for (int x : a) if (x > m) m = x;
        return m;
    }
    int min() {
        int m = a[0];
        for (int x : a) if (x < m) m = x;
        return m;
    }
    double average() {
        int sum = 0;
        for (int x : a) sum += x;
        return (double) sum / a.length;
    }

    public static void main(String[] args) {
        ArrayOperation op = new ArrayOperation(new int[]{12, 5, 30, 8, 20});
        System.out.println("Maximum = " + op.max());
        System.out.println("Minimum = " + op.min());
        System.out.println("Average = " + op.average());
    }
}
```

Output

```
Maximum = 30
Minimum = 5
Average = 15.0
```

Example. For {12, 5, 30, 8, 20}: sum = 75, count = 5, average = 75 / 5 = 15.0.

Q14. Create a class Result containing marks of three subjects. Implement methods to calculate total, average and grade. Create an object and display the complete result.

5 Marks · CO2

```
class Result {
    int m1, m2, m3;

    Result(int a, int b, int c) { m1 = a; m2 = b; m3 = c; }

    int total()      { return m1 + m2 + m3; }
    double average() { return total() / 3.0; }

    char grade() {
        double avg = average();
        if (avg >= 80) return 'A';
        else if (avg >= 60) return 'B';
        else if (avg >= 40) return 'C';
        else return 'F';
    }

    void display() {
        System.out.println("Marks   : " + m1 + ", " + m2 + ", " + m3);
        System.out.println("Total   : " + total());
        System.out.printf("Average : %.2f%n", average());
        System.out.println("Grade   : " + grade());
    }

    public static void main(String[] args) {
        new Result(78, 85, 69).display();
    }
}
```

Output

```
Marks   : 78, 85, 69
Total   : 232
Average : 77.33
Grade   : B
```

Example. Total = 78 + 85 + 69 = 232; average = 232 / 3 = 77.33, which lies in the 60 to 79 range, so the grade is B.

Q15. Define a class Mobile with brand, model and price. Create two objects and implement a method to compare their prices and display the details of the more expensive mobile.

5 Marks · CO2

```
class Mobile {
    String brand, model;
    double price;

    Mobile(String b, String m, double p) { brand = b; model = m; price = p; }

    Mobile costlier(Mobile other) {
        return (price >= other.price) ? this : other;
    }

    void display() {
        System.out.println(brand + " " + model + " - Rs. " + price);
    }

    public static void main(String[] args) {
        Mobile m1 = new Mobile("Samsung", "S23", 62000);
        Mobile m2 = new Mobile("Redmi", "Note 12", 18000);
        System.out.println("More expensive mobile:");
        m1.costlier(m2).display();
    }
}
```

Output

```
More expensive mobile:
Samsung S23 - Rs. 62000.0
```

> **Example.** Since 62000 > 18000, `costlier` returns `this` (the Samsung) and its details are printed. The method returns an object, not just a number.

Bloom's Level: Analyze (L4)

Q1. Analyze the program: Test t1 = new Test(); Test t2 = t1; t2.x = 25; System.out.println(t1.x); System.out.println(t2.x); Determine (a) output (b) number of objects (c) number of references (d) why both references give the same value.

5 Marks · CO3

Given in the question

```
class Test {
    int x = 10;
    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = t1;
        t2.x = 25;
        System.out.println(t1.x);
        System.out.println(t2.x);
    }
}
```

a) **Output:**

Output

```
25
25
```

b) **Number of objects:** one. Only `new Test()` creates an object; the statement `Test t2 = t1;` does not.

c) **Number of references:** two, `t1` and `t2`.

d) **Reason for equal values.** Assigning `t2 = t1` copies the reference (address), not the object. Both variables therefore point to the same heap object. The statement `t2.x = 25` changes the single copy of `x` held in that object from 10 to 25, and it is read back as 25 through either reference.

Step	State of memory
Test t1 = new Test()	t1 refers to object O (x = 10)
Test t2 = t1	t1 and t2 both refer to O
t2.x = 25	O.x becomes 25
Print t1.x, t2.x	Both read O.x, giving 25, 25

Q2. Analyze the class design: class Student { int roll; String name; display() { System.out.println(roll); System.out.println(name); } } Identify the errors, explain why they occur and provide the corrected class.

5 Marks · CO3

Given in the question

```
class Student {
    int roll;
    String name;
    display() {
        System.out.println(roll);
        System.out.println(name);
    }
}
```

Errors and explanation.

- **Missing return type (compile-time error).** In Java every method must declare a return type. `display()` has none, so the compiler treats it as a constructor-like declaration and reports “invalid method declaration; return type required”. (A constructor must have the same name as the class, which `display` does not.) Since the method prints and returns nothing, the return type should be `void`.
- **Design problem 1: public-by-default fields.** `roll` and `name` have default (package) access, so any class in the package can modify them without validation, which breaks encapsulation. They should be `private`.
- **Design problem 2: no way to set the data.** With private fields we need a constructor or setters to initialise them.
- **Design problem 3: poor output.** Two separate prints without labels are hard to read.

Corrected class:

```
class Student {
    private int roll;
    private String name;

    Student(int roll, String name) {
        this.roll = roll;
        this.name = name;
    }

    void display() {
        System.out.println("Roll : " + roll);
        System.out.println("Name : " + name);
    }

    public static void main(String[] args) {
        new Student(12, "Neha").display();
    }
}
```

Output

```
Roll : 12
Name : Neha
```

Q3. Consider the code with class Number { int value; void change(Number n) { n.value = 50; } ... n1.value = 10; n1.change(n1); println(n1.value) }. Determine the output and explain how the object reference passed to the method affects the object's data.

5 Marks · CO3

Given in the question

```
class Number {
    int value;
    void change(Number n) { n.value = 50; }
    public static void main(String[] args) {
        Number n1 = new Number();
        n1.value = 10;
        n1.change(n1);
        System.out.println(n1.value);
    }
}
```

Output:

Output

```
50
```

Explanation. Java passes arguments **by value**, but the value of an object variable is a reference. When `n1.change(n1)` is called, a copy of the reference `n1` is made and stored in the parameter `n`. Now `n1` (in `main`) and `n` (in `change`) both refer to the same object. The statement `n.value = 50` follows the copied reference to the real object and modifies its field. When the method ends, the parameter `n` is discarded, but the change to the object remains. Hence `n1.value` is 50, not 10.

Example. By contrast, if the method had said `n = new Number(); n.value = 50;`, the parameter would be re-pointed to a **new** object, the original object would be untouched, and the output would remain 10. This shows that a method can change the **state** of the object it receives but cannot change which object the caller's variable refers to.

Q4. Two objects of class Employee { int id; String name; } are created: e1 and e2. e1.id = 101; e1.name = "Amit"; e2 = e1; e2.id = 202; Analyze the state of e1 and e2: number of objects, references and final values.

5 Marks · CO3

Given in the question

```
Employee e1 = new Employee();
Employee e2 = new Employee();
e1.id = 101;
e1.name = "Amit";
e2 = e1;
e2.id = 202;
```

Step-by-step analysis.

Statement	Objects in heap	What the references hold
Employee e1 = new Employee();	O1	e1 refers to O1
Employee e2 = new Employee();	O1, O2	e2 refers to O2
e1.id = 101; e1.name = "Amit";	O1 = (101, "Amit")	-
e2 = e1;	O1, O2 (O2 now unreachable)	e1 and e2 both refer to O1
e2.id = 202;	O1 = (202, "Amit")	-

Results.

- **Objects created:** two, O1 and O2. After `e2 = e1` the object O2 is no longer referred to by anyone, so it becomes eligible for garbage collection. Effectively one live object remains.
- **References:** two, `e1` and `e2`, both pointing to O1.
- **Final values:** `e1.id = 202`, `e1.name = "Amit"`; `e2.id = 202`, `e2.name = "Amit"`. The original value 101 has been overwritten, because the single object O1 is shared.

Q5. Analyze the Rectangle class design (length, breadth, area(), perimeter()). Two objects are created with different dimensions. Explain how the class, objects, instance variables and methods work together, and whether length and breadth are shared.

5 Marks · CO3

Given in the question

```
class Rectangle {
    int length, breadth;
    int area()      { return length * breadth; }
    int perimeter() { return 2 * (length + breadth); }
}
```

How the parts work together.

- The **class** `Rectangle` is the blueprint: it declares two instance variables and two methods.
- Each **object** created with `new Rectangle()` receives its own separate copy of `length` and `breadth` in the heap; this is its state.
- The **methods** are defined once in the class and shared by all objects (there is one copy of the code), but whenever a method is called through an object, it works on that object's own `length` and `breadth` (internally through `this`).
- The **instance variables** are therefore not shared; the **method code** is.

Are length and breadth shared? No. They are instance variables, not `static`. Changing one object's values leaves the other untouched.

> **Example.** `Rectangle r1 = new Rectangle(); r1.length = 4; r1.breadth = 3; Rectangle r2 = new Rectangle(); r2.length = 10; r2.breadth = 2;` Then `r1.area()` = 12 and `r2.area()` = 20, `r1.perimeter()` = 14 and `r2.perimeter()` = 24. The same method gives different results because it reads different objects' data. If `length` had been declared `static`, both objects would share one value and the two areas could not differ in this way.

MODULE

3

Constructors, Overloading and Passing Objects

Constructors, this, this(), method overloading, passing and returning objects

Bloom's Level: Remember (L1)

Q1. What is a constructor?

2 Marks · CO1

A constructor is a special member of a class that is called automatically when an object is created with `new`, and whose job is to initialise the instance variables of the new object. It has the same name as the class and has no return type.

> **Example.** `class Student { int roll; Student(int r) { roll = r; } }` and `new Student(5)` creates a Student whose `roll` is 5.

Q2. What is constructor overloading?

2 Marks · CO1

Constructor overloading means defining more than one constructor in the same class, each with a different parameter list (different number, types or order of parameters). The compiler chooses the constructor that matches the arguments given in `new`.

> **Example.** `Box()` creates a unit box, `Box(int s)` a cube of side s, and `Box(int l, int b, int h)` a cuboid; all three are constructors of class `Box`.

Q3. What is method overloading?

2 Marks · CO1

Method overloading means having several methods in a class with the same name but different parameter lists. The return type alone cannot distinguish them. The call is resolved at compile time on the basis of the arguments.

> **Example.** `int add(int a, int b)` and `double add(double a, double b)` are overloaded versions of `add`.

Q4. What is the this keyword?

2 Marks · CO1

`this` is a reference variable available inside instance methods and constructors that refers to the **current object**, that is, the object on which the method or constructor is being executed. It is used to separate instance variables from parameters having the same name, to pass the current object to another method, and to call another constructor.

> **Example.** In `Student(int roll) { this.roll = roll; }`, `this.roll` is the instance variable and `roll` is the parameter.

Q5. What is this()?

2 Marks · CO1

`this()` is a call to another constructor of the same class. The arguments in the parentheses decide which constructor is called. It must be the first statement in a constructor and is used to reuse initialisation code.

> **Example.** In `Demo() { this(10); }`, creating `new Demo()` first runs `Demo(int x)` with x = 10.

Q6. What is constructor chaining?

2 Marks · CO1

Constructor chaining is the technique of calling one constructor from another in the same class using `this()` (or the parent-class constructor using `super()`), so that a series of constructors execute one after another and common initialisation is written only once.

> **Example.** `Point() { this(0, 0); }` calls `Point(int x, int y)`, so the default point is the origin.

Q7. Can a method return an object?

2 Marks · CO1

Yes. The return type of a method may be a class type, and the method then returns a reference to an object, either a new one or an existing one. This is common for operations such as adding two complex numbers.

> **Example.** `Complex add(Complex c) { return new Complex(re + c.re, im + c.im); }` returns a new Complex object.

Q8. Can an object be passed as a method argument?

2 Marks · CO1

Yes. A parameter can be of any class type, and the caller passes an object (more precisely, a copy of its reference). The method can then read and modify the state of that object.

> **Example.** `void show(Student s) { System.out.println(s.roll); }` is called as `show(st);` where `st` is a Student object.

Q9. What is compile-time polymorphism?

2 Marks · CO1

Compile-time (static) polymorphism is the ability of one name to have several forms, with the actual form being decided by the compiler at compile time. In Java it is achieved by **method overloading** (and constructor overloading).

> **Example.** Calls `add(2, 3)` and `add(2.5, 3.5)` invoke different `add` methods, chosen at compile time from the argument types.

Q10. What is an object reference?

2 Marks · CO1

An object reference is a variable that stores the address of an object in heap memory. Operations on the object are carried out through this reference. Two references may refer to the same object.

> **Example.** `Student s = new Student();` here `s` is the reference to the new Student object.

Bloom's Level: Understand (L2)

Q1. Differentiate between this and this().

3 Marks · CO1

Basis	this	this()
What it is	A reference to the current object	A call to another constructor of the same class
Used for	Accessing fields/methods of the current object, avoiding name clashes	Reusing another constructor (constructor chaining)
Where used	Any instance method or constructor	Only inside a constructor, as its first statement
Arguments	None; it is not a method call	May take arguments to select a constructor
Example	`this.roll = roll;`	`this(101, "Rahul");`

> **Example.** In `Student() { this(101, "Rahul"); }` the `this(...)` call invokes the two-argument constructor, while inside that constructor `this.roll = roll;` uses the reference form.

Q2. Why is this() required to be the first statement of a constructor?

2 Marks · CO1

Before a constructor does anything else, the object must be initialised in a well-defined order: the constructor that is called by `this()` completes first, so that the instance variables are set before any other statement of the calling constructor uses them. The Java compiler therefore insists that `this()` (or `super()`) be the first statement, so that initialisation happens exactly once and in a predictable sequence. Placing it elsewhere gives the compile-time error "call to this must be first statement in constructor".

> **Example.** `Demo() { System.out.println("x"); this(5); }` is rejected by the compiler; `Demo() { this(5); System.out.println("x"); }` is accepted.

Q3. What is the difference between a constructor and a method?

3 Marks · CO1

Basis	Constructor	Method
Purpose	Initialises a newly created object	Performs an operation on an object
Name	Same as the class name	Any valid identifier
Return type	None (not even void)	Must have a return type or void
Call	Called automatically by `new`	Called explicitly using an object or class name
Inheritance	Not inherited	Inherited by subclasses

Basis	Constructor	Method
Default	Compiler supplies a default constructor if none is written	No default method is supplied

> **Example.** `Student(int r)` is a constructor; `void display()` is a method of the same class.

Q4. Can constructors be overloaded? Explain.

2 Marks · CO1

Yes. A class may declare many constructors provided that their parameter lists differ in number, type or order of parameters. This allows objects to be created and initialised in different ways, and the compiler selects the proper constructor from the arguments in the `new` expression.

> **Example.** `Rectangle()` makes a 1 x 1 rectangle, `Rectangle(int s)` a square, and `Rectangle(int l, int b)` a general rectangle. `new Rectangle(4)` runs the second.

Q5. Can a constructor be inherited?

2 Marks · CO1

No. Constructors are not members that are inherited by a subclass, because a constructor has the name of its own class. A subclass has its own constructors, but each of them begins by calling a constructor of the superclass (using `super()`, implicitly or explicitly) to initialise the inherited part of the object.

> **Example.** If `class B extends A` and `A` has `A(int x)`, then `new B(5)` is not valid unless `B` declares `B(int)` itself, which may call `super(x)`.

Q6. Why is method overloading called compile-time polymorphism?

3 Marks · CO1

Polymorphism means “many forms”. In overloading, one method name has several forms (versions). Which version is to be executed is decided by the **compiler**, during compilation, by comparing the number, types and order of the arguments in the call with the parameter lists of the available methods. Since the binding of the call to the method body is done before the program runs, it is also called static or early binding, and hence the name compile-time polymorphism.

> **Example.** For `void show(int a)` and `void show(String s)`, the call `show(10)` is bound to the first and `show("Hi")` to the second by the compiler itself; no run-time decision is needed.

Q7. Can two methods be overloaded by changing only their return types?

2 Marks · CO1

No. The compiler uses the method name and the parameter list to identify a method (together called the signature); the return type is not part of the signature. Two methods with the same name and parameters but different return types are a duplicate declaration, and the compiler reports an error.

> **Example.** `int f(int a)` and `double f(int a)` in one class give the error “method f(int) is already defined”.

Q8. How is an object passed to a method?

2 Marks · CO1

An object is passed by supplying a variable of that class type as the argument. Java passes a **copy of the reference** (pass by value of the reference), so the parameter and the argument refer to the same object. The method can therefore change the state of the object, but re-assigning the parameter inside the method does not change the caller's variable.

> **Example.** `static void raise(Employee e) { e.salary += 1000; }` called as `raise(emp);` increases `emp.salary`.

Q9. How does a method return an object?

2 Marks · CO1

The method declares the class as its return type and ends with `return` followed by a reference to an object, which may be a newly created object or an existing one. The caller stores the returned reference in a variable of that class type.

> **Example.** `Time add(Time t) { return new Time(h + t.h, m + t.m); }` is used as `Time total = t1.add(t2);`.

Q10. What is the role of this in Complex addComplex(Complex c)?

3 Marks · CO1

Inside `addComplex`, `this` refers to the object that was used to call the method, while `c` refers to the object passed in the argument. Thus the method works with two objects: `this.real` and `c.real` are the real parts of the two numbers being added. `this` also lets the method distinguish the caller's fields from the parameter's when names are alike.

> **Example.** For `Complex c3 = c1.addComplex(c2);` inside the method `this` is `c1`, so `this.real + c.real` evaluates as `c1.real + c2.real`.

Q11. Explain: Complex c3 = c1.addComplex(c2);

2 Marks · CO1

The statement calls the method `addComplex` on the object `c1`, passing the object `c2` as argument. The method adds the real and imaginary parts of `c1` and `c2` and returns a **new** Complex object holding the sum. The returned reference is stored in `c3`. The objects `c1` and `c2` are unchanged.

> **Example.** If c1 = 2 + 3i and c2 = 4 + 5i, then c3 = 6 + 8i.

Q12. What is the difference between passing a primitive value and passing an object?

3 Marks · CO1

When a **primitive** (int, double, char, ...) is passed, the actual value is copied into the parameter, so the method works on an independent copy and cannot alter the caller's variable. When an **object** is passed, the copy is of the reference, so both the argument and the parameter refer to the same object, and any change made to the object's fields inside the method is visible to the caller.

> **Example.** The method below changes a copy of the primitive but the original array.

```
static void f(int x, int[] a) { x = 99; a[0] = 99; }
// int p = 1; int[] q = {1};  f(p, q);
// p is still 1, but q[0] is now 99
```

Q13. Why would a method return an object?

2 Marks · CO1

A method returns an object when the result is a compound value that cannot be expressed by a single primitive, such as a complex number, a time or a distance made up of two or more fields. It also lets us build new objects from existing ones without changing the originals, and allows calls to be chained.

> **Example.** `Distance d3 = d1.add(d2);` returns both feet and inches together in one Distance object.

Q14. Difference between constructor overloading and method overloading.

3 Marks · CO1

Basis	Constructor Overloading	Method Overloading
Name	All have the class name	All have the same method name
Return type	None	May be different, but cannot alone distinguish methods
Purpose	Different ways of initialising an object	Different ways of performing one operation
Invocation	Implicitly at `new`	Explicitly by a method call
Chaining	Can call each other using `this()`	Call each other by name
Inheritance	Not inherited	Inherited (and can be overloaded in the subclass)

> **Example.** `Student()` and `Student(int r)` are overloaded constructors; `print(int)` and `print(String)` are overloaded methods.

Q15. Explain constructor chaining with an example.

3 Marks · CO1

Constructor chaining is the process of one constructor calling another constructor of the same class using `this(...)`, so that common initialisation code lies in one constructor and the others merely supply default values. It avoids duplication and keeps all constructors consistent. When an object is created, the chain executes in order: the constructor reached last by `this()` finishes first, then control returns to the earlier constructors.

```
class Student {
    int roll; String name;
    Student()                 { this(101, "Unknown"); }
    Student(int r)            { this(r, "Unknown"); }
    Student(int r, String n)  { roll = r; name = n; }

    public static void main(String[] args) {
        Student s = new Student();
```

```
        System.out.println(s.roll + " " + s.name);
    }
}
```

Output

```
101 Unknown
```

> **Example.** `new Student()` calls `Student()`, which calls `Student(int,String)` with (101, "Unknown"); that constructor sets the fields and the control returns.

Bloom's Level: Apply (L3)

Q1. Write a Java program to define a class Student having rollNo, name and marks. Implement a parameterized constructor to initialize the objects and display the student details.

5 Marks · CO2

Approach. The parameterized constructor receives the three values and assigns them to the fields, so the object is fully initialised at the moment of creation.

```
class Student {
    int rollNo;
    String name;
    double marks;

    Student(int rollNo, String name, double marks) {
        this.rollNo = rollNo;
        this.name = name;
        this.marks = marks;
    }

    void display() {
        System.out.println("Roll No : " + rollNo);
        System.out.println("Name    : " + name);
        System.out.println("Marks   : " + marks);
    }

    public static void main(String[] args) {
        Student s1 = new Student(1, "Anita", 88.5);
        Student s2 = new Student(2, "Rohan", 72.0);
        s1.display();
        s2.display();
    }
}
```

Output

```
Roll No : 1
Name    : Anita
Marks   : 88.5
Roll No : 2
Name    : Rohan
Marks   : 72.0
```

Example. `new Student(1, "Anita", 88.5)` immediately gives the object its details; no separate assignment statements are needed, and no object can exist in an uninitialised state.

Q2. Define a class Rectangle with length and breadth. Implement constructor overloading using a default and a parameterized constructor. Create objects using both and display their area.

5 Marks · CO2

```
class Rectangle {
    int length, breadth;

    Rectangle() {                       // default constructor
        length = 1;
        breadth = 1;
    }

    Rectangle(int l, int b) {           // parameterized constructor
        length = l;
        breadth = b;
    }

    int area() { return length * breadth; }

    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        Rectangle r2 = new Rectangle(6, 4);
        System.out.println("Area of r1 = " + r1.area());
        System.out.println("Area of r2 = " + r2.area());
    }
}
```

Output

```
Area of r1 = 1
Area of r2 = 24
```

Example. `new Rectangle()` matches the no-argument constructor, giving 1 x 1 = 1. `new Rectangle(6, 4)` matches the two-int constructor, giving 6 x 4 = 24. The compiler chooses by the number of arguments.

Q3. Write a Java program to demonstrate the use of the this keyword when local variables and instance variables have the same name.

5 Marks · CO2

Approach. When a parameter has the same name as an instance variable, the parameter **shadows** the field inside the constructor or method. `this.name` is then used to refer explicitly to the instance variable.

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;      // field = parameter
        this.age = age;
```

```
    }

    void setName(String name) {
        name = name;           // WRONG: assigns parameter to itself
    }
    void setNameCorrect(String name) {
        this.name = name;      // correct
    }

    public static void main(String[] args) {
        Person p = new Person("Asha", 20);
        p.setName("Ravi");
        System.out.println(p.name);        // still Asha
        p.setNameCorrect("Ravi");
        System.out.println(p.name);        // Ravi
    }
}
```

Output

```
Asha
Ravi
```

Example. In `setName`, `name = name;` assigns the parameter to itself and the field never changes, so "Asha" is printed. In `setNameCorrect`, `this.name = name;` stores the parameter in the object's field, so "Ravi" is printed. Without `this`, the instance variable is hidden.

Q4. Define a class Student with overloaded constructors. Use this() to invoke one constructor from another. Demonstrate the execution using suitable objects.

5 Marks · CO2

```
class Student {
    int roll; String name; String course;

    Student() {
        this(0, "Unknown");
        System.out.println("Student() executed");
    }
    Student(int roll, String name) {
        this(roll, name, "B.Tech");
        System.out.println("Student(int,String) executed");
    }
    Student(int roll, String name, String course) {
        this.roll = roll; this.name = name; this.course = course;
        System.out.println("Student(int,String,String) executed");
    }

    void display() { System.out.println(roll + " " + name + " " + course); }

    public static void main(String[] args) {
        Student a = new Student();
        a.display();
        Student b = new Student(5, "Dev");
        b.display();
    }
}
```

Output

```
Student(int,String,String) executed
Student(int,String) executed
Student() executed
0 Unknown B.Tech
Student(int,String,String) executed
Student(int,String) executed
5 Dev B.Tech
```

Example. For `new Student()`: control goes to `Student()`, which immediately calls `this(0, "Unknown")`; that calls `this(roll, name, "B.Tech")`. The three-argument constructor does the real work first and prints its message; then control returns to the two-argument constructor, then to the no-argument one. Messages therefore appear in reverse order of the calls.

Q5. Define a class Calculator with overloaded add() methods to add two integers, three integers and two floating-point numbers. Demonstrate method overloading.

5 Marks · CO2

```
class Calculator {
    int add(int a, int b)            { return a + b; }
    int add(int a, int b, int c)     { return a + b + c; }
    double add(double a, double b)   { return a + b; }

    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
        System.out.println(c.add(2.5, 3.5));
    }
}
```

Output

```
30
60
6.0
```

Example. The call `add(10, 20)` has two int arguments and matches the first method; `add(10, 20, 30)` has three and matches the second; `add(2.5, 3.5)` has doubles and matches the third. All three have the same name but different parameter lists, and the compiler resolves each call at compile time (compile-time polymorphism).

Q6. Define a class Complex. Create a method that accepts two Complex objects as parameters and returns a Complex object containing their sum.

5 Marks · CO2

```
class Complex {
    double re, im;

    Complex(double re, double im) { this.re = re; this.im = im; }

    static Complex sum(Complex a, Complex b) {
        return new Complex(a.re + b.re, a.im + b.im);
    }

    void show() { System.out.println(re + " + " + im + "i"); }

    public static void main(String[] args) {
        Complex x = new Complex(2, 3);
        Complex y = new Complex(5, -1);
        Complex z = Complex.sum(x, y);
        z.show();
    }
}
```

Output

```
7.0 + 2.0i
```

> **Example.** (2 + 3i) + (5 - 1i) = (2 + 5) + (3 - 1)i = 7 + 2i. The method takes two objects, builds a third, and returns it; x and y remain unchanged.

Q7. Define a class Distance with feet and inches. Create a method add(Distance d) that accepts another Distance object and returns a Distance object containing the sum.

5 Marks · CO2

```
class Distance {
    int feet, inches;

    Distance(int f, int i) { feet = f; inches = i; }

    Distance add(Distance d) {
        int totalInches = inches + d.inches;
        int totalFeet = feet + d.feet + totalInches / 12;
        return new Distance(totalFeet, totalInches % 12);
    }

    void show() { System.out.println(feet + "' " + inches + "\""); }

    public static void main(String[] args) {
        Distance d1 = new Distance(4, 8);
        Distance d2 = new Distance(2, 7);
        d1.add(d2).show();
    }
}
```

Output

```
7' 3"
```

Example. 4 ft 8 in + 2 ft 7 in = 6 ft 15 in. As 15 in = 1 ft 3 in, the result is 7 ft 3 in. The carry is obtained from integer division `15 / 12 = 1`, and the remainder `15 % 12 = 3`.

Q8. Define a class Time with hours and minutes. Implement a method that accepts two Time objects and returns a new Time object representing their total duration.

5 Marks · CO2

```
class Time {
    int hours, minutes;

    Time(int h, int m) { hours = h; minutes = m; }

    static Time total(Time a, Time b) {
        int m = a.minutes + b.minutes;
        int h = a.hours + b.hours + m / 60;
        return new Time(h, m % 60);
    }

    void show() { System.out.println(hours + ":" + String.format("%02d", minutes)); }

    public static void main(String[] args) {
        Time t1 = new Time(1, 50);
        Time t2 = new Time(0, 25);
        Time.total(t1, t2).show();
    }
}
```

Output

```
2:15
```

Example. 1 h 50 min + 0 h 25 min = 1 h 75 min = 2 h 15 min. A new object is returned and the two inputs are not changed.

Q9. Write a Java program to demonstrate passing an object to a method. Define a class Number and implement a method that accepts a Number object and modifies its data member.

5 Marks · CO2

```
class Number {
    int value;

    Number(int v) { value = v; }

    void doubleIt(Number n) {          // object passed as argument
        n.value = n.value * 2;
    }

    public static void main(String[] args) {
        Number a = new Number(15);
        System.out.println("Before: " + a.value);
        a.doubleIt(a);                  // the same object is passed
        System.out.println("After : " + a.value);

        Number b = new Number(4);
        a.doubleIt(b);                  // a different object is passed
        System.out.println("b     : " + b.value);
    }
}
```

Output

```
Before: 15
After : 30
b     : 8
```

Example. When `a.doubleIt(b)` is called, the parameter `n` receives a copy of the reference `b`, so `n.value = n.value * 2` modifies the object that `b` refers to, changing 4 to 8. The method modifies the argument object, not the calling object `a`.

Q10. Define a class Box with length, breadth and height. Implement a method compare(Box b) that accepts another Box object and returns the Box having the greater volume.

5 Marks · CO2

```
class Box {
    double length, breadth, height;

    Box(double l, double b, double h) { length = l; breadth = b; height = h; }

    double volume() { return length * breadth * height; }

    Box compare(Box b) {
        return (this.volume() >= b.volume()) ? this : b;
    }

    public static void main(String[] args) {
        Box b1 = new Box(2, 3, 4);
        Box b2 = new Box(3, 3, 3);
        Box big = b1.compare(b2);
        System.out.println("Greater volume = " + big.volume());
    }
}
```

Output

```
Greater volume = 27.0
```

> **Example.** Volume of b1 = 2 x 3 x 4 = 24; volume of b2 = 3 x 3 x 3 = 27. Since 24 < 27, `compare` returns b, i.e. the box b2 itself (a reference to an existing object, not a copy).

Q11. Create a class Employee with id, name and salary. Implement two constructors using constructor overloading and a method to display employee details.

5 Marks · CO2

```
class Employee {
    int id;
    String name;
    double salary;

    Employee(int id, String name) {            // constructor 1
        this.id = id;
        this.name = name;
        this.salary = 10000;                   // default salary
    }

    Employee(int id, String name, double salary) {   // constructor 2
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println(id + " | " + name + " | " + salary);
    }

    public static void main(String[] args) {
        new Employee(1, "Pooja").display();
        new Employee(2, "Vikram", 45000).display();
    }
}
```

Output

```
1 | Pooja | 10000.0
2 | Vikram | 45000.0
```

> **Example.** `new Employee(1, "Pooja")` has two arguments, so constructor 1 runs and the default salary of 10000 is used. `new Employee(2, "Vikram", 45000)` has three arguments, so constructor 2 runs with the given salary.

Q12. Define a class Number with an integer data member. Implement add(Number n) that returns a new Number containing the sum of the current object and the object passed.

5 Marks · CO2

```
class Number {
    int value;

    Number(int value) { this.value = value; }

    Number add(Number n) {
        return new Number(this.value + n.value);
    }

    public static void main(String[] args) {
        Number n1 = new Number(12);
        Number n2 = new Number(30);
        Number n3 = n1.add(n2);
        System.out.println(n1.value + " " + n2.value + " " + n3.value);
    }
}
```

Output

```
12 30 42
```

> **Example.** `n1.add(n2)` computes 12 + 30 = 42 and creates a third object, which is returned and stored in `n3`. The originals keep their values 12 and 30.

Bloom's Level: Analyze (L4)

Q1. Analyze the program (class Test with Test(), Test(int), Test(double)) and determine the output. Explain the order in which the overloaded constructors are invoked.

3 Marks · CO3

Given in the question

```
class Test {
    Test()          { System.out.println("Default"); }
    Test(int x)     { System.out.println("Integer"); }
    Test(double x)  { System.out.println("Double"); }
    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test(10);
        Test t3 = new Test(10.5);
    }
}
```

Output:

Output

```
Default
Integer
Double
```

Explanation. The constructors are not called in the order of declaration but in the order of the `new` statements, and each call selects the constructor whose parameters match the argument. `new Test()` has no argument and matches `Test()`. `new Test(10)` has an `int` argument, which exactly matches

`Test(int)` (preferred over widening to double). `new Test(10.5)` has a `double` argument and matches `Test(double)`; an int constructor cannot accept it because a double cannot be narrowed to int automatically.

Q2. Analyze the program (class Student with Student(int roll) { this.roll = roll; } and display()) and determine the output. Explain how this distinguishes the instance variable from the parameter.

3 Marks · CO3

Given in the question

```
class Student {
    int roll;
    Student(int roll) { this.roll = roll; }
    void display() { System.out.println(roll); }
}
```

The code as printed has no `main`; if `new Student(7).display();` is executed, the output is

Output

```
7
```

Explanation. In the constructor, the parameter `roll` has the same name as the instance variable `roll`, so inside the constructor the simple name `roll` means the **parameter** (it shadows the field). `this.roll` explicitly means the field of the current object. The statement `this.roll = roll;` therefore copies the parameter's value into the field. Without `this`, `roll = roll;` would copy the parameter onto itself and the field would stay 0. In `display()`, there is no parameter called `roll`, so `roll` means the field, and 7 is printed.

Q3. Analyze the constructor chain (Demo() { this(10); println("C1"); } Demo(int x) { println("C2"); }) and determine the order of execution and output. Explain how this() transfers control.

3 Marks · CO3

Given in the question

```
class Demo {
    Demo()        { this(10); System.out.println("C1"); }
    Demo(int x)   { System.out.println("C2"); }
}
```

For `new Demo();` the output is

Output

```
C2
C1
```

Explanation. `new Demo()` starts the no-argument constructor. Its first statement `this(10)` transfers control to `Demo(int x)`, so that nothing else in `Demo()` runs yet. `Demo(int x)` prints "C2" and returns. Control then comes back to `Demo()` at the statement after `this(10)`, which prints "C1". Hence the called constructor finishes before the remaining statements of the caller.

Q4. Analyze the overloaded methods display(int x), display(double x), display(int x, int y) and determine which is invoked for display(10); display(10.5); display(10,20). Justify.

3 Marks · CO3

- `display(10)` invokes **display(int x)**. The literal 10 is an int and matches exactly; this is preferred over widening to double.
- `display(10.5)` invokes **display(double x)**. The literal 10.5 is a double, and an int method cannot accept a double without an explicit cast.
- `display(10, 20)` invokes **display(int x, int y)**, the only method with two parameters.

The compiler uses these rules in order: (1) exact match, (2) widening conversion (for example int to double), (3) boxing, (4) varargs. It decides by the number of arguments first and then by their types.

> **Example.** If only `display(double x)` and `display(int x, int y)` existed, `display(10)` would call `display(double)` because int is widened to double automatically.

Q5. Class Number { int value; Number(int value){...} void change(Number n){ n.value = 50; } }. If n1.value = 10 and n1.change(n1) is executed, analyze the final value of n1.value. Explain why passing the object affects the original object's state.

3 Marks · CO3

The final value of `n1.value` is **50**.

When `n1.change(n1)` is executed, Java copies the reference stored in `n1` into the parameter `n`. Both `n` and `n1` now refer to the single heap object. The assignment `n.value = 50` follows the reference and changes the field of this one object. When `change()` returns, the parameter `n` disappears, but the object has already been changed, and it is still referred to by `n1`. Objects are therefore changed "in place" when passed to methods, because only the reference is copied, not the object.

> **Example.** Before: n1 -> [value = 10]. During the call: n1 and n both -> [value = 10], then [value = 50]. After: n1 -> [value = 50].

Q6. Analyze Number modify(Number n) { n.value = n.value + 10; return n; } and determine the final state of the objects. Explain what happens to the object passed and the object returned.

3 Marks · CO3

Suppose `Number a = new Number(5); Number b = a.modify(a);` or, in general, `Number b = obj.modify(x);` for any object `x`.

Analysis. The method does not create a new object. It increases the `value` of the object passed in by 10 and then returns a reference to that **same** object. Therefore:

- the object passed is modified (its state changes from v to v + 10);
- the returned reference points to the very same object, not a copy;
- the caller's variable and the returned variable are two references to one object.

> **Example.** With `Number x = new Number(5); Number y = modify(x);` both `x.value` and `y.value` are 15, and `x == y` is true. If the method had returned `new Number(n.value + 10)`, a different object would be returned and `x.value` would stay 5.

Q7. A class Complex has a method add(Complex c) that returns a new Complex object. Analyze how the method performs addition and explain why returning an object is useful.

3 Marks · CO3

How it works. Inside `add`, `this` is the calling object and `c` is the argument. The method forms `this.real + c.real` and `this.imag + c.imag` and creates a **new** Complex object with these two results, which it returns. Neither operand is changed.

Why returning an object is useful.

- A complex number has two components, and a method can return only one value; wrapping them in an object returns both together.
- The originals are preserved, as mathematical addition does not change its operands.
- Calls can be chained, for example `a.add(b).add(c)`.

> **Example.** For `c1 = 1 + 2i` and `c2 = 3 + 4i`: `c1.add(c2)` creates the object (4 + 6i). `c1.add(c2).add(c1)` gives (5 + 8i).

Q8. Analyze a class containing both overloaded constructors and overloaded methods. For a given set of object creations and method calls, determine which constructor and method execute and justify based on parameter matching.

5 Marks · CO3

Consider the following class:

```
class Account {
    double balance; String type;
    Account()                       { balance = 0;  type = "Savings"; }   // C1
    Account(double b)               { balance = b;  type = "Savings"; }   // C2
    Account(double b, String t)     { balance = b;  type = t; }           // C3

    void deposit(double amt)        { balance += amt; }                   // M1
    void deposit(double amt, int n) { balance += amt * n; }               // M2
    void deposit(int amt)           { balance += amt + 1; }               // M3

    public static void main(String[] args) {
        Account a = new Account();                  // C1
        Account b = new Account(500);               // C2
        Account c = new Account(800.0, "Current");  // C3
        a.deposit(100);                             // M3
        b.deposit(100.0);                           // M1
        c.deposit(50.0, 3);                         // M2
    }
}
```

Analysis of the resolution.

Statement	Selected	Reason
new Account()	C1	No arguments; only C1 has an empty parameter list
new Account(500)	C2	One argument; int 500 widens to double
new Account(800.0, “Current”)	C3	Two arguments of types (double, String)

Statement	Selected	Reason
a.deposit(100)	M3	int literal; exact match with deposit(int) is preferred over widening to double
b.deposit(100.0)	M1	double literal; deposit(int) cannot take a double
c.deposit(50.0, 3)	M2	Two arguments (double, int)

The compiler chooses by number of arguments first, then by exact type match, then by widening. The same rules apply to constructors and methods.

Q9. Consider class Number { int value; Number(int value); Number add(Number n) { Number temp = new Number(this.value + n.value); return temp; } } with n1 = Number(10), n2 = Number(20), n3 = n1.add(n2). Analyze the objects created, the values stored in n1, n2, n3, and how the object is returned.

3 Marks · CO3

Objects created: three. `n1` (value 10), `n2` (value 20), and a third object created inside `add` and referred to by `temp`.

Values stored: n1.value = 10, n2.value = 20, n3.value = 30.

How the object is returned. Inside `add`, `this.value + n.value` = 10 + 20 = 30 is passed to the constructor, which makes a new object. The local variable `temp` holds its reference. The statement `return temp;` sends this reference back to the caller, where it is stored in `n3`. The variable `temp` then disappears, but the object survives because `n3` refers to it. n1 and n2 are not modified.

Variable	Refers to	value
n1	Object 1	10
n2	Object 2	20
n3 (and temp inside add)	Object 3, created in add	30

Q10. Differentiate the roles of this and this() by analyzing class Student { Student() { this(101,”Rahul”); } Student(int roll, String name) { this.roll = roll; this.name = name; } }

3 Marks · CO3

Given in the question

```
class Student {
    int roll; String name;
    Student()                       { this(101, "Rahul"); }
    Student(int roll, String name) { this.roll = roll; this.name = name; }
}
```

- **`this(101, "Rahul")` in the first constructor is a** constructor call**. It transfers control to `Student(int, String)` with the given arguments. It does not refer to any object; it is a statement that invokes another constructor of the same class and must be first in the constructor.

- **`this.roll` and `this.name` in the second constructor are** references to the current object**. They denote the fields `roll` and `name` of the object being constructed, distinguishing them from the parameters of the same name.

Consequently `new Student()` produces an object with roll = 101 and name = “Rahul”.

Q11. Consider class Demo { int x; Demo() { this(20); println(“Default Constructor”); } Demo(int x) { this.x = x; println(“Parameterized Constructor”); } } Analyze the constructor execution when new Demo() is executed. Determine the output and final value of x.

3 Marks · CO3

Given in the question

```
class Demo {
    int x;
    Demo()      { this(20); System.out.println("Default Constructor"); }
    Demo(int x) { this.x = x; System.out.println("Parameterized Constructor"); }
}
```

Output of `new Demo();`:

Output

```
Parameterized Constructor
Default Constructor
```

Execution. `Demo()` begins and its first statement `this(20)` calls `Demo(int x)`. There, `this.x = x` stores 20 in the field and the message “Parameterized Constructor” is printed. Control returns to `Demo()`, which prints “Default Constructor”. The final value of `x` is **20**; it was never changed again after the parameterized constructor set it.

MODULE

4

Static Members, Encapsulation and Object Relationships

Static members, access modifiers, getters and setters, aggregation, composition and uses-a

Bloom's Level: Remember (L1)

Q1. What is encapsulation?

2 Marks · CO1

Encapsulation is the OOP principle of binding data (variables) and the methods that operate on that data into a single unit, the class, and restricting direct access to the data from outside. The data is usually made `private` and is reached only through public methods.

> **Example.** In `class Account { private double balance; public double getBalance() { return balance; } }`, outside code cannot touch `balance` directly.

Q2. What is data hiding?

2 Marks · CO1

Data hiding is the practice of hiding the internal data of a class from outside access by declaring it `private`, so that it can be read or changed only through the class's own methods. It protects the data from accidental or invalid modification and is achieved through encapsulation.

> **Example.** A private `age` field with a setter that rejects negative values cannot be set to -5 from outside.

Q3. What is an access modifier?

2 Marks · CO1

An access modifier is a keyword that sets the visibility (accessibility) of a class, variable, method or constructor, that is, which other classes may use it. Java has four levels of access: `private`, default (no keyword), `protected` and `public`.

> **Example.** `private int salary;` can be used only inside its own class; `public void show()` can be called from anywhere.

Q4. Name the access modifiers in Java.

2 Marks · CO1

The access modifiers in Java are:

- `private` — accessible within the same class only.
- **default** (no keyword) — accessible within the same package.
- `protected` — accessible within the same package and in subclasses of other packages.
- `public` — accessible from everywhere.

Q5. What is a static variable?

2 Marks · CO1

A static variable (class variable) is declared with the keyword `static`. Only **one copy** of it exists for the whole class, shared by all objects, and it is created when the class is loaded, not when an object is created.

> **Example.** `static int count = 0;` in class `Student` can be used to count the number of students created.

Q6. What is a static method?

2 Marks · CO1

A static method belongs to the class and not to any object. It can be called with the class name, without creating an object, and it can directly use only static members. It cannot use `this`.

> **Example.** `Math.sqrt(25)` is a static method call; no `Math` object is needed.

Q7. What is a getter?

2 Marks · CO1

A getter (accessor) is a public method that returns the value of a private data member, so that outside code can read the value without having direct access. By convention it is named `getXxx()`.

> **Example.** `public String getName() { return name; }`

Q8. What is a setter?

2 Marks · CO1

A setter (mutator) is a public method that assigns a value to a private data member, often after validating it. By convention it is named `setXxx(value)`.

> **Example.** `public void setAge(int age) { if (age > 0) this.age = age; }`

Q9. What is aggregation?

2 Marks · CO1

Aggregation is a **has-a** relationship in which one class contains a reference to objects of another class, but the contained objects have an independent existence. It is a “weak” form of ownership: the whole does not control the life of its parts.

> **Example.** A `Department` has `Student` objects, but if the department is closed, the students still exist.

Q10. What is composition?

2 Marks · CO1

Composition is a **strong has-a** relationship in which the whole owns its parts and the parts cannot exist without the whole. When the container object is destroyed, the contained objects are destroyed with it.

> **Example.** A `Car` is composed of an `Engine` created inside the car; the engine has no meaning without the car.

Bloom’s Level: Understand (L2)

Q1. What is the difference between static and instance variables?

3 Marks · CO1

Basis	Static Variable	Instance Variable
Keyword	Declared with `static`	Declared without `static`
Copies	One copy for the whole class	One copy per object

Basis	Static Variable	Instance Variable
Memory created	When the class is loaded	When the object is created
Access	By class name (or object)	Only through an object
Shared	Shared by all objects	Private to its object
Use	Common data such as a counter or college name	State specific to each object

> **Example.** In `class Student { int roll; static String college = "AOT"; }`, each Student has its own `roll`, but all share the single `college`.

Q2. Why are static members shared among objects?

2 Marks · CO1

A static member belongs to the class itself, not to any object. It is stored once in the memory area of the class when the class is loaded, and every object of the class (and the class name) accesses this same location. Hence a change made through one object is seen by all.

> **Example.** If three objects each execute `count++` on a static `count`, the value is 3, not 1, because the same variable is updated each time.

Q3. Can a static method directly access an instance variable?

2 Marks · CO1

No. A static method can be called without any object, so there is no "current object" for it, and an instance variable exists only inside an object. The compiler reports "non-static variable cannot be referenced from a static context". A static method can access instance variables only through an explicit object reference.

> **Example.** Illustration:

```
class A {
    int x = 5; static int y = 7;
    static void f() {
        // System.out.println(x);   // error
        System.out.println(y);      // OK
        System.out.println(new A().x);   // OK through an object
    }
}
```

Q4. Why are data members generally declared private?

2 Marks · CO1

Private data members cannot be accessed from outside the class, which protects them from accidental or invalid changes. The class can control how its data is read and changed through public methods, validate values, and later change the internal representation without affecting other classes. This is the essence of encapsulation and data hiding.

> **Example.** If `balance` is private, a rule such as "balance cannot be negative" can be enforced in one place: the `withdraw()` method.

Q5. What is the purpose of getter and setter methods?

3 Marks · CO1

Getters and setters provide controlled access to the private fields of a class.

- A **getter** returns the value of a field, giving read access without allowing modification.
- A **setter** assigns a new value, and can **validate** it first, giving safe write access.
- They allow a field to be made read-only (getter only) or write-only (setter only).
- They hide the internal representation, so it can be changed later without altering the callers.

Example. Illustration:

```
class Student {
    private int marks;
    public int getMarks() { return marks; }
    public void setMarks(int m) { if (m >= 0 && m <= 100) marks = m; }
}
```

`setMarks(150)` is ignored, so `marks` always remains within range.

Q6. How does a setter provide controlled access to an object's data?

2 Marks · CO1

The setter is the only way to change the private field, so it can test the new value before accepting it. If the value is valid it is assigned; otherwise it is rejected, corrected, or an exception is thrown. The object is thereby never left in an invalid state.

Example. `void setBalance(double b) { if (b < 0) throw new IllegalArgumentException("Negative"); balance = b; }`

Q7. Differentiate between aggregation and composition.

3 Marks · CO1

Basis	Aggregation	Composition
Strength	Weak has-a	Strong has-a
Part's existence	Independent of the whole	Depends on the whole
Ownership	Whole only refers to parts	Whole owns and creates its parts
Lifetime	Parts survive if the whole is destroyed	Parts are destroyed with the whole
Creation	Part is usually created outside and passed in	Part is usually created inside the whole
Example	Department and Student	Car and Engine

Q8. What is a has-a relationship?

2 Marks · CO1

A has-a relationship exists when one class contains another class as a member, expressing that an object "has" another object as a part. It is implemented by declaring a field of the other class's type. Aggregation and composition are the two kinds of has-a relationship.

> **Example.** A `Library` has `Book` objects; `class Library { Book[] books; }`.

Q9. Give an example of aggregation.

2 Marks · CO1

`Department` and `Student`. A department holds references to several student objects, but the students are created elsewhere and continue to exist even if the department object is deleted.

```
class Department { String name; ArrayList<Student> students = new ArrayList<>(); }
```

Q10. Give an example of composition.

2 Marks · CO1

`Car` and `Engine`. The engine is created inside the car's constructor and is a part of the car; when the car object is destroyed, the engine goes with it.

```
class Car { private final Engine engine = new Engine(); }
```

Q11. Why is Department-Student generally modelled as aggregation?

3 Marks · CO1

A department has students, so a has-a relationship exists. But a student's existence does not depend on a department object: the student can be created independently, may move to another department, and continues to exist when the department is removed. The department merely holds references to them. Because the parts are independent of the whole, the relationship is aggregation, not composition.

> **Example.** If the "Mechanical" Department object is deleted, the Student object "Ravi" remains and can be added to the "Civil" Department.

Q12. Why can Car-Engine be modelled as composition?

3 Marks · CO1

The engine is an essential part of a car that is normally created together with the car and is not shared with other cars. It has no independent role in the program: when the car object is destroyed, the engine has no use and is destroyed as well. The car controls the life cycle of the engine, which is the defining mark of composition.

> **Example.** `Car() { engine = new Engine(); }` creates the engine inside the car's constructor, and no outside code holds a reference to it.

Q13. What is the difference between private and public?

2 Marks · CO1

`private` members are visible only inside the class in which they are declared, which provides data hiding. `public` members are visible to every other class in every package, which makes them the class's external interface. Typically, data is private and the methods that offer the service are public.

> **Example.** `private int pin;` cannot be read from outside, while `public boolean verify(int p)` can be called by anyone.

Q14. What is the purpose of protected?

2 Marks · CO1

`protected` allows a member to be accessed by classes of the same package and by **subclasses**, even if they are in a different package. It is used when a member should be hidden from the general public but made available to derived classes for extension.

> **Example.** A `protected double salary;` in `Employee` can be used inside `Manager extends Employee`, but not by an unrelated class in another package.

Q15. What happens to a static variable when multiple objects are created?

3 Marks · CO1

Nothing is duplicated: there is still only one copy of the static variable, created when the class was loaded. Every new object shares this single copy, so creating objects does not create new copies, and any object (or the class name) that modifies it changes the value for all.

> **Example.** Illustration:

```
class Counter {
    static int count = 0;
    Counter() { count++; }
    public static void main(String[] a) {
        new Counter(); new Counter(); new Counter();
        System.out.println(Counter.count);   // 3
    }
}
```

Bloom’s Level: Apply (L3)

Q1. Write a Java program to define a class Student containing a static data member collegeName. Create three Student objects and demonstrate that the static member is shared by all objects.

5 Marks · CO2

```
class Student {
    int roll;
    static String collegeName = "Academy of Technology";

    Student(int roll) { this.roll = roll; }

    void display() { System.out.println(roll + " - " + collegeName); }

    public static void main(String[] args) {
        Student s1 = new Student(1);
        Student s2 = new Student(2);
        Student s3 = new Student(3);
        s1.display(); s2.display(); s3.display();

        s2.collegeName = "AOT, Hooghly";   // changed through one object
        System.out.println("After change:");
        s1.display(); s3.display();
    }
}
```

Output

```
1 - Academy of Technology
2 - Academy of Technology
3 - Academy of Technology
After change:
1 - AOT, Hooghly
3 - AOT, Hooghly
```

Example. The static variable `collegeName` exists once. Changing it through `s2` also changes what `s1` and `s3` see. The instance variable `roll` is separate for each object.

Q2. Define a class Employee with a static data member employeeCount. Increment the count whenever an Employee object is created and provide a static method to display the total number of employees.

5 Marks · CO2

```
class Employee {
    int id;
    String name;
    static int employeeCount = 0;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
        employeeCount++;                // updated at every creation
    }

    static void showCount() {
        System.out.println("Total employees = " + employeeCount);
    }

    public static void main(String[] args) {
        Employee.showCount();
        new Employee(1, "Ria");
        new Employee(2, "Sam");
        new Employee(3, "Tara");
        Employee.showCount();
    }
}
```

Output

```
Total employees = 0
Total employees = 3
```

Example. Each `new Employee(...)` runs the constructor, which adds 1 to the shared counter: 0, 1, 2, 3. `showCount()` is static, so it is called with the class name and works even before any object exists.

Q3. Write a Java program to demonstrate a static method that operates on a static data member. Explain how the method is invoked without creating an object of the class.

5 Marks · CO2

```
class Bank {
    static double interestRate = 4.5;

    static void increaseRate(double by) {     // static method
        interestRate += by;
    }
    static void showRate() {
        System.out.println("Interest rate = " + interestRate + "%");
    }

    public static void main(String[] args) {
        Bank.showRate();            // called with class name, no object
        Bank.increaseRate(0.5);
        Bank.showRate();
    }
}
```

Output

```
Interest rate = 4.5%
Interest rate = 5.0%
```

Explanation. A static method belongs to the class and is loaded into memory along with the class. It is therefore invoked as `ClassName.method()` and needs no object; `Bank.showRate()` is such a call. Because no object is involved, there is no `this`, and the method can use only static members: it can use `interestRate`, which is static, but cannot use an instance variable directly. The `main` method itself is static for the same reason: the JVM calls it without creating an object.

Q4. Define a class BankAccount with private accountNo, holderName and balance. Implement getters and setters and demonstrate encapsulation by creating and accessing an object.

5 Marks · CO2

```
class BankAccount {
    private int accountNo;
    private String holderName;
    private double balance;

    public int getAccountNo()            { return accountNo; }
    public void setAccountNo(int n)      { accountNo = n; }
    public String getHolderName()        { return holderName; }
    public void setHolderName(String h)  { holderName = h; }
    public double getBalance()           { return balance; }
    public void setBalance(double b) {
        if (b >= 0) balance = b;
        else System.out.println("Invalid balance");
    }

    public static void main(String[] args) {
        BankAccount a = new BankAccount();
        a.setAccountNo(2001);
        a.setHolderName("Devika");
        a.setBalance(15000);
        a.setBalance(-50);              // rejected
        System.out.println(a.getAccountNo() + " " + a.getHolderName()
                           + " " + a.getBalance());
    }
}
```

Output

```
Invalid balance
2001 Devika 15000.0
```

> **Example.** From any other class, `a.balance = -50;` would not compile, because `balance` is private. The only route is `setBalance`, which refuses the negative value, so the data stays valid. This is encapsulation.

Q5. Create a class Student with private rollNo, name and marks. Provide setters for accepting values and getters for displaying. Ensure that marks cannot be assigned a negative value.

5 Marks · CO2

```
class Student {
    private int rollNo;
    private String name;
    private int marks;

    public void setRollNo(int r)   { rollNo = r; }
    public void setName(String n)  { name = n; }
    public void setMarks(int m) {
        if (m < 0) {
            System.out.println("Marks cannot be negative; ignored");
        } else {
            marks = m;
        }
    }

    public int getRollNo()    { return rollNo; }
    public String getName()   { return name; }
    public int getMarks()     { return marks; }

    public static void main(String[] args) {
        Student s = new Student();
        s.setRollNo(14);
        s.setName("Imran");
        s.setMarks(-20);          // rejected
        s.setMarks(82);           // accepted
        System.out.println(s.getRollNo() + " " + s.getName() + " "
                           + s.getMarks());
    }
}
```

Output

```
Marks cannot be negative; ignored
14 Imran 82
```

Example. The first call `setMarks(-20)` fails the test `m < 0`, so `marks` keeps its default 0. The second call stores 82. A negative mark can therefore never enter the object.

Q6. Write a Java program to demonstrate the use of public, private and protected access modifiers using suitable classes and methods.

5 Marks · CO2

```
class Base {
    public int pub = 1;
    protected int prot = 2;
    int def = 4;                 // default (package) access
    private int priv = 3;

    public int getPriv() { return priv; }       // access through a method
    private void secret() { System.out.println("secret"); }
    public void callSecret() { secret(); }      // OK inside the class
}

class Derived extends Base {
    void test() {
        System.out.println(pub);        // OK
        System.out.println(prot);       // OK (subclass)
        System.out.println(def);        // OK (same package)
        // System.out.println(priv);    // ERROR: private in Base
        System.out.println(getPriv());  // OK via public method
    }
}

public class AccessDemo {
    public static void main(String[] args) {
        Base b = new Base();
        System.out.println(b.pub);       // OK
        System.out.println(b.prot);      // OK (same package)
        // System.out.println(b.priv);   // ERROR
        b.callSecret();                  // OK
        new Derived().test();
    }
}
```

Output

```
1
2
secret
1
2
4
3
```

Modifier	Same class	Same package	Subclass in other package	Other package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Example. The private field `priv` is reachable from outside only through the public method `getPriv()`.

Q7. Define classes Student and Department to represent an aggregation relationship. A Department should contain multiple Student objects, but Students should be able to exist independently. Implement methods to add and display students.

5 Marks · CO2

Approach. The `Student` objects are created **outside** the department and passed to `addStudent`; the department only keeps references to them.

```
import java.util.ArrayList;

class Student {
    int roll; String name;
    Student(int roll, String name) { this.roll = roll; this.name = name; }
    void show() { System.out.println(roll + " " + name); }
}

class Department {
    String deptName;
    ArrayList<Student> students = new ArrayList<>();

    Department(String n) { deptName = n; }
    void addStudent(Student s) { students.add(s); }
    void display() {
        System.out.println("Department: " + deptName);
        for (Student s : students) s.show();
    }
}

public class AggregationDemo {
    public static void main(String[] args) {
        Student s1 = new Student(1, "Ankit");      // exist independently
        Student s2 = new Student(2, "Bina");

        Department cse = new Department("CSE");
        cse.addStudent(s1);
        cse.addStudent(s2);
        cse.display();

        cse = null;                 // department removed
        s1.show();                  // student still exists
    }
}
```

Output

```
Department: CSE
1 Ankit
2 Bina
1 Ankit
```

> **Example.** After `cse = null`, the department is no longer reachable, yet `s1` still works. This independent existence of the parts is the mark of aggregation.

Q8. Define classes Car and Engine to represent a composition relationship. The Engine should be created as part of the Car object. Implement a method to display the details of both objects.

5 Marks · CO2

```
class Engine {
    int horsePower; String type;
    Engine(int hp, String type) { horsePower = hp; this.type = type; }
    void show() { System.out.println("Engine: " + horsePower + " HP, " + type); }
}

class Car {
    String model;
    private final Engine engine;           // owned part

    Car(String model, int hp, String type) {
        this.model = model;
        this.engine = new Engine(hp, type);    // created inside the Car
    }

    void display() {
        System.out.println("Car: " + model);
        engine.show();
    }

    public static void main(String[] args) {
        Car c = new Car("Swift", 90, "Petrol");
        c.display();
    }
}
```

Output

```
Car: Swift
Engine: 90 HP, Petrol
```

> **Example.** The `Engine` is created by the `Car` constructor with `new`, is held in a private final field, and is never given to outsiders. When `c` is no longer reachable, its engine becomes unreachable too: life of the part is tied to the life of the whole. This is composition.

Q9. Define classes Bill and Product to demonstrate a uses-a relationship. Write a method in Bill that accepts a Product object and uses its information to calculate the bill amount.

5 Marks · CO2

Approach. `Bill` does not keep the `Product` as a field; it receives it as a method parameter, uses it, and forgets it. This temporary dependency is the “uses-a” relationship.

```
class Product {
    String name; double price;
    Product(String name, double price) { this.name = name; this.price = price; }
}

class Bill {
    double calculate(Product p, int quantity) {
        return p.price * quantity;
    }
```

```
    public static void main(String[] args) {
        Product soap = new Product("Soap", 35.0);
        Bill b = new Bill();
        double amount = b.calculate(soap, 4);
        System.out.println("Bill for " + soap.name + " = Rs. " + amount);
    }
}
```

Output

```
Bill for Soap = Rs. 140.0
```

Example. 4 x Rs. 35 = Rs. 140. The `Bill` object holds no reference to the product after the method returns.

Q10. Create a class Department containing a collection of Student objects. Implement methods to add students and display their details. Explain through the implementation why this represents a has-a/aggregation relationship.

5 Marks · CO2

```
import java.util.ArrayList;

class Student {
    int roll; String name;
    Student(int r, String n) { roll = r; name = n; }
    public String toString() { return roll + " " + name; }
}

class Department {
    private String name;
    private ArrayList<Student> list = new ArrayList<>();

    Department(String name) { this.name = name; }
    void add(Student s) { list.add(s); }
    void display() {
        System.out.println("Students of " + name + ":");
        for (Student s : list) System.out.println("  " + s);
    }

    public static void main(String[] args) {
        Student a = new Student(10, "Mala");
        Student b = new Student(11, "Nitin");
        Department d = new Department("IT");
        d.add(a); d.add(b);
        d.display();
    }
}
```

Output

```
Students of IT:
  10 Mala
  11 Nitin
```

Why this is has-a / aggregation.

- The `Department` class has a field (the list) of `Student` objects: a department **has** students.

- The students are created outside the department (`a` and `b` in `main`) and merely added, so the department does not own their creation.
- The student objects continue to exist if `d` is discarded, and could be added to another department. The parts are independent of the whole, which makes the relationship aggregation rather than composition.

Q11. Design a Java class Library containing private library details and a collection of Book objects. Implement getters/setters and methods to add and display books. Identify the object relationship involved.

5 Marks · CO2

```
import java.util.ArrayList;

class Book {
    int id; String title;
    Book(int id, String title) { this.id = id; this.title = title; }
}

class Library {
    private String libName;
    private String city;
    private ArrayList<Book> books = new ArrayList<>();

    public String getLibName()        { return libName; }
    public void setLibName(String n)  { libName = n; }
    public String getCity()           { return city; }
    public void setCity(String c)     { city = c; }

    public void addBook(Book b) { books.add(b); }

    public void displayBooks() {
        System.out.println(libName + ", " + city);
        for (Book b : books)
            System.out.println("  " + b.id + " : " + b.title);
    }

    public static void main(String[] args) {
        Library lib = new Library();
        lib.setLibName("Central Library");
        lib.setCity("Hooghly");
        lib.addBook(new Book(1, "Java: The Complete Reference"));
        lib.addBook(new Book(2, "Data Structures"));
        lib.displayBooks();
    }
}
```

Output

```
Central Library, Hooghly
  1 : Java: The Complete Reference
  2 : Data Structures
```

Relationship. `Library` **has** a collection of `Book` objects, a has-a relationship. Since a book can exist without a particular library (it may be moved to another library), it is **aggregation**. The private fields and the getters/setters show encapsulation.

Q12. Create a class Computer containing a Processor object. The Processor should be created when the Computer is created. Implement suitable constructors and a display method. Identify the relationship.

5 Marks · CO2

```
class Processor {
    String brand; double speedGHz;
    Processor(String brand, double speedGHz) {
        this.brand = brand; this.speedGHz = speedGHz;
    }
    void show() { System.out.println("Processor: " + brand + " @ " + speedGHz + " GHz"); }
}

class Computer {
    String name;
    private Processor cpu;

    Computer() {
        this("Generic PC");
    }
    Computer(String name) {
        this.name = name;
        this.cpu = new Processor("Intel i5", 3.2);   // created with the Computer
    }

    void display() {
        System.out.println("Computer: " + name);
        cpu.show();
    }

    public static void main(String[] args) {
        new Computer().display();
        new Computer("Lab-PC-7").display();
    }
}
```

Output

```
Computer: Generic PC
Processor: Intel i5 @ 3.2 GHz
Computer: Lab-PC-7
Processor: Intel i5 @ 3.2 GHz
```

Relationship: composition. The `Processor` is created inside the `Computer` constructor and is not shared. It is part of that computer and does not exist without it, so Computer-Processor is a strong has-a relationship.

Bloom's Level: Analyze (L4)

Q1. Analyze the program (class Student { int roll; static String college = "AOT"; }). Three Student objects are created and college is modified through one object. Determine the output and explain why the static variable is common; analyze the effect on the other objects.

3 Marks · CO3

Given in the question

```
class Student {
    int roll;
    static String college = "AOT";
}
// s1, s2, s3 are created; s1.college = "NIT";  then college is printed through s1, s2, s3
```

Output (printing `college` through each object):

Output

```
NIT
NIT
NIT
```

Why. `college` is static, so it exists once, in the class, not in each object. `s1.college`, `s2.college` and `s3.college` are only three ways of naming the same variable (`Student.college`). Writing through `s1` therefore changes the single shared variable, and the other objects, which read the same variable, see the new value. The instance variable `roll` is unaffected; each object keeps its own.

Q2. Consider a class containing both static and instance variables. Analyze which variables are shared and which are separate. Justify with a suitable object-state representation.

3 Marks · CO3

Take `class Employee { int id; String name; static String company = "TCS"; }` with objects e1 = (101, "Amit") and e2 = (102, "Rita").

Variable	Kind	Where stored	Shared?
id	Instance	In each object	No: e1.id = 101, e2.id = 102
name	Instance	In each object	No: "Amit" and "Rita"
company	Static	Once, in the class	Yes: one value "TCS" for all

Object-state representation. Object e1 holds {id = 101, name = "Amit"}; object e2 holds {id = 102, name = "Rita"}; the class Employee holds {company = "TCS"}, and both e1 and e2 look up `company` from the class.

Justification. Per-object attributes (identity, name) must differ between objects, so they are instance variables. A property common to all objects (the company) is held once, which saves memory and keeps all objects consistent when it changes.

Q3. A class has a private data member balance and provides getBalance() and setBalance() methods. Explain how a setter can be designed to prevent invalid values such as a negative balance.

3 Marks · CO3

Since `balance` is private, every change must pass through `setBalance`, so the method is the right place to enforce the rule. The setter should (1) test the new value, (2) assign it only if valid, and (3) report the error, either by returning a status or by throwing an exception.

```
class Account {
    private double balance;
    public double getBalance() { return balance; }
    public void setBalance(double b) {
        if (b < 0)
            throw new IllegalArgumentException("Balance cannot be negative");
        balance = b;
    }
}
```

> **Example.** `setBalance(2500)` succeeds; `setBalance(-100)` throws an exception and the previous balance is retained, so no negative balance can ever be stored. The constructor should use the same check, otherwise it becomes a loophole.

Q4. A class Student declares all its data members as public. Analyze the design from the perspective of encapsulation and data hiding. Suggest changes using access modifiers and getters/setters.

3 Marks · CO3

Analysis of the problems.

- Any class can change `roll`, `name` or `marks` directly (for example `s.marks = -50;`), so invalid data can enter the object.
- The class cannot control or monitor its own state, which violates encapsulation and data hiding.
- If the representation changes (for example `name` is split into first and last names), every class using the field directly would have to be changed.

Suggested redesign.

```
class Student {
    private int roll;
    private String name;
    private int marks;

    public int getRoll()  { return roll; }
    public void setRoll(int r) { if (r > 0) roll = r; }
    public String getName() { return name; }
    public void setName(String n) { if (n != null && !n.isEmpty()) name = n; }
    public int getMarks() { return marks; }
    public void setMarks(int m) { if (m >= 0 && m <= 100) marks = m; }
}
```

Fields become `private`, and public getters and setters expose them with validation.

Q5. Analyze the access situation: a class contains private, public, protected and default members. Determine which can be accessed from (a) the same class, (b) another class in the same package, (c) a class in another package.

3 Marks · CO3

Member	(a) Same class	(b) Another class, same package	(c) Class in another package
private	Accessible	Not accessible	Not accessible
default	Accessible	Accessible	Not accessible
protected	Accessible	Accessible	Only through a subclass
public	Accessible	Accessible	Accessible

Justification. `private` restricts to the declaring class; default (package-private) widens to the package; `protected` adds subclasses outside the package; `public` is unrestricted.

> **Example.** A class `Shape` in package `p1` has all four kinds of members. Another class `Test` in `p1` can use public, protected and default members, but not private ones. A class `Other` in package `p2` can use only the public members, unless it extends `Shape`, in which case it can also use protected ones through inheritance.

Q6. A Department object containing several Student objects is deleted while the Student objects continue to exist. Determine whether this is aggregation or composition. Justify.

3 Marks · CO3

It is **aggregation**.

Justification. There is a has-a relationship (a department has students). In composition, deleting the whole deletes its parts. Here the students survive the deletion of the department, so their existence does not depend on it. The department only holds references to students that were created independently, and the students could belong to a different department tomorrow. A part that outlives the whole means the relationship is the weak kind, aggregation.

> **Example.** `Student s = new Student(1,"Ravi"); Department d = new Department("CSE"); d.add(s); d = null;` The student `s` is still usable.

Q7. A Car object creates its Engine internally, and the Engine is not designed to exist independently of the Car. Analyze the relationship and explain why it is composition.

3 Marks · CO3

The relationship is **composition**.

Reasons.

- **Creation:** the Engine is created inside the Car (`engine = new Engine()` in the constructor), so the Car owns it.
- **Dependency:** the Engine is not meant to exist independently; it has no role outside the Car.
- **Lifetime:** when the Car object is destroyed, the Engine is destroyed with it, because no other object refers to it.

- **Exclusivity:** the Engine belongs to this Car only and is not shared.

In aggregation, the part would be created outside and passed in, and could outlive the whole. Because the Engine fails all of these tests, it is composition.

Q8. A Bill object receives a Product object as a method parameter, uses its price and quantity to calculate the bill, and does not permanently own the Product. Analyze the relationship.

3 Marks · CO3

The relationship is a **dependency (uses-a)** relationship, which is weaker than aggregation and composition.

Analysis. `Bill` does not store `Product` in a field. The reference exists only during the method call, as a parameter, and ends when the method returns. Bill therefore neither owns nor holds the product; it merely **uses** it for a calculation. Since it has no permanent link, it is not has-a (aggregation or composition).

> **Example.** `double amount = bill.calculate(product, 3);` After the call, the Bill object has no memory of the Product.

Q9. Classify each as aggregation, composition or uses-a: (a) Department-Student (b) Car-Engine (c) Bill-Product. Give justification.

5 Marks · CO3

Pair	Classification	Justification
(a) Department-Student	Aggregation	Department holds students as a field, but students are created outside and exist independently of the department.
(b) Car-Engine	Composition	Engine is created inside the Car, owned by it, and destroyed with it.
(c) Bill-Product	Uses-a (dependency)	Product is only a method parameter; Bill keeps no permanent reference to it.

Discussion. The three differ in the strength of the link. Composition is the strongest (shared life cycle), aggregation is weaker (independent parts but held as fields), and uses-a is the weakest (temporary use in a method). A useful test is to ask two questions: “Is the other object stored as a field?” If no, it is uses-a. If yes, “Does it live and die with the container?” If yes, composition; if no, aggregation.

Q10. A University contains several Department objects, and each Department contains several Student objects. Analyze the object relationships at both levels and identify the appropriate has-a relationships.

5 Marks · CO3

Structure: University -> Departments -> Students.

```
class University { String name; ArrayList<Department> depts; }
class Department { String name;  ArrayList<Student> students; }
class Student    { int roll;     String name; }
```

Level 1: University-Department. A department is normally created as part of a university, is not shared with another university, and has no meaning without it; when the university is closed, its departments cease to exist. This is best modelled as **composition**.

Level 2: Department-Student. Students are independent persons who can exist when a department is closed, and may change departments. This is **aggregation**.

Level	Relationship	Reason
University - Department	Composition (has-a, strong)	Departments depend on the university
Department - Student	Aggregation (has-a, weak)	Students exist independently
University - Student	Indirect (through Department)	No direct reference needed

> **Example.** Closing the "Mechanical" department removes it from the university, but the student "Ravi" can still be enrolled in "Civil". Dissolving the university destroys all its departments, but the students as persons remain.

Q11. Consider a class Employee in which id is private, name is private and companyName is static. Analyze which attributes belong to individual objects and which is shared among all Employee objects.

4 Marks · CO3

Given in the question

```
class Employee {
    private int id;
    private String name;
    static String companyName = "AOT Pvt. Ltd.";
}
```

Attributes of individual objects: `id` and `name`. They are instance variables, so each Employee has its own copies, and because they are private they can be read or changed only through methods of the class.

Attribute shared among all objects: `companyName`. It is static, so only one copy exists, belonging to the class, and all Employee objects see the same value.

Combined view. For e1 = (id 1, "Asha") and e2 = (id 2, "Bipin"), each has distinct private state, while `Employee.companyName` is a single value. If the company is renamed once, all employees reflect the new name. Note also that `companyName` is not private, so it can be changed by any class; for full encapsulation it should be `private static` with a static getter and setter.

Q12. Analyze a given class that uses setter methods to update private data members. Identify possible violations of encapsulation and suggest modifications to ensure that only valid data can be stored.

3 Marks · CO3

A class with private fields and setters may still violate encapsulation in these ways:

- **Setters without validation** (`void setAge(int a) { age = a; }`) accept any value, so the privacy is only cosmetic.
- **Constructor bypasses validation:** the constructor assigns fields directly and can store invalid values.

- **Returning references to mutable internals:** a getter such as `int[] getMarks() { return marks; }` lets callers modify the private array.
- **Unnecessary setters** for fields that should be fixed (for example `setRoll`) allow changes that should not occur.

Modifications.

```
class Person {
    private int age;
    private final int[] marks = new int[3];

    Person(int age) { setAge(age); }              // constructor reuses the setter

    public void setAge(int age) {
        if (age < 0 || age > 120) throw new IllegalArgumentException();
        this.age = age;
    }
    public int[] getMarks() { return marks.clone(); }   // defensive copy
}
```

Validate in every setter, reuse the setter in the constructor, return copies of mutable fields, and omit setters for fields that must not change.

MODULE

5

Inheritance

CA2: IS-A relationship, types of inheritance, overriding, dynamic dispatch, super and casting

Bloom's Level: Remember (L1)

Q1. Define inheritance in object-oriented programming.

2 Marks · CO1

Inheritance is the mechanism by which a new class (the subclass) acquires the fields and methods of an existing class (the superclass), so that existing code can be reused and extended. It models an **IS-A** relationship between the two classes.

> **Example.** `class Dog extends Animal {}`: a `Dog` automatically has all the accessible members of `Animal`, such as `eat()`, and may add its own, such as `bark()`.

Q2. What is a superclass?

2 Marks · CO1

A superclass (parent or base class) is the class whose members are inherited by another class. It contains the common state and behaviour shared by its subclasses.

> **Example.** In `class Student extends Person`, `Person` is the superclass.

Q3. What is a subclass?

2 Marks · CO1

A subclass (child or derived class) is a class that inherits from another class using `extends`. It gets the accessible members of its superclass and may add new members or override inherited methods.

> **Example.** In `class Student extends Person`, `Student` is the subclass.

Q4. What is the purpose of the extends keyword in Java?

2 Marks · CO1

`extends` declares that a class is derived from another class, thereby establishing inheritance. The class named after `extends` becomes the direct superclass. A Java class can extend only one class (an interface may extend several interfaces).

> **Example.** `class Car extends Vehicle { }` makes `Vehicle` the superclass of `Car`.

Q5. List the different types of inheritance.

2 Marks · CO1

The types of inheritance are: (1) **single**, (2) **multilevel**, (3) **hierarchical**, (4) **multiple** and (5) **hybrid**. Java supports the first three through classes; multiple and hybrid inheritance are achieved only through interfaces.

Q6. What is single inheritance?

2 Marks · CO1

In single inheritance, a subclass inherits from exactly one superclass.

> **Example.** `class B extends A {}`: B has one parent, A.

Q7. What is multilevel inheritance?

2 Marks · CO1

In multilevel inheritance, a class is derived from a class which is itself derived from another class, forming a chain of inheritance.

> **Example.** `class A {}`, `class B extends A {}`, `class C extends B {}`: C inherits the members of both B and A.

Q8. What is hierarchical inheritance?

2 Marks · CO1

In hierarchical inheritance, more than one subclass inherits from the same single superclass.

> **Example.** `class Circle extends Shape {}` and `class Rectangle extends Shape {}`: both share the parent `Shape`.

Q9. What is multiple inheritance?

2 Marks · CO1

Multiple inheritance means that a class inherits from more than one parent at the same time. Java does **not** allow it with classes, to avoid ambiguity, but a class can implement several interfaces to get the same effect.

> **Example.** `class Duck implements Flyable, Swimmable {}` is permitted; `class Duck extends Bird, Fish {}` is a compile-time error.

Q10. What is meant by an IS-A relationship?

2 Marks · CO1

An IS-A relationship states that a subclass object is a specialised kind of its superclass. It is implemented by inheritance and tested by the sentence “X is a Y”.

> **Example.** A Dog **is an** Animal, so `Dog extends Animal`. A Car is **not** an Engine, so Car should not extend Engine.

Q11. What is method overriding?

2 Marks · CO1

Method overriding occurs when a subclass provides its own implementation of a method already defined in its superclass, with the same name, parameter list and a compatible return type. The version called is decided by the actual object at run time.

> **Example.** `Animal has void sound(); Dog redefines void sound() { System.out.println("Bark"); }`.

Q12. What is dynamic method dispatch?

2 Marks · CO1

Dynamic method dispatch is the run-time mechanism by which a call to an overridden method through a superclass reference is resolved to the version in the actual (subclass) object. It is how Java implements run-time polymorphism.

> **Example.** `Animal a = new Dog(); a.sound();` calls `Dog`'s `sound()`, not `Animal`'s.

Q13. What is upcasting?

2 Marks · CO1

Upcasting is assigning a subclass object to a superclass reference. It is done automatically and is always safe.

> **Example.** `Animal a = new Dog();` (the reference can then use only members declared in `Animal`, but overridden methods run the `Dog` version).

Q14. What is downcasting?

2 Marks · CO1

Downcasting is converting a superclass reference back to a subclass type, using an explicit cast. It is valid only if the object really is of that subclass; otherwise a `ClassCastException` occurs at run time.

> **Example.** `Animal a = new Dog(); Dog d = (Dog) a;` is valid, while `Animal b = new Animal(); Dog e = (Dog) b;` throws an exception.

Q15. What is the purpose of the super keyword in inheritance?

2 Marks · CO1

`super` refers to the immediate superclass. It is used (i) to call a superclass constructor with `super(...)`, (ii) to access a superclass field hidden by a subclass field, and (iii) to call a superclass method that has been overridden.

> **Example.** `super.show();` calls the parent's `show()` from inside the child's `show()`.

Q16. Is a constructor inherited by a subclass in Java?

2 Marks · CO1

No. Constructors are not members that are inherited, as each has the name of its own class. A subclass constructor, however, always calls a superclass constructor (explicitly through `super(...)` or implicitly through `super()`).

Q17. Which class is the superclass of all Java classes?

2 Marks · CO1

The class `java.lang.Object`. Every class that does not extend any class explicitly extends `Object` implicitly, so methods such as `toString()`, `equals()` and `hashCode()` are available to all objects.

Q18. Which keyword can be used to prevent a class from being inherited?

2 Marks · CO1

The keyword `final`. A class declared `final` cannot be extended.

> **Example.** `final class Math2 {}; class X extends Math2 {}` gives a compile-time error. `java.lang.String` is a final class.

Bloom's Level: Understand (L2)

Q1. Explain the concept of inheritance with a suitable Java example.

3 Marks · CO2

Inheritance lets a new class be built on an existing class. The existing class (superclass) holds the common members; the new class (subclass), declared with `extends`, receives them automatically and adds or changes whatever is special to it. Its main benefits are **code reusability**, **extensibility**, and the ability to achieve **run-time polymorphism**.

```
class Animal {
    void eat() { System.out.println("Eating"); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}
public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();      // inherited from Animal
        d.bark();     // own method
    }
}
```

Output

```
Eating
Barking
```

> **Example.** `Dog` never defines `eat()`, yet `d.eat()` works because it is inherited from `Animal`.

Q2. Explain the relationship between a superclass and a subclass.

3 Marks · CO2

The relationship is one of **generalisation and specialisation**, expressed as IS-A. The superclass is the general form containing the common members; the subclass is a specialised form that has everything the superclass has and may add more. Consequently:

- a subclass object can be used wherever a superclass object is expected (upcasting);
- the subclass inherits the non-private members of the superclass, but private members are not directly accessible;
- the subclass may override inherited methods;
- the superclass knows nothing about its subclasses and cannot use their new members.

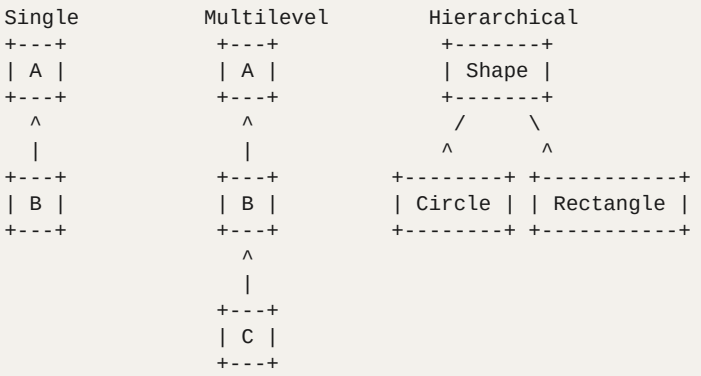
> **Example.** `Employee` is the superclass with `name` and `salary`; `Manager extends Employee` adds `bonus`. Every Manager is an Employee, but not every Employee is a Manager.

Q3. Explain the different types of inheritance supported through classes in Java with suitable diagrams.

3 Marks · CO2

Java supports three types through classes: single, multilevel and hierarchical. (Multiple and hybrid inheritance are not supported through classes.)

Diagram



- **Single:** `class B extends A`.
- **Multilevel:** `class B extends A`, `class C extends B`; C inherits from both.
- **Hierarchical:** `class Circle extends Shape`, `class Rectangle extends Shape`.

> **Example.** For **multiple** inheritance (not allowed with classes) the diagram would show C with two arrows, to A and to B; Java allows this only with interfaces: `class C implements I1, I2`.

Q4. Explain single inheritance with a suitable Java example.

3 Marks · CO2

In single inheritance a subclass is derived from only one superclass. It is the simplest form and is used when one class is a direct specialisation of another.

```
class Person {
    String name = "Ravi";
    void show() { System.out.println("Name: " + name); }
}
class Student extends Person {
    int roll = 7;
    void display() { show(); System.out.println("Roll: " + roll); }
    public static void main(String[] a) { new Student().display(); }
}
```

Output

```
Name: Ravi
Roll: 7
```

> **Example.** `Student` reuses `name` and `show()` of `Person` and adds `roll`.

Q5. Explain multilevel inheritance with a suitable example.

3 Marks · CO2

In multilevel inheritance, a class inherits from a subclass, forming a chain: grandparent, parent, child. The lowest class receives the members of every class above it.

```
class Vehicle { void move() { System.out.println("Moves"); } }
class Car extends Vehicle { void fuel() { System.out.println("Petrol"); } }
class SportsCar extends Car {
    void speed() { System.out.println("200 km/h"); }
```

```
    public static void main(String[] a) {
        SportsCar s = new SportsCar();
        s.move(); s.fuel(); s.speed();
    }
}
```

Output

```
Moves
Petrol
200 km/h
```

> **Example.** `SportsCar` uses `move()` from its grandparent `Vehicle`, `fuel()` from its parent `Car` and its own `speed()`.

Q6. Explain hierarchical inheritance with a suitable example.

3 Marks · CO2

In hierarchical inheritance, several subclasses inherit from one common superclass. The common code is written once in the parent and each child adds its own specifics.

```
class Shape { void info() { System.out.println("I am a shape"); } }
class Circle extends Shape { void draw() { System.out.println("Circle"); } }
class Square extends Shape { void draw() { System.out.println("Square"); } }
class Demo {
    public static void main(String[] a) {
        Circle c = new Circle(); Square s = new Square();
        c.info(); c.draw(); s.info(); s.draw();
    }
}
```

Output

```
I am a shape
Circle
I am a shape
Square
```

> **Example.** `Circle` and `Square` are siblings; neither can use the other's `draw()`, but both share `info()`.

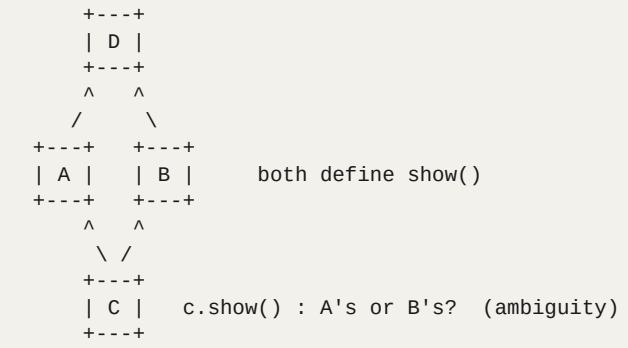
Q7. Explain why Java does not support multiple inheritance through classes.

5 Marks · CO2

Java deliberately forbids a class from extending more than one class. The reasons are:

- **The diamond (ambiguity) problem.** If class C extends both A and B, and both define `show()`, a call `c.show()` leaves it unclear which version C inherits.
- **Shared ancestor state.** If A and B both extend D, C would receive two copies of D's fields (or a complicated rule to merge them).
- **Constructor chaining difficulty.** Each parent constructor must run first, and the order and arguments would be unclear.
- **Complexity and maintenance.** Deep multiple hierarchies are hard to understand, to debug and to change; Java's designers preferred simplicity and reliability.

Diagram



Java solves the genuine need through **interfaces**: a class may `implement` many interfaces. Before Java 8 interfaces had no method bodies, so no conflict arose; with default methods, a class that inherits two identical defaults is forced by the compiler to override and choose explicitly.

> **Example.** `class C extends A, B { }` gives the error "'{' expected". `class C implements I1, I2 { }` compiles; if both interfaces declare `void show();` the class supplies a single body that serves both.

Q8. Explain how inheritance promotes code reusability.

3 Marks · CO2

Common fields and methods are written once in the superclass and automatically become available to all subclasses, so they need not be rewritten. This reduces duplication and the number of errors, because a bug fixed in the parent is fixed for all children, and an improvement made in the parent is available everywhere. New classes are added by writing only what is new.

> **Example.** `Employee` has `name`, `id` and `showDetails()`. `Manager`, `Clerk` and `Developer` extend `Employee` and reuse these three members; each needs only its own extra code (bonus, overtime, language).

Q9. Explain the IS-A relationship with suitable examples.

3 Marks · CO2

IS-A is the relationship between a general class and its specialised class, implemented by `extends` (or `implements` for interfaces). To check it, read the sentence "Subclass is a Superclass"; if it is sensible, inheritance is suitable. The relationship is **one-directional**: a Dog is an Animal, but an Animal is not necessarily a Dog.

```
class Animal { }
class Dog extends Animal { }      // Dog IS-A Animal
class Puppy extends Dog { }       // Puppy IS-A Dog, and IS-A Animal
```

> **Example.** "Car is a Vehicle" is IS-A (inheritance). "Car has an Engine" is HAS-A (composition: a field `Engine engine;`).

Q10. Explain method overriding in the context of inheritance.

3 Marks · CO2

When a subclass needs a different behaviour for a method it inherits, it redefines the method with the **same signature** (name and parameters) and a compatible return type. Rules: the access level cannot be reduced; `final`, `private` and `static` methods cannot be overridden; the annotation `@Override` asks the compiler to verify the override. Overriding gives run-time polymorphism.

```
class Bank { double rate() { return 5.0; } }
class SBI extends Bank {
    @Override double rate() { return 6.5; }
}
// new SBI().rate() returns 6.5
```

> **Example.** `Bank b = new SBI(); b.rate();` returns 6.5, because the method of the actual object is chosen.

Q11. Explain dynamic method dispatch and its relationship with run-time polymorphism.

3 Marks · CO2

Dynamic method dispatch is the process by which the JVM decides, **at run time**, which version of an overridden method to execute, based on the **type of the object** the superclass reference actually points to and not on the type of the reference. Run-time polymorphism (one interface, many forms) is the behaviour that results: the same call `ref.method()` behaves differently for different objects. Dispatch is the mechanism; run-time polymorphism is the effect.

```
class Shape  { void draw() { System.out.println("Shape"); } }
class Circle extends Shape { void draw() { System.out.println("Circle"); } }
class Test {
    public static void main(String[] a) {
        Shape s = new Circle();
        s.draw();           // Circle
        s = new Shape();
        s.draw();           // Shape
    }
}
```

Output

```
Circle
Shape
```

> **Example.** The same statement `s.draw()` gave two different results because `s` pointed to two different kinds of objects.

Q12. Explain upcasting and downcasting with suitable examples.

5 Marks · CO2

Upcasting is the conversion of a subclass reference to a superclass reference. It is implicit and always safe, because a subclass object is-a superclass object. After upcasting, only the members declared in the superclass can be accessed through the reference, but overridden methods still execute the subclass version.

Downcasting is the conversion of a superclass reference to a subclass reference. It needs an explicit cast and is safe only if the object is truly an instance of the subclass. It is used to reach members that exist only in the subclass. An invalid downcast throws `ClassCastException`, so it is guarded with `instanceof`.

```
class Animal { void eat() { System.out.println("Animal eats"); } }
class Dog extends Animal {
    void eat()  { System.out.println("Dog eats"); }
    void bark() { System.out.println("Dog barks"); }
}
public class CastDemo {
    public static void main(String[] args) {
        Animal a = new Dog();       // upcasting
        a.eat();                    // Dog eats  (overridden version)
        // a.bark();                // error: not visible in Animal
        if (a instanceof Dog) {
            Dog d = (Dog) a;        // downcasting
            d.bark();
        }
        Animal x = new Animal();
        // Dog y = (Dog) x;         // ClassCastException at run time
    }
}
```

Output

```
Dog eats
Dog barks
```

Basis	Upcasting	Downcasting
Direction	Subclass to superclass	Superclass to subclass
Cast operator	Not needed	Required
Safety	Always safe	May throw ClassCastException
Access gained	Only superclass members	Subclass-specific members too

Q13. Explain the use of the super keyword in a subclass.

3 Marks · CO2

`super` refers to the superclass part of the current object. It has three uses:

- `super(args)` calls a superclass constructor; it must be the first statement of the subclass constructor.
- `super.field` accesses a superclass field that is hidden by a subclass field of the same name.
- `super.method()` calls the superclass version of a method that the subclass has overridden.

```
class A { int x = 10; void show() { System.out.println("A"); } }
class B extends A {
    int x = 20;
    void show() {
        super.show();
        System.out.println(x + " " + super.x);
    }
    public static void main(String[] a) { new B().show(); }
}
```

Output

```
A
20 10
```

Q14. Explain why constructors are not inherited by subclasses.

3 Marks · CO2

A constructor's name is the name of its class, and its job is to initialise the fields of **that** class. A constructor named `Person` cannot be a constructor of `Student`. If constructors were inherited, `new Student(...)` could create an object using a parent's constructor without initialising the fields that Student adds, leaving the object incomplete. Hence each class defines its own constructors, and a subclass constructor calls a superclass constructor with `super(...)` to initialise the inherited part first.

> **Example.** If `Person(String n)` is the only constructor of `Person`, then `new Student("Ravi")` is an error unless `Student` declares `Student(String n) { super(n); }`.

Q15. Explain the difference between inheritance and composition with suitable examples.

3 Marks · CO2

Basis	Inheritance	Composition
Relationship	IS-A	HAS-A
Keyword / form	`extends`	A field of another class type
Coupling	Tight; child depends on parent's internals	Loose; uses only the public interface
Flexibility	Fixed at compile time	Can change the contained object at run time
Reuse	Of a parent's code	Of a part's functionality
Example	Dog is an Animal	Car has an Engine

```
class Engine { void start() { System.out.println("Engine on"); } }
class Car {
    private Engine engine = new Engine();     // composition
    void start() { engine.start(); }
}
```

> **Example.** `Car extends Engine` would be wrong (a car is not an engine), whereas `Car` containing an `Engine` expresses the real-world relation correctly.

Bloom’s Level: Apply (L3)

Q1. Write a Java program to implement single inheritance using Person and Student classes.

5 Marks · CO3

Approach. `Person` holds the common data (name, age); `Student extends Person` adds roll number and course.

```
class Person {
    String name; int age;
    Person(String name, int age) { this.name = name; this.age = age; }
    void showPerson() { System.out.println("Name: " + name + ", Age: " + age); }
}
class Student extends Person {
    int roll; String course;
    Student(String name, int age, int roll, String course) {
        super(name, age);
        this.roll = roll; this.course = course;
    }
    void showStudent() {
        showPerson();                       // inherited method
        System.out.println("Roll: " + roll + ", Course: " + course);
    }
    public static void main(String[] args) {
        new Student("Anil", 20, 15, "B.Tech CSE").showStudent();
    }
}
```

Output

```
Name: Anil, Age: 20
Roll: 15, Course: B.Tech CSE
```

> **Example.** `Student` reuses `name`, `age` and `showPerson()` without rewriting them; the constructor passes name and age up with `super(...)`.

Q2. Write a Java program to implement multilevel inheritance using Person, Employee, and Manager classes.

5 Marks · CO3

```
class Person {
    String name;
    Person(String n) { name = n; }
    void showName() { System.out.println("Name   : " + name); }
}
class Employee extends Person {
    int empId; double salary;
    Employee(String n, int id, double s) { super(n); empId = id; salary = s; }
    void showEmployee() {
        showName();
        System.out.println("Emp ID : " + empId + "\nSalary : " + salary);
    }
}
class Manager extends Employee {
    double bonus;
    Manager(String n, int id, double s, double b) { super(n, id, s); bonus = b; }
    void showManager() {
        showEmployee();
        System.out.println("Bonus  : " + bonus);
        System.out.println("Total  : " + (salary + bonus));
    }
    public static void main(String[] args) {
        new Manager("Sunita", 501, 60000, 8000).showManager();
    }
}
```

Output

```
Name   : Sunita
Emp ID : 501
Salary : 60000.0
Bonus  : 8000.0
Total  : 68000.0
```

Example. The constructor chain runs `Manager` -> `Employee` -> `Person`; Manager inherits from both ancestors.

Q3. Develop a Java program to demonstrate hierarchical inheritance using Shape, Circle, and Rectangle classes.

5 Marks · CO3

```
class Shape {
    String color;
    Shape(String color) { this.color = color; }
    void showColor() { System.out.println("Colour: " + color); }
}
class Circle extends Shape {
    double r;
    Circle(String c, double r) { super(c); this.r = r; }
    double area() { return Math.PI * r * r; }
}
class Rectangle extends Shape {
    double l, b;
    Rectangle(String c, double l, double b) { super(c); this.l = l; this.b = b; }
    double area() { return l * b; }
}
class Main {
    public static void main(String[] args) {
        Circle c = new Circle("Red", 7);
        Rectangle r = new Rectangle("Blue", 5, 4);
        c.showColor(); System.out.printf("Circle area = %.2f%n", c.area());
        r.showColor(); System.out.println("Rectangle area = " + r.area());
    }
}
```

Output

```
Colour: Red
Circle area = 153.94
Colour: Blue
Rectangle area = 20.0
```

Example. Both subclasses share `color` and `showColor()` from `Shape`, and each has its own `area()`.

Q4. Write a Java program in which a subclass accesses the data members and methods of its superclass using inheritance.

5 Marks · CO3

```
class Account {
    protected String holder;
    protected double balance;
    Account(String h, double b) { holder = h; balance = b; }
    void deposit(double amt) { balance += amt; }
    void showBalance() { System.out.println(holder + ": " + balance); }
}
class Savings extends Account {
    double rate = 4.0;
    Savings(String h, double b) { super(h, b); }
    void addInterest() {
        double interest = balance * rate / 100;      // inherited data member
        deposit(interest);                           // inherited method
    }
    public static void main(String[] args) {
        Savings s = new Savings("Gita", 10000);
        s.addInterest();
        s.showBalance();
    }
}
```

Output

```
Gita: 10400.0
```

> **Example.** `Savings` reads `balance` (declared `protected` in `Account`) and calls `deposit()` as though they were its own. Interest = 10000 x 4 / 100 = 400.

Q5. Develop a Java program to demonstrate method overriding using a superclass Vehicle and subclasses Car and Bike.

5 Marks · CO3

```
class Vehicle {
    void run() { System.out.println("Vehicle is running"); }
    void wheels() { System.out.println("Number of wheels varies"); }
}
class Car extends Vehicle {
    @Override void run() { System.out.println("Car runs on 4 wheels"); }
}
class Bike extends Vehicle {
    @Override void run() { System.out.println("Bike runs on 2 wheels"); }
}
class Test {
    public static void main(String[] args) {
        Vehicle v = new Vehicle(); v.run();
        Car c = new Car();         c.run();
        Bike b = new Bike();       b.run();
        c.wheels();                // not overridden: inherited as is
    }
}
```

Output

```
Vehicle is running
Car runs on 4 wheels
Bike runs on 2 wheels
Number of wheels varies
```

Example. `run()` is overridden in both subclasses, so each prints its own message; `wheels()` is not overridden, so the inherited version runs.

Q6. Write a Java program to demonstrate run-time polymorphism using dynamic method dispatch.

5 Marks · CO3

```
class Bank { double interest() { return 0; } }
class SBI extends Bank  { double interest() { return 6.5; } }
class HDFC extends Bank { double interest() { return 7.1; } }
class ICICI extends Bank{ double interest() { return 6.9; } }

public class Poly {
    public static void main(String[] args) {
        Bank b;                         // superclass reference
        b = new SBI();   System.out.println("SBI   : " + b.interest());
        b = new HDFC();  System.out.println("HDFC  : " + b.interest());
        b = new ICICI(); System.out.println("ICICI : " + b.interest());
    }
}
```

Output

```
SBI   : 6.5
HDFC  : 7.1
ICICI : 6.9
```

Example. The same call `b.interest()` runs a different method each time, chosen at run time from the actual object. This is dynamic method dispatch.

Q7. Develop a Java program demonstrating the use of the super keyword to access superclass data members and methods.

5 Marks · CO3

```
class Parent {
    String name = "Parent";
    void display() { System.out.println("Parent display()"); }
}
class Child extends Parent {
    String name = "Child";                  // hides Parent's name
    void display() {
        System.out.println("Child display()");
        super.display();                    // superclass method
    }
    void show() {
        System.out.println("this.name  = " + this.name);
        System.out.println("super.name = " + super.name);
        display();
    }
    public static void main(String[] args) { new Child().show(); }
}
```

Output

```
this.name  = Child
super.name = Parent
Child display()
Parent display()
```

Example. `name` exists in both classes; `super.name` reaches the hidden parent field, and `super.display()` calls the overridden parent method.

Q8. Write a Java program demonstrating the use of super() to invoke a superclass constructor.

5 Marks · CO3

```
class Person {
    String name; int age;
    Person(String name, int age) {
        this.name = name; this.age = age;
        System.out.println("Person constructor");
    }
}
class Employee extends Person {
    double salary;
    Employee(String name, int age, double salary) {
        super(name, age);                   // must be first
        this.salary = salary;
        System.out.println("Employee constructor");
    }
    void show() { System.out.println(name + " " + age + " " + salary); }
    public static void main(String[] args) {
        new Employee("Rekha", 30, 45000).show();
    }
}
```

Output

```
Person constructor
Employee constructor
Rekha 30 45000.0
```

Example. `super(name, age)` runs first, so the parent part is initialised before the child's body executes. Because `Person` has no no-argument constructor, omitting the `super(...)` call would be a compile-time error.

Q9. Develop a Java program to demonstrate upcasting using a superclass reference referring to a subclass object.

5 Marks · CO3

```
class Animal {
    void sleep() { System.out.println("Animal sleeps"); }
    void sound() { System.out.println("Some sound"); }
}
class Cat extends Animal {
    void sound() { System.out.println("Meow"); }
    void climb() { System.out.println("Cat climbs"); }
}
public class Upcast {
    public static void main(String[] args) {
        Animal a = new Cat();       // upcasting (implicit)
        a.sleep();                  // inherited method
        a.sound();                  // overridden: Cat's version runs
        // a.climb();               // compile error: not in Animal
    }
}
```

Output

```
Animal sleeps
Meow
```

Example. Through the `Animal` reference only `sleep()` and `sound()` are visible; `sound()` runs the `Cat` version, but `climb()` is hidden until a downcast is performed.

Q10. Write a Java program to demonstrate downcasting of a superclass reference to a subclass reference.

5 Marks · CO3

```
class Animal { void eat() { System.out.println("Animal eats"); } }
class Dog extends Animal { void bark() { System.out.println("Dog barks"); } }
class Cat extends Animal { }

public class Downcast {
    public static void main(String[] args) {
        Animal a = new Dog();            // upcast
        Dog d = (Dog) a;                 // downcast (explicit)
        d.eat();
        d.bark();

        Animal b = new Cat();
        if (b instanceof Dog) {
            ((Dog) b).bark();
        } else {
            System.out.println("b is not a Dog; cast avoided");
        }
        Dog bad = (Dog) b;               // ClassCastException
    }
}
```

Output

```
Animal eats
Dog barks
b is not a Dog; cast avoided
Exception in thread "main" java.lang.ClassCastException: Cat cannot be cast to Dog
```

> **Example.** The first cast works because the object really is a Dog. The check `instanceof` prevents an invalid cast; the last line has no check and fails at run time.

Q11. Develop a Java program containing Employee, Manager, and Developer classes where each subclass overrides a method calculateSalary().

5 Marks · CO3

```
class Employee {
    String name; double basic;
    Employee(String n, double b) { name = n; basic = b; }
    double calculateSalary() { return basic; }
}
class Manager extends Employee {
    Manager(String n, double b) { super(n, b); }
    @Override double calculateSalary() { return basic + 0.40 * basic; }   // 40% allowance
}
class Developer extends Employee {
    Developer(String n, double b) { super(n, b); }
    @Override double calculateSalary() { return basic + 0.25 * basic + 5000; }
}
class Payroll {
    public static void main(String[] args) {
        Employee[] e = { new Employee("Clerk", 20000),
                         new Manager("Mita", 50000),
                         new Developer("Joy", 40000) };
        for (Employee x : e)
            System.out.println(x.name + " : " + x.calculateSalary());
    }
}
```

Output

```
Clerk : 20000.0
Mita : 70000.0
Joy : 55000.0
```

> **Example.** Manager: 50000 + 20000 = 70000. Developer: 40000 + 10000 + 5000 = 55000. One loop handles all kinds of employees (polymorphism).

Q12. Write a Java program to model a banking system using Account as the superclass and SavingsAccount and CurrentAccount as subclasses.

5 Marks · CO3

```
class Account {
    int accNo; double balance;
    Account(int n, double b) { accNo = n; balance = b; }
    void deposit(double amt) { balance += amt; }
    void withdraw(double amt) {
        if (amt <= balance) balance -= amt; else System.out.println("Insufficient funds");
    }
    void show() { System.out.println(accNo + " : " + balance); }
}
class SavingsAccount extends Account {
    SavingsAccount(int n, double b) { super(n, b); }
    void addInterest() { balance += balance * 0.04; }
}
class CurrentAccount extends Account {
    double overdraft = 5000;
    CurrentAccount(int n, double b) { super(n, b); }
    @Override void withdraw(double amt) {
        if (amt <= balance + overdraft) balance -= amt;
        else System.out.println("Overdraft limit exceeded");
    }
}
class Bank {
    public static void main(String[] args) {
        SavingsAccount s = new SavingsAccount(1, 10000);
        s.deposit(2000); s.addInterest(); s.show();
        CurrentAccount c = new CurrentAccount(2, 3000);
        c.withdraw(6000); c.show();
        c.withdraw(5000);
    }
}
```

Output

```
1 : 12480.0
2 : -3000.0
Overdraft limit exceeded
```

> **Example.** Savings: (10000 + 2000) x 1.04 = 12480. Current: 3000 - 6000 = -3000, allowed by the 5000 overdraft; a further 5000 would exceed it.

Q13. Develop a Java program to demonstrate how multiple inheritance can be achieved in Java using interfaces.

5 Marks · CO3

```
interface Printable { void print(); }
interface Showable  { void show(); }

class Document implements Printable, Showable {     // two parents
    public void print() { System.out.println("Printing document"); }
    public void show()  { System.out.println("Showing document"); }

    public static void main(String[] args) {
        Document d = new Document();
        d.print();
        d.show();
        Printable p = d;  p.print();        // interface reference
        Showable  s = d;  s.show();
    }
}
```

Output

```
Printing document
Showing document
Printing document
Showing document
```

> **Example.** `Document` inherits the two abstract methods from two interfaces and supplies the bodies itself, so no ambiguity can arise. This is Java's way of multiple inheritance.

Q14. Write a Java program to implement an inheritance hierarchy for Animal, Dog, and Cat, and demonstrate overridden methods.

5 Marks · CO3

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
    void eat()   { System.out.println("Animal eats"); }
}
class Dog extends Animal {
    @Override void sound() { System.out.println("Dog barks: Woof"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Cat meows: Meow"); }
    @Override void eat()   { System.out.println("Cat eats fish"); }
}
class Zoo {
    public static void main(String[] args) {
        Animal[] zoo = { new Animal(), new Dog(), new Cat() };
        for (Animal a : zoo) { a.sound(); a.eat(); }
    }
}
```

Output

```
Animal makes a sound
Animal eats
Dog barks: Woof
Animal eats
```

```
Cat meows: Meow
Cat eats fish
```

> **Example.** `Dog` overrides only `sound()` and inherits `eat()`; `Cat` overrides both.

Q15. Develop a Java program to demonstrate dynamic method dispatch using a superclass reference and objects of different subclasses.

5 Marks · CO3

```
class Shape { double area() { return 0; } }
class Circle extends Shape {
    double r; Circle(double r) { this.r = r; }
    double area() { return 3.14 * r * r; }
}
class Square extends Shape {
    double s; Square(double s) { this.s = s; }
    double area() { return s * s; }
}
class Triangle extends Shape {
    double b, h; Triangle(double b, double h) { this.b = b; this.h = h; }
    double area() { return 0.5 * b * h; }
}
public class Dispatch {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle(2), new Square(4), new Triangle(6, 5) };
        for (Shape s : shapes)
            System.out.println(s.getClass().getSimpleName() + " area = " + s.area());
    }
}
```

Output

```
Circle area = 12.56
Square area = 16.0
Triangle area = 15.0
```

> **Example.** All three objects are held by `Shape` references, yet each `area()` call executes the subclass version, selected at run time.

Bloom's Level: Analyze (L4)

Q1. Analyze whether Car-Vehicle and Car-Engine represent an IS-A or HAS-A relationship. Justify.

5 Marks · CO4

Car-Vehicle: IS-A. The test "A Car is a Vehicle" is true. A car has all the properties and behaviour of a vehicle (moves, has speed) and is a specialised kind of it. It is therefore correctly modelled by inheritance: `class Car extends Vehicle`.

Car-Engine: HAS-A. The test "A Car is an Engine" is false; "A Car has an Engine" is true. The engine is a **part** of the car, not a type of car. It is correctly modelled by composition: a field `Engine engine` in `Car`.

```
class Vehicle { void move() { } }
class Engine  { void start() { } }
class Car extends Vehicle {            // IS-A
    private Engine engine = new Engine();   // HAS-A
}
```

> **Example.** Using inheritance for Car-Engine would let a Car be passed where an Engine is expected and would expose engine methods as car methods, producing a wrong and fragile design.

Q2. Analyze why Java does not support multiple inheritance through classes. Discuss the ambiguity when two parent classes contain methods with the same signature.

5 Marks · CO4

Java forbids `class C extends A, B` to avoid the **diamond problem**. Suppose both A and B define `void show()` with different bodies and C does not override it. In `C c = new C(); c.show();` the compiler cannot know whether A's or B's `show()` is meant; any rule (first parent wins, last parent wins) would be arbitrary and would break when a parent is changed. A further issue arises when A and B both extend D: C would inherit D's members twice, so it is unclear which copy of D's fields to use and how D's constructor should run.

The designers therefore chose compile-time rejection, which keeps the language simple and programs predictable. The need is met by interfaces, where a class `implements` many interfaces. If two interfaces supply the same default method, the compiler forces the class to override it, and the class may name one explicitly with `I1.super.show()`.

```
interface I1 { default void show() { System.out.println("I1"); } }
interface I2 { default void show() { System.out.println("I2"); } }
class C implements I1, I2 {
    public void show() { I1.super.show(); }   // conflict resolved by the class
}
```

> **Example.** Output of `new C().show()` is `I1`. Without the override in C the code would not compile; the ambiguity is turned into a compile-time error instead of a hidden run-time surprise.

Q3. Analyze the difference between inheritance and composition for representing a Car and an Engine. Justify which relationship is conceptually appropriate.

5 Marks · CO4

Aspect	Inheritance (Car extends Engine)	Composition (Car has an Engine)
Semantic test	"Car is an Engine": false	"Car has an Engine": true
Coupling	Very tight; Car depends on Engine's internals	Loose; Car uses only Engine's public methods
Flexibility	Cannot replace the engine at run time	Engine can be replaced (petrol, diesel, electric)
Interface exposed	Car would expose start(), rev(), which are not car operations	Car decides what to expose

Aspect	Inheritance (Car extends Engine)	Composition (Car has an Engine)
Substitutability	A Car could be used where an Engine is required: wrong	Not applicable: no false substitution

Conclusion. Composition is the appropriate model. A car is not a kind of engine; the engine is a component. Composition also gives flexibility, for example `Car(Engine e)` can accept an `ElectricEngine` later. The guideline "favour composition over inheritance unless a true IS-A relationship exists" applies.

```
class Car {
    private Engine engine;
    Car(Engine e) { engine = e; }
    void start() { engine.start(); }
}
```

Q4. Analyze a multilevel hierarchy Person -> Employee -> Manager with respect to code reusability, coupling and maintainability.

5 Marks · CO4

Reusability (strength). `Person` fields (name, age) are written once and reused by Employee and Manager; Employee's salary and ID are reused by Manager. New levels only add what is new.

Coupling (weakness). Each level is tightly coupled to the one above. Manager depends on Employee's behaviour, and indirectly on Person's. A change in `Person` (for example the type of `name`) ripples down to every descendant, and `Manager` cannot be understood without reading three classes.

Maintainability. A fix in `Person.show()` is automatically available to all (positive), but a careless change can break subclasses (the fragile base class problem). Depth makes debugging harder: to find which `calculatePay()` runs, one must trace upward through the chain. Constructors must also be chained correctly (`super(...)` at every level).

Factor	Effect of the 3-level chain
Reusability	High: common code once at the top
Coupling	High: lower classes depend on all ancestors
Maintainability	Good for small stable chains; poor if the hierarchy grows deep or the base changes often

> **Example.** Adding `age` to `Person` immediately gives Employee and Manager an age, with no new code, but changing `Person`'s constructor signature breaks both child constructors.

Q5. A superclass reference points to a subclass object. Analyze which overridden method is executed and explain the role of dynamic method dispatch.

5 Marks · CO4

Analysis. The **subclass** version of the overridden method is executed. The compiler checks only that the method exists in the reference's (static) type; the actual selection of the body is postponed to run time and based on the **object's** (dynamic) type.

Role of dynamic method dispatch. The JVM looks at the real class of the object at the moment of the call and invokes the most specific overriding method found there (searching upward only if the subclass did not override it). This enables run-time polymorphism.

```
class A { void f() { System.out.println("A.f"); } void g() { System.out.println("A.g"); } }
class B extends A { void f() { System.out.println("B.f"); } }
// A r = new B();  r.f();  r.g();
```

Output

```
B.f
A.g
```

> **Example.** `r.f()` runs B's overriding version; `r.g()` is not overridden, so A's version runs. Fields and static methods, in contrast, are resolved by the reference type, not dynamically.

Q6. Analyze the design: Shape superclass and Circle, Rectangle, Triangle subclasses. Explain how overriding supports run-time polymorphism.

5 Marks · CO4

`Shape` declares `double area()`. Each subclass overrides it with its own formula. A client can then be written against `Shape` only:

```
double total(Shape[] list) {
    double sum = 0;
    for (Shape s : list) sum += s.area();     // which area()? decided at run time
    return sum;
}
```

Analysis.

- The loop has no knowledge of circles, rectangles or triangles; the same code works for any kind of shape.
- At each iteration the JVM dispatches `area()` to the actual object's class (Circle, Rectangle or Triangle).
- A new subclass, say `Pentagon`, can be added without changing `total()`, so the design is **open for extension and closed for modification**.
- Overriding provides the different behaviours; the superclass reference with dynamic dispatch provides the single uniform call.

> **Example.** For {Circle(1), Rectangle(2,3), Triangle(4,5)} with 3.14, the total is 3.14 + 6 + 10 = 19.14, computed by three different `area()` bodies from one call.

Q7. Analyze the role of the superclass constructor when a subclass object is created. Explain the order in which the superclass and subclass constructors are executed.

5 Marks · CO4

A subclass object contains the fields of its superclass as well as its own. The superclass constructor is responsible for initialising the **inherited part**, so it must run before the subclass constructor initialises the new part. Each subclass constructor therefore begins with a call to a superclass constructor: either explicit `super(...)` or an implicit `super()` inserted by the compiler (which fails to compile if the parent has no no-argument constructor).

Order. For `C extends B extends A`, `new C()` executes: (1) `C()` is entered, which immediately calls `B()`; (2) `B()` calls `A()`; (3) `A()` body executes first; (4) then `B()` body; (5) finally `C()` body. Execution therefore proceeds from the top of the hierarchy down to the subclass.

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
class C extends B { C() { System.out.println("C"); } }
// new C();
```

Output

```
A
B
C
```

Q8. Analyze the statement: “Constructors are not inherited, but a superclass constructor is involved when a subclass object is created.” Justify with an example.

5 Marks · CO4

Both halves are true.

- **Not inherited:** the constructor `Person(String)` is not a member of `Student`; `new Student("Ravi")` is illegal unless `Student` declares its own matching constructor.
- **Still involved:** a `Student` object contains a `Person` part. Student’s constructor must call a `Person` constructor (using `super(...)`) to initialise that part before its own code runs.

```
class Person {
    String name;
    Person(String name) { this.name = name; System.out.println("Person()"); }
}
class Student extends Person {
    int roll;
    Student(String name, int roll) {
        super(name);                    // involves superclass constructor
        this.roll = roll;
        System.out.println("Student()");
    }
    public static void main(String[] a) {
        Student s = new Student("Ravi", 4);
        // Student t = new Student("Ravi");   // error: no such constructor
    }
}
```

Output

```
Person()
Student()
```

> **Example.** The call is a **use** of the parent’s constructor, not an inheritance of it: `Student` has exactly one constructor, the one it declares.

Q9. Analyze the use of upcasting and downcasting in an inheritance hierarchy. Discuss the conditions under which downcasting may cause a run-time exception.

5 Marks · CO4

Upcasting gives generality: objects of many subclasses can be stored in one array or passed to one method that accepts the superclass, and overridden methods dispatch correctly. It loses direct access to subclass-only members.

Downcasting recovers specificity: it lets the program use members that exist only in the subclass, but the compiler cannot verify it, so the check is made at run time.

A `ClassCastException` occurs when **the object referred to is not** an instance of the target subclass (or its subclass). The conditions are:

- the object was created as the superclass itself (`Animal a = new Animal(); (Dog) a`);
- the object belongs to a **sibling** class (`Animal a = new Cat(); (Dog) a`);
- the reference was obtained from a generic source (a collection or method returning `Object`) and the real type differs from the assumed one.

Note that `(Dog) null` succeeds, giving null. The standard safeguard is `if (a instanceof Dog) { Dog d = (Dog) a; ... }`.

> **Example.** `Animal[] zoo = {new Dog(), new Cat()};` Casting `zoo[1]` to Dog fails with an exception, whereas casting `zoo[0]` to Dog succeeds.

Q10. Analyze the problems of extending a large hierarchy Employee -> Manager -> SeniorManager -> Director further, and discuss how composition or interfaces could provide greater flexibility.

5 Marks · CO4

Problems of a deep hierarchy.

- **Rigidity:** a change in `Employee` affects all three levels below (fragile base class).
- **Tight coupling:** Director depends on the behaviour of four classes; it is hard to understand and test.
- **Class explosion:** each new combination of roles (for example a technical director, part-time manager) needs yet another subclass.
- **Wrong abstractions:** roles change over time; a Manager promoted to Director would require destroying and re-creating the object, since an object's class cannot change.
- **Single-inheritance limit:** a class cannot inherit from two branches (for example Director and Trainer).

Alternatives.

- **Composition:** model the role as a part. `Employee` has a `Role` object (`ManagerRole`, `DirectorRole`) that can be replaced at run time when promoted.
- **Interfaces:** define capabilities such as `Approver`, `Trainer`, `Reportable`; a class implements any combination, bringing multiple inheritance of type without state.

```
interface Role { double bonus(); }
class Employee { String name; Role role;  // promoted by: e.role = new DirectorRole();
    double pay(double basic) { return basic + role.bonus(); } }
```

> **Example.** `e.role = new DirectorRole();` promotes the same employee object without altering any class, which a deeper hierarchy cannot do.

Q11. Analyze whether a class hierarchy should be used for Student, Address, and Department. Identify the appropriate IS-A/HAS-A relationships and justify.

5 Marks · CO4

Tests.

- "A Student is an Address": false. "A Student has an Address": true. **HAS-A (composition)**: a student's address is a part of the student's details and is normally created with the student.
- "A Student is a Department": false. "A Department has Students": true. **HAS-A (aggregation)**: a department holds many students, but a student exists independently of any one department.
- "An Address is a Department" or the reverse: false, and no relationship other than possibly HAS-A (a Department has an Address).

Conclusion. No inheritance hierarchy is justified among these three classes, since none is a specialised kind of another. Using `extends` would produce meaningless substitutions (a student treated as an address). All relationships are HAS-A, implemented by fields.

```
class Address { String city, pin; }
class Student { int roll; Address address; }        // composition
class Department { String name; ArrayList<Student> students; Address address; }
```

A valid IS-A case would be `class GraduateStudent extends Student`.

Q12. A superclass has a method display() and two subclasses override it. Analyze the output when objects of the subclasses are accessed through a superclass reference.

5 Marks · CO4

```
class Base { void display() { System.out.println("Base display"); } }
class Sub1 extends Base { void display() { System.out.println("Sub1 display"); } }
class Sub2 extends Base { void display() { System.out.println("Sub2 display"); } }
class Test {
    public static void main(String[] args) {
        Base b;
        b = new Sub1();  b.display();
        b = new Sub2();  b.display();
        b = new Base();  b.display();
    }
}
```

Output

```
Sub1 display
Sub2 display
Base display
```

Analysis. The reference `b` has the static type `Base`; the compiler accepts `b.display()` because `Base` declares it. At run time, the JVM invokes the version belonging to the **object** `b` points to. In the first call the object is a `Sub1`, in the second a `Sub2`, and in the third a plain `Base`. If `Sub2` had not overridden `display()`, the second line would have printed "Base display", the inherited version.

MODULE

6

Abstract Classes and Interfaces

CA2: abstract and final, interfaces, multiple inheritance, run-time polymorphism and late binding

Bloom's Level: Remember (L1)

Q1. Define an abstract class in Java.

2 Marks · CO1

An abstract class is a class declared with the keyword `abstract`. It may contain abstract methods (without body) as well as concrete methods, fields and constructors, but it cannot be instantiated. It is meant to be extended by subclasses that complete it.

> **Example.** `abstract class Shape { abstract double area(); }`

Q2. What is an abstract method?

2 Marks · CO1

An abstract method is a method declared with the `abstract` keyword and without a body, ending in a semicolon. It only gives the signature; every concrete subclass must implement it.

> **Example.** `abstract void start();`

Q3. What is a concrete class?

2 Marks · CO1

A concrete class is a class that has implementations for all of its methods (including any inherited abstract ones) and can therefore be instantiated with `new`.

> **Example.** `class Circle extends Shape { double area() { return 3.14 * r * r; } }` is concrete if it implements `area()`.

Q4. What is a final class?

2 Marks · CO1

A final class is declared with the keyword `final` and cannot be extended. It is used to prevent alteration of its behaviour through inheritance, for security or immutability.

> **Example.** `final class Constants { };` `java.lang.String` and the wrapper classes are final.

Q5. What is a final method?

2 Marks · CO1

A final method is declared with `final` and cannot be overridden in any subclass, although it can be inherited and called.

> **Example.** `final void showId() { System.out.println(id); }`

Q6. What is an interface in Java?

2 Marks · CO1

An interface is a reference type that specifies a set of abstract methods (and constants) which a class must implement. It describes **what** a class can do without saying how, and is used to achieve full abstraction and multiple inheritance of type.

> **Example.** `interface Printable { void print(); }`

Q7. What is method overriding?

2 Marks · CO1

Method overriding is the redefinition, in a subclass (or implementing class), of a method of the superclass (or interface) with the same name and parameters, so that the subclass gives its own behaviour.

> **Example.** `class Dog extends Animal { void sound() { System.out.println("Bark"); } }`

Q8. What is dynamic method dispatch?

2 Marks · CO1

It is the run-time process of deciding which overridden method is to be called, according to the actual object referred to by a superclass or interface reference.

> **Example.** `Shape s = new Circle(); s.area();` runs `Circle.area()`.

Q9. What is run-time polymorphism?

2 Marks · CO1

Run-time polymorphism is the ability of a single call through a superclass or interface reference to behave differently according to the object actually referred to. It is implemented by method overriding and dynamic method dispatch.

> **Example.** `Animal a = new Cat(); a.sound();` prints “Meow”, and after `a = new Dog();` the same call prints “Bark”.

Q10. What is multiple inheritance?

2 Marks · CO1

Multiple inheritance is the ability of a class to inherit from more than one parent. Java does not allow it for classes, but permits it for interfaces: a class may implement many interfaces.

> **Example.** `class Duck implements Flyable, Swimmable { }`

Q11. Which keyword is used to declare an abstract class?

2 Marks · CO1

The keyword `abstract`, placed before `class`: `abstract class Vehicle { }`.

Q12. Which keyword is used to declare an abstract method?

2 Marks · CO1

The keyword `abstract`, placed in the method declaration, which ends with a semicolon instead of a body: `abstract void run();`.

Q13. Which keyword prevents a class from being inherited?

2 Marks · CO1

The keyword `final`: `final class A { }` cannot be extended.

Q14. Which keyword prevents a method from being overridden?

2 Marks · CO1

The keyword `final`: a method declared `final void f() { }` cannot be overridden by subclasses.

Q15. Which keyword is used by a class to implement an interface?

2 Marks · CO1

The keyword `implements`: `class Student implements Printable { ... }`. (An interface extending another interface uses `extends`.)

Q16. Can an abstract class be instantiated directly?

2 Marks · CO1

No. `new Shape();` for an abstract class `Shape` is a compile-time error, because the class may be incomplete. It can only be instantiated through a concrete subclass, and a reference of the abstract type may hold such an object.

> **Example.** `Shape s = new Circle();` is valid.

Q17. Can an abstract class have a constructor?

2 Marks · CO1

Yes. Although the abstract class cannot be instantiated, its constructor exists to initialise the fields of the abstract class, and is executed when a subclass object is created (through `super(...)`).

Q18. Can an abstract class contain concrete methods?

2 Marks · CO1

Yes. An abstract class can have any number of concrete (implemented) methods along with abstract ones. This allows common behaviour to be shared by all subclasses.

> **Example.** `abstract class Employee { void showId() { ... } abstract double salary(); }`

Q19. Can an abstract method have a method body?

2 Marks · CO1

No. An abstract method must end with a semicolon and cannot have a body (`abstract void f() { }` is an error). If a body is needed, the method must be concrete.

Q20. Can an abstract method be declared final?

2 Marks · CO1

No. `abstract` demands that subclasses override the method, while `final` forbids overriding; the two contradict each other, so the combination is a compile-time error.

Q21. Can an abstract method be declared static?

2 Marks · CO1

No. A static method belongs to the class and is not overridden, but an abstract method has no body and must be overridden to be useful. Hence `abstract static` is illegal.

Q22. Can a final class be inherited?

2 Marks · CO1

No. A final class cannot be extended; `class B extends A` is an error if `A` is final.

Q23. Can a final method be overridden?

2 Marks · CO1

No. A final method may be inherited and called but any attempt to override it is a compile-time error.

Q24. Can an interface have a constructor?

2 Marks · CO1

No. An interface cannot be instantiated and holds no instance state to initialise, so it has no constructors.

Q25. Can a class implement more than one interface?

2 Marks · CO1

Yes. The interface names are separated by commas: `class Duck implements Flyable, Swimmable { }`. The class must implement all their abstract methods (or be abstract).

Q26. Can an interface extend more than one interface?

2 Marks · CO1

Yes. An interface may extend any number of interfaces: `interface C extends A, B { }`. This is permitted because interfaces carry no implementation state.

Q27. Does Java support multiple inheritance through classes?

2 Marks · CO1

No. A class can extend only one class. Multiple inheritance is available only through interfaces.

Q28. What is an interface reference?

2 Marks · CO1

An interface reference is a variable whose type is an interface. It can refer to an object of any class that implements that interface and gives access to the interface's methods.

> **Example.** `Printable p = new Student(); p.print();`

Q29. What is late binding?

2 Marks · CO1

Late (dynamic) binding means that the association of a method call with the method body is made at **run time**, not at compile time. In Java, calls to overridden instance methods are bound late, based on the actual object.

Q30. What is a marker interface?

2 Marks · CO1

A marker interface is an interface with no methods or constants; it merely marks a class as having some capability, which is checked at run time with `instanceof`.

Example. `Serializable` and `Cloneable` are marker interfaces. `class Emp implements Serializable { }` indicates that `Emp` objects may be serialised.

Bloom's Level: Understand (L2)

Q1. Explain the concept of an abstract class with a suitable example.

3 Marks · CO2

An abstract class represents a concept that is general and incomplete, such as Shape or Vehicle, for which real objects do not exist. It carries what is common to all its subclasses (fields, constructors and concrete methods) and declares abstract methods for what must be provided differently by each subclass. It cannot be instantiated; a subclass that implements all abstract methods is concrete.

```
abstract class Shape {
    String name;
    Shape(String n) { name = n; }
    abstract double area();                    // no body
    void show() { System.out.println(name + " area = " + area()); }   // concrete
}
class Circle extends Shape {
    double r;
    Circle(double r) { super("Circle"); this.r = r; }
    double area() { return 3.14 * r * r; }
}
// new Circle(2).show();  prints  Circle area = 12.56
```

Example. `Shape s = new Shape("x");` fails to compile; `Shape s = new Circle(2);` works.

Q2. Explain the need for abstract classes and abstract methods.

3 Marks · CO2

They are needed when a class defines a **general concept** whose details cannot be known until a specific subclass exists.

- **Common code in one place:** shared fields and methods are written once in the abstract class.
- **Compulsion to implement:** an abstract method forces every concrete subclass to provide its own version, so no subclass can forget it.
- **Prevents meaningless objects:** an object of a plain "Shape" or "Employee" has no meaning; abstractness forbids it.
- **Supports polymorphism:** a reference of the abstract type can hold any subclass object, and calls are dispatched at run time.

Example. `abstract double area();` in `Shape` cannot have a formula (a circle and a rectangle differ), yet every shape must have an area; abstract declaration guarantees it.

Q3. Explain why an abstract class cannot be instantiated directly.

3 Marks · CO2

An abstract class may contain abstract methods without bodies. If an object of it could be created, a call to such a method would have nothing to execute. The compiler prevents this by rejecting `new` on abstract classes. The class is intended only as a base: it gives a template, and a subclass that overrides every abstract method completes the picture and can be instantiated. Note that a reference of the abstract type is allowed; it simply points to a subclass object.

```
abstract class Vehicle { abstract void start(); }
// Vehicle v = new Vehicle();          // error: Vehicle is abstract
// Vehicle v = new Car();              // OK (Car implements start())
```

Q4. Explain the role of a constructor in an abstract class.

3 Marks · CO2

Although an abstract class is never instantiated by itself, it often has fields that need initialisation. Its constructor does this. The constructor runs when an object of a concrete subclass is created, because every subclass constructor calls it, explicitly with `super(...)` or implicitly. Thus the abstract class's part of the object is initialised first, as in any inheritance chain.

```
abstract class Employee {
    String name;
    Employee(String n) { name = n; System.out.println("Employee constructor"); }
    abstract double pay();
}
class Clerk extends Employee {
    Clerk(String n) { super(n); System.out.println("Clerk constructor"); }
    double pay() { return 15000; }
}
// new Clerk("Ali");   prints  Employee constructor  then  Clerk constructor
```

Q5. Explain how a subclass implements an abstract method.

3 Marks · CO2

A concrete subclass provides a method with the **same name, same parameter list and a compatible return type** as the abstract method, and with a body. The access level must be the same or wider. If the subclass does not implement all inherited abstract methods, it must itself be declared abstract. The annotation `@Override` helps the compiler verify the signature.

```
abstract class Animal { abstract void sound(); }
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Woof"); }     // implementation
}
```

> **Example.** If `Dog` omitted `sound()` the compiler reports "Dog is not abstract and does not override abstract method sound()".

Q6. Differentiate between an abstract class and a concrete class.

3 Marks · CO2

Basis	Abstract Class	Concrete Class
Keyword	Declared with `abstract`	Declared without `abstract`
Instantiation	Cannot be instantiated	Can be instantiated with `new`
Methods	May have abstract and concrete methods	All methods must have bodies
Purpose	Base for subclasses, defines a template	Complete, usable class
Subclass duty	Subclass must implement the abstract methods or be abstract itself	None
Example	`abstract class Shape`	`class Circle extends Shape`

Q7. Explain the difference between an abstract class and an interface.

3 Marks · CO2

Basis	Abstract Class	Interface
Keyword	`abstract class`, extended with `extends`	`interface`, implemented with `implements`
Methods	Abstract and concrete	Abstract by default; `default` and `static` since Java 8
Variables	Any kind of instance and static variables	Only `public static final` constants
Constructors	Allowed	Not allowed
Multiple inheritance	A class extends only one	A class implements many
Access of members	Any modifier	Methods are `public`
Use	IS-A with shared state and code	A capability or contract (CAN-DO)

```
abstract class Vehicle { int speed; abstract void run(); void stop() { } }
interface Chargeable { void charge(); }
```

Q8. Explain the purpose of a final class.

3 Marks · CO2

A final class cannot be subclassed. It is used (i) to **guarantee that behaviour cannot be altered** by a subclass, which is important for security-critical classes; (ii) to make a class **immutable**, because nobody can add mutable state through inheritance; and (iii) to let the compiler and JVM optimise. `java.lang.String` and the wrapper classes such as `Integer` are final for these reasons.

```
final class Account { }
// class Savings extends Account { }     // error: cannot inherit from final Account
```

Q9. Explain the purpose of a final method.

3 Marks · CO2

A final method may not be overridden. It is used when the method's implementation is **critical to the class's correctness** and must stay the same in all subclasses, for example a method that applies a fixed rule or a step of a template algorithm. The method can still be inherited and called by subclasses.

```
class Exam {
    final void instructions() { System.out.println("No mobiles allowed"); }
    void paper() { System.out.println("General paper"); }
}
class Final extends Exam {
    // void instructions() { }       // error: cannot override final method
    void paper() { System.out.println("Final paper"); }   // allowed
}
```

Q10. Differentiate between a final class and a final method.

3 Marks · CO2

Basis	Final Class	Final Method
Prevents	Inheritance of the whole class	Overriding of one method
Subclassing	Not possible	Possible
Other methods	None can be overridden (no subclass exists)	Other methods can be overridden
Declaration	`final class A { }`	`final void f() { }`
Example	`String`	`Object.getClass()`
Use	Strong immutability or security	Fix one critical behaviour but allow extension

Q11. Explain why an abstract class cannot be declared as final.

3 Marks · CO2

`abstract` and `final` have opposite aims. An abstract class is **incomplete** and is meant to be extended so that the abstract methods can be implemented. A final class **cannot be extended**. If both were allowed, the class could never be completed or used, since it cannot be instantiated and cannot have a subclass. The compiler therefore reports an error: "illegal combination of modifiers: abstract and final".

```
// abstract final class A { }       // compile-time error
```

Q12. Explain why an abstract method cannot be declared as final.

3 Marks · CO2

An abstract method has no body and exists so that subclasses can override it. A final method cannot be overridden. A method that must be overridden and also must not be overridden is a contradiction, so the compiler rejects the combination.

```
abstract class A {
    // abstract final void f();       // error: illegal combination of modifiers
    abstract void f();                // correct: subclasses supply the body
```

```
}
```

Q13. Explain why an abstract method cannot be declared as static.

3 Marks · CO2

A static method is bound to the class, is resolved at compile time and cannot be overridden (only hidden). An abstract method must be overridden in a subclass and is resolved at run time. Moreover, a static method could be called as `A.f()` without any object, but an abstract method has no body to execute. Because the two properties conflict, `abstract static` is a compile-time error.

```
abstract class A {
    // abstract static void f();      // error
    static void g() { }               // allowed, with a body
}
```

Q14. Explain why an abstract method cannot be declared as private.

3 Marks · CO2

A private method is visible only inside its own class and is therefore never inherited or overridden. An abstract method, on the other hand, must be visible in the subclass so that it can be implemented there. If it were private, the subclass could not see or implement it and the class could never become concrete. Hence `abstract private` is illegal; abstract methods must be `public`, `protected` or default.

```
abstract class A {
    // abstract private void f();     // error
    protected abstract void f();      // correct
}
```

Q15. Explain the concept of an interface with a suitable example.

3 Marks · CO2

An interface is a contract that lists methods a class promises to provide, without implementing them. All methods are `public abstract` unless declared `default` or `static`, and all variables are `public static final`. A class that `implements` the interface must define all its abstract methods. Interfaces give 100% abstraction, loose coupling and multiple inheritance of type.

```
interface Printable {
    int MAX = 10;                  // public static final
    void print();                  // public abstract
}
class Student implements Printable {
    public void print() { System.out.println("Printing student"); }
    public static void main(String[] a) { new Student().print(); }
}
```

Output

```
Printing student
```

Q16. Explain how a class implements an interface.

3 Marks · CO2

The class uses the `implements` clause in its header and defines **every** abstract method of the interface with the same signature and the `public` modifier (interface methods are public, and an overriding method cannot reduce access). If it leaves any method undefined, the class must be declared `abstract`.

```
interface Drawable { void draw(); }
class Circle implements Drawable {
    public void draw() { System.out.println("Circle drawn"); }
}
class Test { public static void main(String[] a) { Drawable d = new Circle(); d.draw(); } }
```

> **Example.** Omitting `public` in `void draw()` of `Circle` causes the error “attempting to assign weaker access privileges”.

Q17. Explain how interfaces support abstraction in Java.

5 Marks · CO2

Abstraction means showing **what** an object does and hiding **how** it does it. Interfaces support it completely:

- **Pure contract:** an interface lists only method signatures; the user of an interface sees the operations but not the implementation.
- **Separation of specification and implementation:** the code using `Payment` depends only on the interface, not on `UPI` or `CreditCard`; the implementation may be replaced without changing the client.
- **Loose coupling:** client classes hold interface references, so they are independent of concrete classes.
- **Multiple abstractions:** a class can implement several interfaces, presenting different views of itself to different clients.
- **Polymorphism:** an interface reference can hold any implementer, with dynamic dispatch selecting the right behaviour.

```
interface Payment { void pay(double amount); }
class UPI implements Payment {
    public void pay(double a) { System.out.println("Paid " + a + " by UPI"); }
}
class Card implements Payment {
    public void pay(double a) { System.out.println("Paid " + a + " by card"); }
}
class Shop {
    static void checkout(Payment p, double amt) { p.pay(amt); }   // knows only the interface
    public static void main(String[] args) {
        checkout(new UPI(), 500);
        checkout(new Card(), 1200);
    }
}
```

Output

```
Paid 500.0 by UPI
Paid 1200.0 by card
```

> **Example.** `checkout()` never learns how payment is made; this hiding of detail is abstraction. A new `NetBanking` class can be added without altering `checkout()`.

Q18. Explain how interfaces can be used to achieve multiple inheritance in Java.

3 Marks · CO2

Because interfaces have no instance state and (traditionally) no method bodies, a class can safely implement several of them. The class thereby inherits the **types and method contracts** of all of them, which is multiple inheritance of type, and supplies the single implementation of each method itself, so that there is no ambiguity.

```
interface Flyable   { void fly(); }
interface Swimmable { void swim(); }
class Duck implements Flyable, Swimmable {
    public void fly()  { System.out.println("Duck flies"); }
    public void swim() { System.out.println("Duck swims"); }
}
```

> **Example.** A `Duck` object can be assigned to both a `Flyable` and a `Swimmable` reference.

Q19. Explain why Java does not support multiple inheritance through classes.

3 Marks · CO2

Java does not allow `class C extends A, B` because of ambiguity and complexity. If both A and B have a method `show()` with different bodies, an object of C would have two competing versions, and Java cannot decide which to use. If A and B themselves extend a common class, the diamond problem causes the inherited state to be duplicated or inconsistent. Constructor invocation order also becomes unclear. To keep the language simple and reliable, Java restricts classes to single inheritance and provides interfaces for the legitimate need.

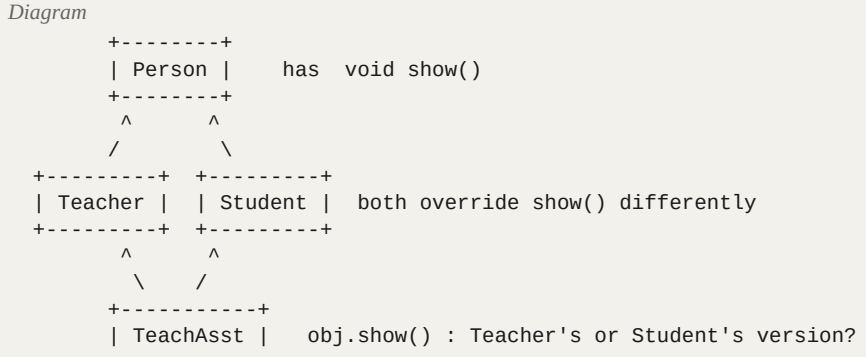
> **Example.** `class C extends A, B { }` gives a compile-time error; `class C implements I1, I2 { }` is allowed.

Q20. Explain the ambiguity problem associated with multiple inheritance.

3 Marks · CO2

The ambiguity (diamond) problem arises when a class inherits from two parents that both provide a member of the same name. When the child calls it, the compiler or runtime cannot tell which parent's member is meant.

Diagram



```
        +-----------+
```

The problem also covers duplicated fields of the common ancestor. Languages like C++ solve it with virtual inheritance and scope resolution; Java avoids it by disallowing multiple inheritance of classes.

Q21. Explain dynamic method dispatch with an interface-based example.

3 Marks · CO2

When an interface reference holds an object of an implementing class and an interface method is called, the JVM selects the implementation belonging to the **actual object** at run time.

```
interface Animal { void sound(); }
class Dog implements Animal { public void sound() { System.out.println("Bark"); } }
class Cat implements Animal { public void sound() { System.out.println("Meow"); } }
class Test {
    public static void main(String[] args) {
        Animal a;
        a = new Dog(); a.sound();
        a = new Cat(); a.sound();
    }
}
```

Output

```
Bark
Meow
```

> **Example.** The same statement `a.sound()` produced two different outputs, depending on the object in `a`.

Q22. Explain why dynamic method dispatch is called run-time polymorphism.

3 Marks · CO2

Polymorphism means one name showing many forms. In dynamic dispatch, one call `ref.method()` takes different forms (executes different bodies) depending on the object. Because this decision cannot be made by the compiler (the same reference may hold different objects while the program runs, perhaps depending on user input) and is made while the program is **running**, it is called run-time polymorphism. It is also called dynamic or late binding.

> **Example.** If the user types “dog” the program does `a = new Dog();`, otherwise `a = new Cat();`. Only at run time is it known which `sound()` will run.

Q23. Explain the relationship between method overriding, late binding and dynamic method dispatch.

3 Marks · CO2

The three terms describe three aspects of one mechanism:

- **Method overriding** is the **programming feature**: a subclass redefines an inherited method with the same signature.
- **Late binding** is the **timing**: the method body to be executed is bound to the call at run time and not at compile time.

- **Dynamic method dispatch** is the **runtime procedure** that carries out the late binding: the JVM looks at the actual object's class and invokes its overriding method.

Overriding supplies the alternatives, late binding says the choice is deferred, and dispatch makes the choice. Together they produce run-time polymorphism.

> **Example.** `Shape s = new Circle(); s.area();` Overriding: `Circle` redefines `area()`. Late binding: the choice is made when the line executes. Dispatch: the JVM calls `Circle.area()`.

Q24. Differentiate between compile-time polymorphism and run-time polymorphism.

3 Marks · CO2

Basis	Compile-Time Polymorphism	Run-Time Polymorphism
Achieved by	Method (and constructor) overloading	Method overriding
Binding	Early / static, by the compiler	Late / dynamic, by the JVM
Decided from	Number and types of arguments	Type of the actual object
Inheritance needed	No	Yes (or interface)
Speed	Slightly faster	Slightly slower (lookup at run time)
Example	`add(int,int)` and `add(double,double)`	`Shape s = new Circle(); s.area();`

Q25. Explain the difference between extends and implements in Java.

3 Marks · CO2

`extends` is used (i) by a class to inherit from another **class** (only one allowed) and (ii) by an interface to inherit from other **interfaces** (many allowed). `implements` is used by a **class** to implement one or more **interfaces**; the class must then define all their abstract methods.

Basis	extends	implements
Used between	Class-class, interface-interface	Class-interface
How many	One class; many interfaces	Many interfaces
Inherits	Fields and methods with bodies	Only method contracts (and constants)

```
class B extends A { }
interface J extends I1, I2 { }
class C extends B implements I1, I2 { }
```

Q26. Explain how a class can implement multiple interfaces.

3 Marks · CO2

The names of the interfaces are written after `implements`, separated by commas. The class must implement all the abstract methods of **every** interface (or be declared abstract). If two interfaces declare the same method signature, one implementation in the class satisfies both.

```
interface Printable { void print(); }
interface Showable  { void show(); }
class Document implements Printable, Showable {
    public void print() { System.out.println("print"); }
    public void show()  { System.out.println("show"); }
}
```

Example. A `Document` object can be referred to by a `Printable` reference or a `Showable` reference.

Bloom's Level: Apply (L3)

Q1. Write a Java program using an abstract class Shape with an abstract method area() and implement it in Circle.

5 Marks · CO3

```
abstract class Shape {
    abstract double area();                         // abstract method
    void display() { System.out.println("Area = " + area()); }   // concrete
}
class Circle extends Shape {
    double radius;
    Circle(double r) { radius = r; }
    @Override double area() { return Math.PI * radius * radius; }
}
public class ShapeDemo {
    public static void main(String[] args) {
        Shape s = new Circle(5);
        System.out.printf("Circle area = %.2f%n", s.area());
        s.display();
    }
}
```

Output

```
Circle area = 78.54
Area = 78.53981633974483
```

Example. Area = 3.14159 x 5 x 5 = 78.54. `Shape s = new Shape();` would not compile.

Q2. Develop a Java program using an abstract class Vehicle with an abstract method start(), implemented by Car and Bike.

5 Marks · CO3

```
abstract class Vehicle {
    String name;
    Vehicle(String name) { this.name = name; }
    abstract void start();
    void stop() { System.out.println(name + " stopped"); }
}
class Car extends Vehicle {
    Car() { super("Car"); }
    void start() { System.out.println("Car starts with a key"); }
}
class Bike extends Vehicle {
    Bike() { super("Bike"); }
    void start() { System.out.println("Bike starts with a kick"); }
}
public class VehicleDemo {
    public static void main(String[] args) {
        Vehicle[] v = { new Car(), new Bike() };
        for (Vehicle x : v) { x.start(); x.stop(); }
    }
}
```

Output

```
Car starts with a key
Car stopped
Bike starts with a kick
Bike stopped
```

> **Example.** `start()` is implemented separately by each subclass, while `stop()` is shared.

Q3. Write a Java program in which an abstract class Employee contains an abstract method calculateSalary(), implemented by Manager and Developer.

5 Marks · CO3

```
abstract class Employee {
    String name; double basic;
    Employee(String n, double b) { name = n; basic = b; }
    abstract double calculateSalary();
    void show() { System.out.println(name + " earns " + calculateSalary()); }
}
class Manager extends Employee {
    double bonus;
    Manager(String n, double b, double bonus) { super(n, b); this.bonus = bonus; }
    double calculateSalary() { return basic + bonus; }
}
class Developer extends Employee {
    int overtimeHours;
    Developer(String n, double b, int h) { super(n, b); overtimeHours = h; }
    double calculateSalary() { return basic + overtimeHours * 500; }
}
public class Pay {
    public static void main(String[] args) {
        new Manager("Rina", 60000, 10000).show();
        new Developer("Tom", 40000, 12).show();
    }
}
```

Output

```
Rina earns 70000.0
Tom earns 46000.0
```

Example. Manager = 60000 + 10000 = 70000; Developer = 40000 + 12 x 500 = 46000.

Q4. Write a Java program demonstrating an abstract class containing both abstract and concrete methods.

5 Marks · CO3

```
abstract class Bank {
    String bankName = "National Bank";
    abstract double interestRate();                 // abstract
    void welcome() {                                // concrete
        System.out.println("Welcome to " + bankName);
    }
    double interest(double principal, int years) {  // concrete, uses abstract
        return principal * interestRate() * years / 100;
    }
}
class SBI extends Bank {
    double interestRate() { return 6.5; }
}
public class BankDemo {
    public static void main(String[] args) {
        Bank b = new SBI();
        b.welcome();
        System.out.println("Interest = " + b.interest(10000, 2));
    }
}
```

Output

```
Welcome to National Bank
Interest = 1300.0
```

Example. The concrete `interest()` calls the abstract `interestRate()`, whose body is supplied later by the subclass (template method idea). 10000 x 6.5 x 2 / 100 = 1300.

Q5. Develop a Java program demonstrating the use of an abstract-class constructor through a subclass object.

5 Marks · CO3

```
abstract class Person {
    String name;
    Person(String name) {
        this.name = name;
        System.out.println("Person constructor: " + name);
    }
    abstract void role();
}
class Teacher extends Person {
    String subject;
    Teacher(String name, String subject) {
        super(name);                          // calls the abstract class constructor
        this.subject = subject;
        System.out.println("Teacher constructor: " + subject);
    }
    void role() { System.out.println(name + " teaches " + subject); }
    public static void main(String[] args) {
        Person p = new Teacher("Mr. Das", "Java");
        p.role();
    }
}
```

Output

```
Person constructor: Mr. Das
Teacher constructor: Java
Mr. Das teaches Java
```

Example. `new Person(..)` is not allowed, but creating a `Teacher` causes the constructor of the abstract class to run first and initialise `name`.

Q6. Write a Java program to define an interface Printable and implement it in a class Student.

5 Marks · CO3

```
interface Printable {
    void print();
}
class Student implements Printable {
    int roll; String name;
    Student(int roll, String name) { this.roll = roll; this.name = name; }
    @Override
    public void print() {
        System.out.println("Roll: " + roll + ", Name: " + name);
    }
    public static void main(String[] args) {
        Student s = new Student(21, "Mohan");
        s.print();
        Printable p = s;          // interface reference
        p.print();
    }
}
```

Output

```
Roll: 21, Name: Mohan
Roll: 21, Name: Mohan
```

> **Example.** `print()` must be `public` in `Student`, since interface methods are public.

Q7. Develop a Java program in which a class implements two interfaces, Printable and Showable.

5 Marks · CO3

```
interface Printable { void print(); }
interface Showable  { void show(); }

class Report implements Printable, Showable {
    String title;
    Report(String t) { title = t; }
    public void print() { System.out.println("Printing report: " + title); }
    public void show()  { System.out.println("Showing report: " + title); }
}
public class TwoInterfaces {
    public static void main(String[] args) {
        Report r = new Report("Annual");
        r.print();
        r.show();
        Printable p = r;  p.print();
        Showable  s = r;  s.show();
    }
}
```

Output

```
Printing report: Annual
Showing report: Annual
Printing report: Annual
Showing report: Annual
```

> **Example.** The class has to define both `print()` and `show()`; omitting either is a compile-time error unless the class is abstract.

Q8. Write a Java program where class Duck implements both Flyable and Swimmable interfaces.

5 Marks · CO3

```
interface Flyable   { void fly(); }
interface Swimmable { void swim(); }

class Duck implements Flyable, Swimmable {
    public void fly()  { System.out.println("Duck flies low"); }
    public void swim() { System.out.println("Duck swims in the pond"); }
    void quack()       { System.out.println("Quack!"); }

    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly(); d.swim(); d.quack();
        Flyable f = d;  f.fly();
        Swimmable s = d;  s.swim();
    }
}
```

Output

```
Duck flies low
Duck swims in the pond
Quack!
Duck flies low
Duck swims in the pond
```

> **Example.** Through the `Flyable` reference only `fly()` is visible; `quack()` is available only on a `Duck` reference.

Q9. Develop a Java program demonstrating multiple inheritance using two interfaces.

5 Marks · CO3

```
interface Father { default String skill() { return "Gardening"; }  int AGE = 50; }
interface Mother { default String talent() { return "Music"; } }

class Child implements Father, Mother {
    void showAll() {
        System.out.println("From Father: " + skill());
        System.out.println("From Mother: " + talent());
        System.out.println("Father's age: " + Father.AGE);
    }
    public static void main(String[] args) { new Child().showAll(); }
}
```

Output

```
From Father: Gardening
From Mother: Music
Father's age: 50
```

Example. `Child` inherits behaviour from two parents without ambiguity because the method names differ. Had both interfaces defined `skill()` as default, `Child` would be required to override it.

Q10. Write a Java program using a Payment interface implemented by UPI, CreditCard, and CashPayment.

5 Marks · CO3

```
interface Payment { void pay(double amount); }

class UPI implements Payment {
    public void pay(double amt) { System.out.println("Paid Rs." + amt + " via UPI"); }
}
class CreditCard implements Payment {
    public void pay(double amt) { System.out.println("Paid Rs." + amt + " via Credit Card"); }
}
class CashPayment implements Payment {
    public void pay(double amt) { System.out.println("Paid Rs." + amt + " in Cash"); }
}
public class Checkout {
    public static void main(String[] args) {
        Payment[] modes = { new UPI(), new CreditCard(), new CashPayment() };
        for (Payment p : modes) p.pay(750);
    }
}
```

Output

```
Paid Rs.750.0 via UPI
Paid Rs.750.0 via Credit Card
Paid Rs.750.0 in Cash
```

Example. One loop over `Payment` references uses three different implementations.

Q11. Develop a Java program using an interface Animal and classes Dog and Cat to demonstrate dynamic method dispatch.

5 Marks · CO3

```
interface Animal { void sound(); }
class Dog implements Animal { public void sound() { System.out.println("Dog: Woof"); } }
class Cat implements Animal { public void sound() { System.out.println("Cat: Meow"); } }

public class DispatchDemo {
    static void makeSound(Animal a) { a.sound(); }      // decided at run time
    public static void main(String[] args) {
        Animal a = new Dog();  a.sound();
        a = new Cat();         a.sound();
        makeSound(new Dog());
        makeSound(new Cat());
    }
}
```

Output

```
Dog: Woof
Cat: Meow
Dog: Woof
Cat: Meow
```

Example. `makeSound` is compiled once, but at run time `a.sound()` goes to whichever class the argument belongs.

Q12. Write a Java program using a Shape interface and classes Circle and Rectangle to demonstrate run-time polymorphism.

5 Marks · CO3

```
interface Shape {
    double area();
    double perimeter();
}
class Circle implements Shape {
    double r; Circle(double r) { this.r = r; }
    public double area()      { return Math.PI * r * r; }
    public double perimeter() { return 2 * Math.PI * r; }
}
class Rectangle implements Shape {
    double l, b; Rectangle(double l, double b) { this.l = l; this.b = b; }
    public double area()      { return l * b; }
    public double perimeter() { return 2 * (l + b); }
}
public class ShapeTest {
    public static void main(String[] args) {
        Shape[] list = { new Circle(3), new Rectangle(4, 6) };
        for (Shape s : list)
            System.out.printf("%s: area=%.2f perimeter=%.2f%n",
                s.getClass().getSimpleName(), s.area(), s.perimeter());
    }
}
```

Output

```
Circle: area=28.27 perimeter=18.85
Rectangle: area=24.00 perimeter=20.00
```

Example. The loop calls the same two methods on each element, and the correct formulas are chosen at run time.

Q13. Develop a Java program in which an interface reference refers to objects of different implementing classes.

5 Marks · CO3

```
interface Greeting { String greet(String name); }
class English implements Greeting { public String greet(String n) { return "Hello, " + n; } }
class Hindi   implements Greeting { public String greet(String n) { return "Namaste, " + n; } }
class Bengali implements Greeting { public String greet(String n) { return "Nomoskar, " + n; } }

public class InterfaceRef {
    public static void main(String[] args) {
        Greeting g;                          // one interface reference
        g = new English();  System.out.println(g.greet("Ravi"));
        g = new Hindi();    System.out.println(g.greet("Ravi"));
        g = new Bengali();  System.out.println(g.greet("Ravi"));
    }
}
```

Output

```
Hello, Ravi
Namaste, Ravi
Nomoskar, Ravi
```

Example. The single variable `g` refers in turn to three different objects.

Q14. Write a Java program demonstrating method overriding and dynamic method dispatch using a superclass reference.

5 Marks · CO3

```
class Employee {
    void work() { System.out.println("Employee works"); }
    void salary() { System.out.println("Basic salary"); }
}
class Engineer extends Employee {
    @Override void work() { System.out.println("Engineer designs systems"); }
}
class Teacher extends Employee {
    @Override void work() { System.out.println("Teacher teaches"); }
    @Override void salary() { System.out.println("Salary + research grant"); }
}
public class OverrideDemo {
    public static void main(String[] args) {
        Employee e;                         // superclass reference
        e = new Engineer(); e.work(); e.salary();
        e = new Teacher();  e.work(); e.salary();
    }
}
```

Output

```
Engineer designs systems
Basic salary
Teacher teaches
Salary + research grant
```

Example. `Engineer` overrides only `work()`, so `salary()` is inherited; `Teacher` overrides both. The version run depends on the object, not on the reference type `Employee`.

Q15. Develop a Java program demonstrating a final method in a superclass and other overridable methods in the subclass.

5 Marks · CO3

```
class Account {
    final void showBankName() {                  // cannot be overridden
        System.out.println("Bank: National Bank");
    }
    void interest() { System.out.println("Interest: 4%"); }     // can be overridden
    void charges()  { System.out.println("Charges: Rs.100"); }
}
class Premium extends Account {
    // void showBankName() { }                   // error: cannot override final
    @Override void interest() { System.out.println("Interest: 6%"); }
    @Override void charges()  { System.out.println("Charges: Rs.0"); }
    public static void main(String[] args) {
        Account a = new Premium();
        a.showBankName(); a.interest(); a.charges();
    }
}
```

Output

```
Bank: National Bank
Interest: 6%
Charges: Rs.0
```

> **Example.** Removing the comment marks before the commented line gives: “showBankName() in Premium cannot override showBankName() in Account; overridden method is final”.

Q16. Write a Java program demonstrating a final class that cannot be inherited.

5 Marks · CO3

```
final class Constants {
    static final double PI = 3.14159;
    static double circleArea(double r) { return PI * r * r; }
}
// class MyConstants extends Constants { }   // error: cannot inherit from final Constants

public class FinalClassDemo {
    public static void main(String[] args) {
        System.out.println("Area = " + Constants.circleArea(2));
    }
}
```

Output

```
Area = 12.56636
```

> **Example.** The class itself is usable, but writing `class MyConstants extends Constants` produces the compile-time error “cannot inherit from final Constants”. `String` behaves the same way.

Q17. Develop a Java program combining an abstract class, interface implementation and run-time polymorphism.

5 Marks · CO3

```
interface Payable { double payAmount(); }

abstract class Staff implements Payable {
    String name;
    Staff(String name) { this.name = name; }
    abstract String role();
    void show() { System.out.println(role() + " " + name + " : " + payAmount()); }
}
class Professor extends Staff {
    Professor(String n) { super(n); }
    String role() { return "Professor"; }
    public double payAmount() { return 90000; }
}
class Clerk extends Staff {
    Clerk(String n) { super(n); }
    String role() { return "Clerk"; }
    public double payAmount() { return 25000; }
}
public class College {
    public static void main(String[] args) {
        Staff[] s = { new Professor("Dr. Sen"), new Clerk("Mr. Paul") };
        double total = 0;
        for (Staff x : s) { x.show(); total += x.payAmount(); }
        Payable p = s[0];               // interface reference
        System.out.println("Total = " + total + ", first = " + p.payAmount());
    }
}
```

Output

```
Professor Dr. Sen : 90000.0
Clerk Mr. Paul : 25000.0
Total = 115000.0, first = 90000.0
```

Example. `Staff` is abstract and implements the interface without defining `payAmount()`; each concrete subclass defines it, and the calls are dispatched at run time.

Q18. Write a Java program in which one class inherits from an abstract class and implements two interfaces.

5 Marks · CO3

```
abstract class Animal {
    String name;
    Animal(String n) { name = n; }
    abstract void eat();
    void sleep() { System.out.println(name + " sleeps"); }
}
interface Flyable   { void fly(); }
interface Swimmable { void swim(); }

class Duck extends Animal implements Flyable, Swimmable {
    Duck() { super("Duck"); }
    void eat()          { System.out.println("Duck eats grains"); }
    public void fly()   { System.out.println("Duck flies"); }
    public void swim()  { System.out.println("Duck swims"); }
    public static void main(String[] args) {
        Duck d = new Duck();
        d.eat(); d.sleep(); d.fly(); d.swim();
    }
}
```

Output

```
Duck eats grains
Duck sleeps
Duck flies
Duck swims
```

> **Example.** Syntax: `extends` comes first, then `implements`. `Duck` gets code from `Animal` and contracts from both interfaces.

Bloom's Level: Analyze (L4)

Q1. Analyze why an abstract class cannot be instantiated but can have a constructor.

5 Marks · CO4

Why it cannot be instantiated. An abstract class is, by design, **incomplete**: it may contain abstract methods without a body. If `new Shape()` were allowed, a call to `area()` on that object would have nothing to execute. The compiler therefore forbids creating objects directly from it. The class represents a concept (Shape) for which no real object exists.

Why it still has a constructor. An abstract class often has instance fields (name, id, colour) that must be initialised. A concrete subclass object contains the abstract class's part as well, and the part is created and initialised by the abstract class's constructor, which the subclass constructor calls through `super(...)`. Thus the constructor is never used by `new Shape()`, only by `new Circle()`.

```
abstract class Shape {
    String colour;
    Shape(String c) { colour = c; System.out.println("Shape ctor"); }
}
class Circle extends Shape {
    Circle() { super("red"); System.out.println("Circle ctor"); }
}
```

```
// new Circle();  prints  Shape ctor, then Circle ctor
```

> **Example.** The abstract constructor executes as part of building the subclass object; it never creates an object of the abstract type on its own.

Q2. Analyze why an abstract class cannot be declared as final.

5 Marks · CO4

`abstract` means "this class is unfinished and must be extended". `final` means "this class is finished and must not be extended". Declaring both gives a class that **cannot be instantiated** (abstract) and **cannot be extended** (final), so no object of it could ever exist and its abstract methods could never be implemented. Such a class would be useless, and the compiler rejects the combination with the error "illegal combination of modifiers: abstract and final".

Modifier	Instantiate directly?	Extend?
abstract	No	Yes (needed)
final	Yes (if concrete)	No
abstract final	No	No: impossible to use

> **Example.** `abstract final class A { abstract void f(); }`: nobody could ever write the body of `f()`, so the declaration is meaningless.

Q3. Analyze why an abstract method cannot be declared final, static, or private.

5 Marks · CO4

An abstract method exists to be **overridden** by a subclass. Each of the three modifiers prevents that:

Combination	Effect of the modifier	Why it conflicts
abstract final	Method cannot be overridden	Abstract requires overriding; contradiction
abstract static	Method belongs to the class, is resolved at compile time and can only be hidden	Static methods cannot be overridden, and an abstract method has no body to call
abstract private	Method is visible only in its own class	A subclass cannot see it and so cannot implement it

In each case the method could never obtain a body, the class could never become concrete, and the compiler reports "illegal combination of modifiers".

```
abstract class A {
    // abstract final void f();     // error
    // abstract static void g();    // error
    // abstract private void h();   // error
    abstract void ok();             // legal (default access, or protected/public)
}
```

Q4. Analyze a class hierarchy in which an abstract superclass contains common state and behaviour while subclasses provide different implementations.

5 Marks · CO4

Consider `abstract class Account` with the common state `accNo` and `balance`, common behaviour `deposit()` and `show()`, and the abstract method `calculateInterest()`. `Savings` and `Current` extend it with different interest rules.

```
abstract class Account {
    int accNo; double balance;
    Account(int n, double b) { accNo = n; balance = b; }
    void deposit(double a) { balance += a; }                  // common behaviour
    abstract double calculateInterest();                      // varies
}
class Savings extends Account {
    Savings(int n, double b) { super(n, b); }
    double calculateInterest() { return balance * 0.04; }
}
class Current extends Account {
    Current(int n, double b) { super(n, b); }
    double calculateInterest() { return 0; }
}
```

Analysis.

- **Reuse:** state and `deposit()` are written once; subclasses cannot forget to inherit them.
- **Variation isolated:** only what differs, the interest rule, is abstract.
- **Polymorphism:** a list of `Account` references can be processed uniformly; `calculateInterest()` dispatches at run time.
- **Safety:** an `Account` object itself cannot be created, which prevents meaningless "generic" accounts.
- **Extensibility:** a new type (`FixedDeposit`) only needs to implement one method.
- **Limitation:** each subclass is tied to this one parent (single inheritance), and changes to the base class affect all.

> **Example.** For a Savings account of 10000, `calculateInterest()` gives 400; for a Current account of 10000, it gives 0, through the same call.

Q5. Analyze the difference between using an abstract class and an interface for designing a common Shape abstraction.

5 Marks · CO4

Criterion	Abstract class Shape	Interface Shape
Common state	Can hold fields such as `color` and a constructor	No instance state (constants only)
Shared code	Can provide concrete methods like `describe()` once	Only `default` methods, no state
Inheritance	A subclass can extend only one class, so Circle could not extend any other class	A class can implement many interfaces and keep its own superclass
Relationship	Strong IS-A with a common base	Capability / contract

Criterion	Abstract class Shape	Interface Shape
Evolution	Adding a concrete method is safe; adding an abstract one breaks subclasses	Adding an abstract method breaks implementers; default methods soften this
Coupling	Tighter	Looser

Analysis. If all shapes share state (colour, name) and common code, an **abstract class** reduces duplication. If shapes are unrelated by ancestry (for example `Circle` already extends `Ellipse`, or a `Button` can also behave as a Shape) or only a contract is needed, an **interface** is better. A common good design uses both: `interface Shape` for the contract and `abstract class BaseShape implements Shape` for shared state and defaults.

```
interface Shape { double area(); }
abstract class BaseShape implements Shape {
    String colour; BaseShape(String c) { colour = c; }
}
class Circle extends BaseShape { double r; Circle(String c, double r) { super(c); this.r = r; }
    public double area() { return Math.PI * r * r; } }
```

Q6. Analyze why a final method cannot be overridden by a subclass.

5 Marks · CO4

The `final` modifier on a method is a declaration by the author that **this implementation is complete and must not change**. The compiler enforces it by rejecting any subclass method with the same signature. The reasons for such a design are:

- **Correctness and security:** a method that enforces a rule (validation, password check) must not be replaced by a subclass that bypasses it.
- **Class invariants:** other methods may rely on the exact behaviour of this method, and an override could break them.
- **Template method pattern:** the fixed skeleton of an algorithm is `final`, while the steps are overridable.
- **Optimisation:** the JVM can inline final methods since the call has only one target.

```
class Parent { final void lock() { System.out.println("Locked"); } }
class Child extends Parent {
    // void lock() { }     // error: lock() in Child cannot override lock() in Parent
}
```

> **Example.** A final method can still be **inherited and called**: `new Child().lock();` prints “Locked”. Only the redefinition is prohibited.

Q7. Analyze the difference between a private method and a final method with respect to inheritance and method overriding.

5 Marks · CO4

Aspect	private method	final method
Visible to subclass?	No	Yes (if public/protected/default in package)

Aspect	private method	final method
Inherited?	No	Yes
Can a subclass define a method with the same signature?	Yes, but it is a **new, unrelated** method (not an override)	No: compile-time error
Overriding / dynamic dispatch	Not applicable; bound at compile time	Not allowed; bound at compile time (one target)
Purpose	Hide an implementation detail	Allow reuse but forbid change

```
class A {
    private void p() { System.out.println("A.p"); }
    final void f()   { System.out.println("A.f"); }
    void call()      { p(); f(); }
}
class B extends A {
    void p() { System.out.println("B.p"); }      // legal: new method, not override
    // void f() { }                              // error: final
    public static void main(String[] a) { new B().call(); }
}
```

Output

```
A.p
A.f
```

Analysis. `call()` runs `A.p`, not `B.p`: because `p()` is private, `B.p()` does not override it. Whereas `f()` is inherited by B and accessible, but cannot be redefined.

Q8. Analyze whether declaring an entire class as final is necessary when only one method must not be overridden.

5 Marks · CO4

It is **not necessary** and is usually undesirable. Declaring a single method `final` already guarantees that no subclass can change that method, while still letting the class be extended and its other methods overridden. Making the whole class `final` prevents all inheritance, which removes useful extensibility.

Requirement	Best choice
Only one method must stay fixed	`final` method
Nobody should subclass at all (immutability, security)	`final` class
Several, but not all, methods fixed	`final` on each of those methods

```
class Report {
    final void header() { System.out.println("== Company Report =="); }   // fixed
    void body() { System.out.println("General body"); }                   // overridable
}
class SalesReport extends Report {
    void body() { System.out.println("Sales figures"); }
```

```
}
```

> **Example.** Here `SalesReport` could be created because `Report` is not final, and `header()` is still protected from change. Use a final class only when the whole behaviour, including state, must stay as designed (for example `String`).

Q9. Analyze why Java does not support multiple inheritance through classes. Discuss the ambiguity when two parent classes contain methods with the same signature.

5 Marks · CO4

Java prohibits `class C extends A, B` because of the **ambiguity** it introduces. When A and B both contain `void show()`, a call `c.show()` on a C object has two candidate bodies. If the child does not override it, the language would need an arbitrary rule (such as the order of parents) which makes programs fragile: simply reordering the parents in the declaration would silently change behaviour. When A and B share a parent, the **diamond** also duplicates the grandparent's fields and raises doubts about how many times its constructor should run.

Diagram

```
   A: show()      B: show()
       ^            ^
        \          /
         \        /
           C extends A, B    c.show() -> A.show() or B.show() ?
```

Java's solution. Single inheritance of classes, plus multiple inheritance of **type** through interfaces. Interface methods have no state, and where two interfaces supply the same `default` method the compiler requires the implementing class to override it, resolving the ambiguity explicitly.

```
interface I1 { default void show() { System.out.println("I1"); } }
interface I2 { default void show() { System.out.println("I2"); } }
class C implements I1, I2 {
    public void show() { I1.super.show(); I2.super.show(); }
}
```

> **Example.** `new C().show()` prints I1 and then I2. The ambiguity is settled by the programmer in plain sight.

Q10. Analyze how interfaces provide a mechanism for achieving multiple inheritance in Java.

5 Marks · CO4

A class can implement any number of interfaces, so it can be treated as many types at the same time: this is multiple inheritance of **type and behaviour contract**. It is safe in Java for four reasons:

- Interfaces contain no instance fields, so there is no duplicated state (no diamond of data).
- Abstract methods carry no body, so there is nothing to conflict; the implementing class supplies a single body for each method.
- For default methods with identical signatures, the compiler forces an explicit resolution.
- A class's own superclass is unique, so there is still one chain of implementation inheritance.

```
interface Walkable { void walk(); }
interface Talkable { void talk(); }
class Robot implements Walkable, Talkable {
    public void walk() { System.out.println("Robot walks"); }
    public void talk() { System.out.println("Robot talks"); }
}
```

A `Robot` can be used as a `Walkable` in one method and as a `Talkable` in another, and interfaces can themselves extend several interfaces (`interface Android extends Walkable, Talkable`).

> **Example.** A method `void parade(Walkable w)` accepts the Robot; `void chat(Talkable t)` accepts it too. A single object thereby plays two roles.

Q11. Analyze the difference between multiple inheritance through classes and multiple interface implementation with respect to ambiguity and flexibility.

5 Marks · CO4

Aspect	Multiple inheritance of classes (not in Java)	Multiple interface implementation (Java)
Ambiguity	High: same method bodies or fields in two parents; diamond	None for abstract methods (one body in the class); default-method clashes must be overridden
State	Inherits fields from all parents, possibly duplicated	No instance state to inherit
Constructor chaining	Complicated order and arguments	Not involved
Flexibility	Reuses code of several parents, but is rigid and tangled	Combines several roles and keeps the class free to extend one class
Coupling	Strong, through implementation	Loose, through contracts
Change impact	Edits to either parent can break the child	Changing an interface affects only implementers' signatures

Analysis. Multiple class inheritance gives code reuse from many parents but at the price of ambiguity and tight coupling. Interface implementation gives up code sharing (apart from default methods) and in return gets clarity, loose coupling and the freedom to be used in many roles. Where code reuse is also needed, composition (delegating to an object) can be combined with interfaces.

Q12. Analyze the following code and determine which implementation of draw() will execute. Justify your answer.

5 Marks · CO4

The code is not reproduced in the question bank, so the standard version of this question is analysed below.

Given in the question

```
interface Drawable { void draw(); }
class Shape implements Drawable {
    public void draw() { System.out.println("Shape drawn"); }
}
class Circle extends Shape {
```

```
    public void draw() { System.out.println("Circle drawn"); }
}
public class Test {
    public static void main(String[] args) {
        Drawable d = new Circle();
        d.draw();
        Shape s = new Shape();
        s.draw();
    }
}
```

Output:

Output

```
Circle drawn
Shape drawn
```

Justification.

- `Drawable d = new Circle();` The reference type is the interface `Drawable`, but the object is a `Circle`. The compiler only checks that `Drawable` declares `draw()`.
- At run time dynamic method dispatch selects the implementation in the actual class of the object, `Circle`; `Circle.draw()` overrides `Shape.draw()`, so “Circle drawn” is printed.
- For `Shape s = new Shape();` the object is a `Shape`, so `Shape.draw()` runs and “Shape drawn” is printed.

General rule: the executed `draw()` is the most specific one found by searching from the object’s own class upward; if `Circle` had no `draw()`, `Shape`‘s version would execute.

MODULE

7

Packages and Inner Classes

CA2: nested and inner classes, packages, import, static import and access protection

Bloom's Level: Remember (L1)

Q1. Define an inner class in Java.

2 Marks · CO1

An inner class is a class defined inside another class (the outer or enclosing class). A non-static inner class is associated with an instance of the outer class and can access all its members, including private ones.

> **Example.** `class Outer { class Inner { void show() { } } }`

Q2. What is a nested class in Java?

2 Marks · CO1

A nested class is any class declared within another class. Nested classes are of two kinds: **static nested classes** (declared `static`) and **inner classes** (non-static). It is a way of logically grouping classes that are used only in one place.

Q3. List the different types of inner/nested classes available in Java.

2 Marks · CO1

- Static nested class
- Member inner class (non-static)
- Local inner class (inside a method or block)
- Anonymous inner class (without a name)

Q4. What is a member inner class?

2 Marks · CO1

A member inner class is a non-static class declared as a member of the outer class, outside any method. Its object needs an object of the outer class and it can use all the outer class's members.

> **Example.** `class Outer { int x = 5; class Inner { int get() { return x; } } }`

Q5. What is a local inner class?

2 Marks · CO1

A local inner class is a class declared inside a method (or block) of the outer class. It is visible only within that method and can use the method's local variables that are final or effectively final.

> **Example.** `void f() { class Local { void hi() { System.out.println("Hi"); } } new Local().hi(); }`

Q6. What is an anonymous inner class?

2 Marks · CO1

An anonymous inner class is a class without a name that is declared and instantiated in a single expression, normally to implement an interface or override a method of a class on the spot.

> **Example.** `Runnable r = new Runnable() { public void run() { System.out.println("Run"); } };`

Q7. What is a static nested class?

2 Marks · CO1

A static nested class is a nested class declared with `static`. It is associated with the outer class itself and not with an object, so it can be instantiated without an outer object, and it can access only the static members of the outer class directly.

> **Example.** `Outer.Nested n = new Outer.Nested();`

Q8. Define a package in Java.

2 Marks · CO1

A package is a named group of related classes, interfaces and sub-packages. It provides a namespace that avoids name conflicts, organises code and supports access protection.

> **Example.** `java.util` is a package that contains `ArrayList`, `Scanner` and others.

Q9. What is an in-built package in Java?

2 Marks · CO1

An in-built (predefined) package is a package supplied with the Java platform in its class library, such as `java.lang`, `java.util`, `java.io`, `java.net` and `java.awt`.

Q10. What is a user-defined package in Java?

2 Marks · CO1

A user-defined package is a package created by the programmer to group his or her own classes, using the `package` statement at the top of the source file.

> **Example.** `package mypackage;` followed by `public class Student { }`.

Q11. What is the purpose of the package keyword in Java?

2 Marks · CO1

The `package` keyword declares that the classes in the source file belong to a particular package. It must be the first statement in the file (before imports) and the package name corresponds to the directory structure.

Q12. What is the purpose of the import statement in Java?

2 Marks · CO1

The `import` statement makes the classes of another package available by their simple names, so the full package-qualified name need not be written each time.

> **Example.** With `import java.util.ArrayList;` we write `ArrayList`, not `java.util.ArrayList`.

Q13. Name any four commonly used in-built Java packages.

2 Marks · CO1

`java.lang` (basic classes such as String, Math), `java.util` (collections, Scanner), `java.io` (input and output), `java.net` (networking). Others are `java.awt`, `javax.swing` and `java.sql`.

Q14. What are access specifiers/access modifiers in Java?

2 Marks · CO1

They are keywords that control the visibility of classes, variables, methods and constructors. They are `private`, default (no keyword), `protected` and `public`.

Q15. Name the four access levels available in Java.

2 Marks · CO1

`private` (class only), default or package-private (same package), `protected` (same package plus sub-classes elsewhere) and `public` (everywhere).

Q16. Which package is automatically available without an explicit import statement in every Java program?

2 Marks · CO1

The package `java.lang`, which contains `String`, `Math`, `System`, `Object`, wrapper classes and so on.

Q17. What is a static import in Java?

2 Marks · CO1

A static import (`import static`) lets the static members (variables and methods) of a class be used directly by their simple names without the class name prefix.

> **Example.** `import static java.lang.Math.sqrt;` allows `sqrt(16)` instead of `Math.sqrt(16)`.

Q18. What is the purpose of the * symbol in an import statement?

2 Marks · CO1

The asterisk is a wildcard that imports **all** classes of a package at once. It does not import classes of sub-packages.

> **Example.** `import java.util.*;` makes `ArrayList`, `Scanner`, `HashMap` and the other `java.util` classes available.

Q19. Write the syntax for declaring a user-defined package in Java.

2 Marks · CO1

The statement `package packagename;` is written as the first non-comment line of the source file.

```
package mypackage;           // single level
package com.college.dept;    // nested, maps to com/college/dept/
```

Q20. Which access modifier provides the widest accessibility in Java?

2 Marks · CO1

`public`: a public member is accessible from every class in every package.

Bloom's Level: Understand (L2)

Q1. Explain the concept of an inner class with a suitable example.

3 Marks · CO2

An inner class is a class written inside another class. It is used when a class is meaningful only in association with its outer class, so grouping them improves organisation and encapsulation. Since an inner class belongs to an instance of the outer class, it can directly use all outer members, even private ones, and it is hidden from other classes.

```
class Outer {
    private int x = 10;
    class Inner {
        void show() { System.out.println("x = " + x); }   // uses outer private member
    }
    public static void main(String[] args) {
        Outer o = new Outer();
        Outer.Inner i = o.new Inner();
        i.show();
    }
}
```

Output

```
x = 10
```

Q2. Explain the different types of inner classes in Java.

3 Marks · CO2

Type	Where declared	Key properties
Member inner class	Inside a class, outside methods, non-static	Needs an outer object; accesses all outer members
Static nested class	Inside a class with `static`	No outer object needed; accesses only outer static members directly
Local inner class	Inside a method or block	Visible only in that block; uses final/effectively final locals
Anonymous inner class	In an expression, without a name	Defined and instantiated at once; implements an interface or extends a class

```
class Demo {
    class Member { }
    static class Nested { }
    void f() { class Local { }  Runnable r = new Runnable() { public void run() { } }; }
}
```

Q3. Differentiate between an inner class and a static nested class.

3 Marks · CO2

Basis	Inner (non-static) Class	Static Nested Class
Keyword	No `static`	Declared `static`
Outer object	Required to create: `outer.new Inner()`	Not required: `new Outer.Nested()`
Access to outer members	All, including instance and private	Only static members directly
Can have static members	Not (before Java 16)	Yes
Association	With an object of outer class	With the outer class itself

```
class Outer { static class N { } class I { } }
// Outer.N n = new Outer.N();       Outer.I i = new Outer().new I();
```

Q4. Explain how an inner class can access the instance variables of its enclosing class.

3 Marks · CO2

Every inner class object holds a hidden reference to the outer object that created it (`Outer.this`). Through this reference the inner class can read and modify any instance variable of that outer object, including private ones, by simply using the variable name or `Outer.this.name`. If the inner class has a variable of the same name, `Outer.this.x` distinguishes the outer one.

```
class Outer {
    private int x = 5;
    class Inner {
        int x = 50;
        void show() {
            System.out.println(x);               // 50: Inner's own
            System.out.println(Outer.this.x);    // 5 : outer's
            Outer.this.x++;                       // modifies outer variable
        }
    }
}
```

Q5. Explain the concept of a local inner class with a suitable example.

3 Marks · CO2

A local inner class is declared inside a method (or any block). Its scope is limited to that block, so it can be instantiated only there. It is used when a helper class is required by one method alone. It can access the members of the outer class and the method's local variables that are final or effectively final. It cannot have access modifiers or be `static`.

```
class Outer {
    void calc() {
        class Square {
            int of(int n) { return n * n; }
        }
        Square s = new Square();
        System.out.println(s.of(6));
    }
    public static void main(String[] a) { new Outer().calc(); }
}
```

Output

```
36
```

Q6. Explain how a local inner class can access variables of the enclosing method.

3 Marks · CO2

A local inner class may use the local variables and parameters of the enclosing method only if they are **final or effectively final** (never reassigned after initialisation). The reason is that the inner object may live longer than the method call; the compiler therefore copies the variable's value into the inner object, and a later change would make the copy inconsistent. Trying to modify such a variable gives a compile-time error.

```
void f() {
    int base = 10;                  // effectively final
    class Adder { int add(int k) { return base + k; } }
    System.out.println(new Adder().add(5));     // 15
    // base++;                      // would make 'base' not effectively final: error above
}
```

Q7. Explain the concept of an anonymous inner class and mention its common uses.

3 Marks · CO2

An anonymous inner class has no name; it is declared and instantiated in one expression by writing `new Type() { body }`, where `Type` is an interface or a class. It is used for a one-time implementation.

```
interface Greeter { void greet(); }
public class Anon {
    public static void main(String[] args) {
        Greeter g = new Greeter() {
            public void greet() { System.out.println("Hello from anonymous class"); }
        };
        g.greet();
    }
}
```

Common uses: event handlers in GUI programs (`ActionListener`), threads (`new Thread(new Runnable() { ... })`), comparators for sorting (`new Comparator<T>() { ... }`), and overriding a method for a single object. Since Java 8, lambda expressions replace many of them.

Q8. Explain why an instance of a non-static inner class requires an instance of its enclosing class.

3 Marks · CO2

A non-static inner class is a member of each outer **object**, like an instance method; its code uses the outer object's instance variables through a hidden reference. For that reference to be valid, an outer object must exist when the inner object is created. Hence the syntax `outerObject.new Inner()`, or `new Inner()` from inside an instance method of the outer class where `this` supplies the outer object. Without it, the inner object would have no outer state to work with.

```
Outer o = new Outer();
Outer.Inner i = o.new Inner();       // outer object o is required
// Outer.Inner j = new Outer.Inner();   // error
```

Q9. Explain the purpose of packages in Java.

3 Marks · CO2

Packages are used to:

- **Group related classes** and interfaces into one unit (for example all input/output classes in `java.io`).
- **Avoid name conflicts**: two classes of the same name can exist in different packages (`java.util.Date` and `java.sql.Date`).
- **Provide access control**, through the default and protected levels.
- **Make code reusable and maintainable** and easy to locate, since the package name maps to a directory.
- **Allow distribution** of libraries as a unit (JAR files).

Q10. Explain the advantages of using packages in Java.

3 Marks · CO2

- **Organisation:** large programs are divided into meaningful modules.
- **Name-space management:** the fully qualified name (`com.aot.Student`) removes conflicts between classes of the same simple name.
- **Access protection:** package-private and protected access control who can use what.
- **Reusability:** a package is compiled once and used by many programs through `import`.
- **Easier maintenance and teamwork:** teams work in separate packages.
- **Encapsulation:** implementation classes can be hidden in the package while only a few are public.

Q11. Differentiate between in-built packages and user-defined packages.

3 Marks · CO2

Basis	In-built Package	User-Defined Package
Created by	Java platform developers	The programmer
Location	In the JDK class library	In the programmer's folders
Examples	`java.lang`, `java.util`, `java.io`	`mypackage`, `com.aot.exam`
Import needed	Yes, except `java.lang`	Yes, if in a different package
Purpose	General-purpose facilities	Application-specific classes
Declaration	Already written	Using the `package` statement

Q12. Explain how a user-defined package is created and accessed in Java.

3 Marks · CO2

Creating. (1) Write `package mypackage;` as the first line of the source file. (2) Declare the classes (and members that others need) `public`. (3) Save the file in a folder named `mypackage`. (4) Compile with `javac -d . Student.java`, which creates the folder `mypackage` and puts `Student.class` inside it.

Accessing. In another file, use `import mypackage.Student;` (or `import mypackage.*;`), or the full name `mypackage.Student`. Compile and run with the package's parent directory on the class path.

```
// File: mypackage/Student.java
package mypackage;
public class Student { public void show() { System.out.println("Student"); } }

// File: Test.java
import mypackage.Student;
class Test { public static void main(String[] a) { new Student().show(); } }
```

Output

```
javac -d . mypackage/Student.java
javac Test.java
java Test
```

Q13. Explain the difference between import package.*; and import package.ClassName;.

3 Marks · CO2

`import package.*;` imports **all** the public classes of the package on demand, so all of them may be used by simple name. `import package.ClassName;` imports **only the named class**. The wildcard form is shorter but can cause name clashes and makes it less clear where a class comes from; the specific form is clearer. Neither affects run-time performance, because classes are loaded only when used.

```
import java.util.*;            // ArrayList, Scanner, HashMap ... all visible
import java.util.ArrayList;    // only ArrayList visible
```

Q14. Explain whether importing a package also imports its sub-packages, with suitable justification.

3 Marks · CO2

No. Packages in Java are not truly hierarchical for importing: `java.util` and `java.util.regex` are separate packages even though one name contains the other. The wildcard `import java.util.*;` imports only the **classes directly inside** `java.util`, not those in `java.util.regex`. The sub-package must be imported separately.

```
import java.util.*;
// Pattern p;                     // error: Pattern is in java.util.regex
import java.util.regex.*;         // needed separately
```

The folder structure is nested, but the language treats each package as an independent namespace.

Q15. Explain the purpose of static imports in Java with a suitable example.

3 Marks · CO2

A static import allows the static members of a class (constants and methods) to be used without qualifying them with the class name, making code that uses them heavily shorter and easier to read. It is useful for mathematical constants and utility methods, but should be used sparingly because it can hide where a name comes from.

```
import static java.lang.Math.*;
public class StaticImp {
    public static void main(String[] args) {
        System.out.println(sqrt(25));      // instead of Math.sqrt(25)
        System.out.println(PI);
    }
```

```
}
```

Output

```
5.0
3.141592653589793
```

Q16. Explain the four levels of access protection available in Java.

3 Marks · CO2

Modifier	Same class	Same package	Subclass in another package	Other package
private	Yes	No	No	No
default (none)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

`private` gives the strongest hiding (only the class itself); default restricts to the package; `protected` extends default access to subclasses in other packages; `public` is unrestricted. A top-level class itself can be only `public` or default.

Q17. Explain the accessibility of private, default, protected and public members in Java.

3 Marks · CO2

- **private:** visible only inside the class that declares the member. It is used for data hiding. Subclasses and other classes cannot use it directly.
- **default (package-private):** visible to all classes in the **same package**; invisible to classes outside, even subclasses.
- **protected:** visible in the same package and also to **subclasses** in other packages (through inheritance).
- **public:** visible everywhere.

```
package p1;
public class A {
    private int a = 1;  int b = 2;  protected int c = 3;  public int d = 4;
}
// In class B in the same package p1:    b, c, d accessible; a is not.
// In class C extends A in package p2:   c, d accessible; a, b are not.
// In class D (unrelated) in package p2: only d accessible.
```

Q18. Explain how access protection works when a class belongs to a different package.

3 Marks · CO2

When the accessing class is in another package, access depends on two checks: first, the **class** itself must be `public` (a default-access class is invisible outside its package); second, the **member** must permit access. From another package:

- `public` members are accessible.
- `protected` members are accessible only inside a **subclass**, and only through the subclass's own reference (`this`, or an object of the subclass), not through a plain object of the parent class.

- default and `private` members are not accessible.

```
package p2;
import p1.A;
class C extends A {
    void test() { System.out.println(c + d); }   // protected c, public d: OK
}
class D { void test(A a) { System.out.println(a.d); } }   // only public d is OK
```

Q19. Explain the difference between access modifiers and access specifiers in Java.

3 Marks · CO2

In Java the two terms are used almost interchangeably; Java's official documentation says "access modifiers". The distinction sometimes drawn in textbooks is: **access specifiers** are the four visibility levels (`public`, `protected`, default, `private`), which specify *who* may access a member; **access modifiers** is the broader term including the specifiers as well as other modifiers such as `static`, `final`, `abstract`, `synchronized`, which *modify* how a member behaves. Practically, `public static final int X = 1;` has the specifier `public` and the modifiers `static` and `final`.

Term	Meaning	Examples
Access specifier	Controls visibility	public, protected, default, private
Modifier (non-access)	Changes behaviour	static, final, abstract, synchronized

Bloom's Level: Apply (L3)

Q1. Write a Java program to create an inner class inside an outer class and access a method of the inner class.

5 Marks · CO3

```
class Outer {
    class Inner {
        void greet() { System.out.println("Hello from the Inner class"); }
    }
    public static void main(String[] args) {
        Outer outer = new Outer();                 // outer object first
        Outer.Inner inner = outer.new Inner();     // then the inner object
        inner.greet();
    }
}
```

Output

```
Hello from the Inner class
```

> **Example.** The inner object is created through an outer object using `outer.new Inner()`. Compilation produces two files, `Outer.class` and `Outer$Inner.class`.

Q2. Write a Java program in which an inner class accesses an instance variable of its enclosing class.

5 Marks · CO3

```
class Bank {
    private double balance = 5000;                // private outer variable
    class Atm {
        void withdraw(double amt) {
            if (amt <= balance) {
                balance -= amt;                   // modifies outer variable
                System.out.println("Withdrawn " + amt + ", left " + balance);
            } else System.out.println("Insufficient balance");
        }
    }
    void show() { System.out.println("Balance = " + balance); }

    public static void main(String[] args) {
        Bank b = new Bank();
        Bank.Atm atm = b.new Atm();
        atm.withdraw(1500);
        atm.withdraw(9000);
        b.show();
    }
}
```

Output

```
Withdrawn 1500.0, left 3500.0
Insufficient balance
Balance = 3500.0
```

> **Example.** The inner class `Atm` reads and changes the private field `balance` of `Bank`; a separate top-level class could not do so.

Q3. Write a Java program to demonstrate a local inner class defined inside a method.

5 Marks · CO3

```
class Outer {
    void process() {
        class Helper {                            // local inner class
            void show(String msg) { System.out.println("Helper: " + msg); }
        }
        Helper h = new Helper();                  // usable only inside process()
        h.show("working inside the method");
    }
    public static void main(String[] args) {
        new Outer().process();
    }
}
```

Output

```
Helper: working inside the method
```

> **Example.** `Helper` is invisible outside `process()`; `new Helper()` in `main` would not compile.

Q4. Write a Java program to demonstrate the use of an anonymous inner class.

5 Marks · CO3

```
interface Message { void show(); }

abstract class Greeting { abstract void hello(); }

public class AnonymousDemo {
    public static void main(String[] args) {
        Message m = new Message() {                 // anonymous class implementing interface
            public void show() { System.out.println("Anonymous implementation of Message"); }
        };
        m.show();

        Greeting g = new Greeting() {               // anonymous subclass of abstract class
            void hello() { System.out.println("Hello from anonymous subclass"); }
        };
        g.hello();

        Thread t = new Thread() {                   // anonymous subclass of Thread
            public void run() { System.out.println("Thread body"); }
        };
        t.start();
    }
}
```

Output

```
Anonymous implementation of Message
Hello from anonymous subclass
Thread body
```

> **Example.** No named class such as `MessageImpl` is needed; the class is defined and instantiated in one expression. The compiler creates files such as `AnonymousDemo$1.class`.

Q5. Write a Java program to create a static nested class and access its members.

5 Marks · CO3

```
class University {
    static String name = "AOT";
    int students = 900;

    static class Department {                       // static nested class
        String dept = "CSE";
        void show() {
            System.out.println(dept + " dept of " + name);   // static outer member: OK
            // System.out.println(students);        // error: instance member of outer
        }
    }
    public static void main(String[] args) {
        University.Department d = new University.Department();   // no outer object
        d.show();
        System.out.println(d.dept);
    }
}
```

Output

```
CSE dept of AOT
CSE
```

Example. `new University.Department()` needs no `University` object. The nested class can use `name` (static) directly but not the instance variable `students` without an outer object.

Q6. Write a Java program to create a user-defined package named mypackage containing a class Student.

5 Marks · CO3

Step 1. Create the folder `mypackage` and the file `mypackage/Student.java`:

```
// File: mypackage/Student.java
package mypackage;

public class Student {
    private int roll;
    private String name;

    public Student(int roll, String name) { this.roll = roll; this.name = name; }

    public void display() {
        System.out.println("Roll: " + roll + ", Name: " + name);
    }
}
```

Step 2. Compile from the parent folder, which writes `mypackage/Student.class`:

Output

```
javac -d . mypackage/Student.java
```

Example. The class and the members to be used outside must be `public`; the `package` statement must be the first statement in the file.

Q7. Write a Java program to import and use a class from a user-defined package.

5 Marks · CO3

Using the package `mypackage` and its class `Student` created above:

```
// File: UseStudent.java  (in the parent folder of mypackage)
import mypackage.Student;

public class UseStudent {
    public static void main(String[] args) {
        Student s = new Student(12, "Kabir");
        s.display();
    }
}
```

Output

```
javac UseStudent.java
java UseStudent
Roll: 12, Name: Kabir
```

Example. Without the import, the full name is needed: `mypackage.Student s = new mypackage.Student(12, "Kabir");`. If `Student` or its constructor were not public, the compiler would report an access error.

Q8. Create two user-defined packages and write a program to demonstrate accessing a public class from one package in another package.

5 Marks · CO3

```
// File: pack1/Calculator.java
package pack1;
public class Calculator {
    public int add(int a, int b) { return a + b; }
}

// File: pack2/Display.java
package pack2;
import pack1.Calculator;               // class from the other package

public class Display {
    public void showSum(int a, int b) {
        Calculator c = new Calculator();
        System.out.println("Sum = " + c.add(a, b));
    }
}

// File: Main.java
import pack2.Display;
public class Main {
    public static void main(String[] args) {
        new Display().showSum(7, 8);
    }
}
```

Output

```
javac -d . pack1/Calculator.java pack2/Display.java Main.java
java Main
Sum = 15
```

Example. `Main` knows only `pack2`; `pack2.Display` in turn uses the public class of `pack1`. Both the class and the method used across packages are `public`.

Q9. Write a Java program using the java.util package to create and manipulate an ArrayList.

5 Marks · CO3

```
import java.util.ArrayList;
import java.util.Collections;

public class ListDemo {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Mango");
        list.add("Apple");
        list.add("Banana");
        System.out.println("List     : " + list);
        list.add(1, "Orange");                    // insert at index 1
        list.remove("Apple");                     // remove element
        System.out.println("Modified : " + list);
        Collections.sort(list);
        System.out.println("Sorted   : " + list);
        System.out.println("Size     : " + list.size());
        System.out.println("Has Mango: " + list.contains("Mango"));
    }
}
```

Output

```
List     : [Mango, Apple, Banana]
Modified : [Mango, Orange, Banana]
Sorted   : [Banana, Mango, Orange]
Size     : 3
Has Mango: true
```

Example. `ArrayList` and `Collections` belong to `java.util`, which is why the import is needed.

Q10. Write a Java program using the java.io package to perform a basic file operation.

5 Marks · CO3

```
import java.io.*;

public class FileDemo {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("data.txt");      // create/write
            fw.write("Hello, file handling!\nSecond line");
            fw.close();

            BufferedReader br = new BufferedReader(new FileReader("data.txt"));
            String line;
            while ((line = br.readLine()) != null)           // read line by line
                System.out.println(line);
            br.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output

```
Hello, file handling!
Second line
```

Example. `FileWriter` creates `data.txt` and writes the text; `BufferedReader` reads it back. I/O operations may fail, so they are enclosed in try-catch for `IOException`.

Q11. Write a Java program using static import to directly access sqrt() or another static member of java.lang.Math.

5 Marks · CO3

```
import static java.lang.Math.sqrt;
import static java.lang.Math.pow;
import static java.lang.Math.PI;

public class StaticImportDemo {
    public static void main(String[] args) {
        System.out.println("sqrt(81)  = " + sqrt(81));
        System.out.println("pow(2,10) = " + pow(2, 10));
        double r = 3;
        System.out.println("Area      = " + PI * r * r);
    }
}
```

Output

```
sqrt(81)  = 9.0
pow(2,10) = 1024.0
Area      = 28.274333882308138
```

Example. Without the static import the code would be `Math.sqrt(81)`, `Math.pow(2, 10)` and `Math.PI`.

Q12. Write a Java program demonstrating the use of import package.*;.

5 Marks · CO3

```
// File: shapes/Circle.java
package shapes;
public class Circle {
    public double area(double r) { return Math.PI * r * r; }
}
// File: shapes/Square.java
package shapes;
public class Square {
    public double area(double s) { return s * s; }
}
// File: Main.java
import shapes.*;                    // imports every public class in shapes

public class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
        Square s = new Square();
        System.out.printf("Circle: %.2f%n", c.area(2));
        System.out.printf("Square: %.2f%n", s.area(2));
    }
}
```

Output

```
Circle: 12.57
Square: 4.00
```

Example. One wildcard import gives access to both `Circle` and `Square`.

Q13. Write a Java program demonstrating the use of import package.ClassName;.

5 Marks · CO3

```
// File: shapes/Circle.java
package shapes;
public class Circle { public double area(double r) { return Math.PI * r * r; } }

// File: shapes/Square.java
package shapes;
public class Square { public double area(double s) { return s * s; } }

// File: Main.java
import shapes.Circle;               // only Circle is imported

public class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
        System.out.printf("Circle: %.2f%n", c.area(2));
        // Square s = new Square();   // error: cannot find symbol Square
        shapes.Square s = new shapes.Square();   // full name is still possible
        System.out.println("Square: " + s.area(3));
    }
}
```

Output

```
Circle: 12.57
Square: 9.0
```

Example. Only the named class can be referred to by its simple name; others require the full name.

Q14. Develop a Java program demonstrating the accessibility of private, default, protected and public members.

5 Marks · CO3

```
// File: p1/Base.java
package p1;
public class Base {
    private int a = 1;
    int b = 2;                      // default
    protected int c = 3;
    public int d = 4;
    public void showAll() {         // inside the class: all four are accessible
        System.out.println(a + " " + b + " " + c + " " + d);
    }
}
// File: p1/SamePkg.java
package p1;
public class SamePkg {
    public static void main(String[] args) {
        Base x = new Base();
        x.showAll();
        // System.out.println(x.a);            // error: private
        System.out.println(x.b + " " + x.c + " " + x.d);   // default, protected, public OK
    }
}
```

Output

```
1 2 3 4
2 3 4
```

Member	Same class	Same package
private a	OK	Error
default b	OK	OK
protected c	OK	OK
public d	OK	OK

Example. From a different package, only `d` is accessible (and `c` through a subclass).

Q15. Write a Java program to demonstrate access to a protected member from a subclass belonging to another package.

5 Marks · CO3

```
// File: p1/Parent.java
package p1;
public class Parent {
    protected int salary = 50000;
    protected void showSalary() { System.out.println("Salary = " + salary); }
}
// File: p2/Child.java
package p2;
import p1.Parent;

public class Child extends Parent {
    public void test() {
        System.out.println("Accessing inherited: " + salary);   // OK
        showSalary();                                           // OK
        Child c = new Child();
        System.out.println(c.salary);                           // OK: via subclass object
        // Parent p = new Parent();
        // System.out.println(p.salary);                       // error: protected access
    }
    public static void main(String[] args) { new Child().test(); }
}
```

Output

```
Accessing inherited: 50000
Salary = 50000
50000
```

> **Example.** In another package a protected member is accessible only through inheritance (the subclass or an object of the subclass), not through a plain `Parent` reference.

Q16. Write a Java program to demonstrate which members are accessible from a non-subclass class belonging to another package.

5 Marks · CO3

```
// File: p1/Data.java
package p1;
public class Data {
    private int a = 1;
    int b = 2;
    protected int c = 3;
    public int d = 4;
    public int getA() { return a; }          // public accessor for private a
}
// File: p2/Other.java
package p2;
import p1.Data;

public class Other {                          // NOT a subclass of Data
    public static void main(String[] args) {
        Data x = new Data();
        System.out.println(x.d);              // public: OK
        System.out.println(x.getA());         // access private through public method
        // System.out.println(x.a);           // error: private
        // System.out.println(x.b);           // error: default (package-private)
        // System.out.println(x.c);           // error: protected, and not a subclass
    }
}
```

Output

```
4
1
```

> **Example.** Only public members are accessible directly; the private field can only be reached through its public getter.

Q17. Write a Java program containing a local inner class inside a method and demonstrate its access to a local variable of the enclosing method.

5 Marks · CO3

```
public class LocalAccess {
    private String company = "AOT";                    // outer instance variable

    void compute(final int base) {                     // parameter: effectively final
        int bonus = 500;                               // effectively final local variable
        class Calculator {
            int total(int extra) {
                return base + bonus + extra;           // uses method's variables
            }
            String owner() { return company; }         // uses outer field
        }
        Calculator c = new Calculator();
        System.out.println(c.owner() + " total = " + c.total(100));
        // bonus = 600;     // error if uncommented: local variable must be effectively final
    }
    public static void main(String[] args) {
        new LocalAccess().compute(1000);
    }
}
```

Output

```
AOT total = 1600
```

> **Example.** 1000 + 500 + 100 = 1600. Re-assigning `bonus` would make the variable no longer effectively final, and the local class would then fail to compile.

Q18. Write a Java program demonstrating how an instance of a non-static inner class is created from a non-static method of its enclosing class.

5 Marks · CO3

```
public class School {
    private String name = "Green Valley";

    class Student {                                    // non-static inner class
        String sname;
        Student(String n) { sname = n; }
        void show() { System.out.println(sname + " studies at " + name); }
    }
    void admit() {                                     // non-static method of outer
        Student s = new Student("Ria");                // 'this' supplies the outer object
        s.show();
        Student t = this.new Student("Sam");           // equivalent explicit form
        t.show();
    }
    public static void main(String[] args) {
        new School().admit();
    }
}
```

Output

```
Ria studies at Green Valley
Sam studies at Green Valley
```

> **Example.** Inside a non-static method, `new Student(...)` works because the current object (`this`) is the enclosing instance. From `static main`, `new Student()` would be an error; `new School().new Student("Ria")` would be needed.

Bloom's Level: Analyze (L4)

Q1. Analyze the different types of inner classes in Java and determine the appropriate type for a given programming situation. Justify your choice.

5 Marks · CO4

Situation	Suitable type	Justification
A helper class closely tied to the state of one outer object (for example an `Iterator` over a collection)	Member inner class	Needs the outer object's data and is used by many methods
A utility/data class logically grouped with the outer class but needing no outer instance (for example `Node` of a linked list, `Builder`)	Static nested class	No hidden reference to the outer object, so it is lighter and avoids memory leaks
A small class used by only one method	Local inner class	Scope limited to that method; hides it from the rest of the program
A one-time implementation of an interface or a method override (event handler, comparator, Runnable)	Anonymous inner class (or lambda)	No name or separate file required; defined at the point of use

Decision procedure. (1) Does it need outer instance data? If no, prefer a **static nested** class. (2) If yes, is it used in several methods? Then a **member** inner class. (3) If only in one method, **local**. (4) If it is used once and implements a single interface or overrides one method, **anonymous**.

```
class LinkedList {
    private static class Node { int data; Node next; }   // static nested: no outer data
    class Itr { Node cur; }                              // member inner: iterates this list
}
```

Q2. Analyze how an inner class accesses members of its enclosing class and explain the relationship between the two classes with a suitable example.

5 Marks · CO4

Mechanism. The compiler gives every non-static inner class object a hidden field referring to the outer object that created it (`this$0` in the byte code, written `Outer.this` in source). When the inner code names an outer member, the compiler translates it into a use of this reference. As a result, the inner class can use all members of the outer instance, including `private` ones, without getters. The reverse is not true: the outer class cannot use the inner members directly; it must create an inner object, after which it can access even the inner private members.

Relationship. The two are tightly coupled: an inner object cannot exist without an outer object (**composition-like**), the inner class lives and is garbage-collected with the outer unless referenced elsewhere, and each can see the other's private members.

```
class Outer {
    private int count = 0;
    class Counter {
        private int step = 2;
        void inc() { count += step; }              // inner uses outer private
    }
    void run() {
        Counter c = new Counter();
        c.inc(); c.inc();
        System.out.println(count + " " + c.step);   // outer uses inner private
    }
    public static void main(String[] a) { new Outer().run(); }
}
```

Output

```
4 2
```

Q3. Examine a local inner class defined inside a method and determine which variables of the enclosing method it can access and under what conditions.

5 Marks · CO4

A local inner class can access:

- all members (including private) of the **enclosing class**; in an instance method, through the outer `this`; in a static method, only static members;
- the **local variables and parameters** of the enclosing method, **only if they are final or effectively final**, meaning they are assigned once and never changed (Java 8 and later; before that the keyword `final` was required);
- its own members and the members of its superclass.

Reason for the condition. A local object may outlive the method call (it can be returned or stored), after which the stack frame no longer exists. The compiler therefore copies the values of the used locals into the object. A later change of the original would not be reflected in the copy, so the language forbids the change.

```
void m() {
    int a = 5;            // effectively final: usable
    int b = 7;
    b++;                  // modified: NOT usable in the local class
    class L {
```

```
        int f() { return a; }          // OK
        // int g() { return b; }       // error: local variables referenced from an
                                       // inner class must be final or effectively final
    }
}
```

Q4. Analyze the difference between a static nested class and a non-static inner class with respect to object creation and access to members of the enclosing class.

5 Marks · CO4

Aspect	Static Nested Class	Non-Static Inner Class
Object creation	`new Outer.Nested()`; no outer object	`outerObj.new Inner()`; outer object required
Hidden outer reference	None	Yes (`Outer.this`)
Access to outer instance members	Not directly; only through an explicit outer object	Directly, including private
Access to outer static members	Yes, directly	Yes
Static members of its own	Allowed	Not allowed in older versions (allowed from Java 16)
Memory	Lighter	Holds the outer object alive
Typical use	Grouping utility or data classes (Node, Entry)	Helpers bound to one outer instance (Iterator)

```
class Outer {
    static int s = 1;   int i = 2;
    static class N { int f() { return s; /* i would be an error */ } }
    class I { int f() { return s + i; } }
    public static void main(String[] a) {
        Outer.N n = new Outer.N();          // no outer object
        Outer.I x = new Outer().new I();    // outer object needed
        System.out.println(n.f() + " " + x.f());
    }
}
```

Output

```
1 3
```

Q5. Examine a Java program containing an anonymous inner class and determine its purpose and execution behavior. Justify your analysis.

5 Marks · CO4

Given in the question

```
interface Greeting { void sayHello(); }
public class Test {
    public static void main(String[] args) {
        Greeting g = new Greeting() {
            public void sayHello() { System.out.println("Hello!"); }
        };
        g.sayHello();
    }
}
```

Purpose. To provide a **one-time implementation** of the interface `Greeting` right at the point of use, without writing a separate named class. It is typical for callbacks and event handlers.

Execution behaviour.

1. At run time, `new Greeting() { ... }` creates an object of an unnamed class that implements `Greeting` (the compiler generates it as `Test$1`).
2. The reference to this object is held in `g`, of the interface type, which is **upcasting**.
3. `g.sayHello()` is resolved by dynamic method dispatch to the anonymous class's body, so the output is

Output

```
Hello!
```

Justification. An interface cannot be instantiated, so `new Greeting()` is legal only because the braces supply the missing method body, creating a subclass on the spot. The anonymous class cannot have an explicit constructor and cannot be reused elsewhere; if reuse is required, a named class is better, and for a single-method interface a lambda (`() -> System.out.println("Hello!")`) is shorter.

Q6. Analyze how inner classes are represented after compilation and explain the relationship between the generated .class files of the outer and inner classes.

5 Marks · CO4

The JVM knows nothing about "inner classes"; the compiler converts each nested class into an **ordinary, separate class file**. For a source file `Outer.java` containing one member class and one anonymous class it generates:

Source construct	Generated file
Outer class	`Outer.class`
Member inner or static nested class `Inner`	`Outer$Inner.class`
Local class `Helper` in a method	`Outer$1Helper.class`
Anonymous class (first, second...)	`Outer$1.class`, `Outer$2.class`

Relationship.

- The name `Outer$Inner` records the nesting; the `$` separates the names.
- The compiler adds a **hidden field** `this$0` in `Outer$Inner` that stores the outer object, and the inner constructor receives the outer object as an extra, hidden first argument.
- To let the two separate classes use each other's private members, which the JVM would normally forbid, the compiler generates package-level **synthetic accessor methods** (modern versions use nestmate attributes).
- At run time the two classes are loaded independently, but the inner one is useless without the outer.

```
class Outer { class Inner { } }
// javac Outer.java  ->  Outer.class   Outer$Inner.class
```

Q7. Analyze why the package concept is useful for organizing and maintaining large Java applications.

5 Marks · CO4

A large application may contain thousands of classes. Packages bring order and control to that scale.

- **Modular structure:** classes are grouped by function (`com.shop.billing`, `com.shop.inventory`), so developers find code quickly and teams own separate packages.
- **Unique names:** reverse-domain naming (`com.shop.billing.Invoice`) prevents conflicts between classes written by different teams or libraries, such as `java.util.Date` and `java.sql.Date`.
- **Access control:** package-private and protected access allow internal classes to be hidden from the rest of the application while a small public API is exposed, which reduces coupling.
- **Reuse and distribution:** a package can be compiled, packed into a JAR and reused in other applications.
- **Maintainability:** a change inside a package affects only its public interface's users; unit testing and dependency management are done package-wise.
- **Layered design:** packages mirror layers (UI, service, data), making the architecture visible in the folder structure.

> **Example.** In a billing system, `Invoice` is public but its helper `TaxTable` is package-private in `com.shop.billing`; no other package can depend on `TaxTable`, so it can be rewritten freely.

Q8. Analyze the difference between importing an entire package and importing a particular class from that package, with suitable examples.

5 Marks · CO4

Aspect	`import pkg.*;`	`import pkg.ClassName;`
Scope	All public classes of the package, but not sub-packages	Only the named class
Readability	Origin of a class is not obvious	Clear which classes are used
Name clashes	Possible if two imported packages have a class of the same name (ambiguity error on use)	Avoided; a specific import takes priority over a wildcard

Aspect	`import pkg.*;`	`import pkg.ClassName;`
Typing effort	Less	More
Run time / class size	Same; classes are loaded only when used	Same

```
import java.util.*;      // List, Date, ArrayList ...
import java.sql.*;       // also has a Date
// Date d;              // error: reference to Date is ambiguous
import java.util.Date;   // specific import resolves it
```

Analysis. The wildcard is convenient for programs using many classes of one package; the specific import is preferred in professional code for clarity and to avoid clashes. Neither affects the generated byte code, because import is only a compile-time name shortcut.

Q9. Analyze whether importing a package automatically makes its subpackages available. Give suitable justification.

5 Marks · CO4

No, it does not. Although `java.util` and `java.util.regex` appear nested in the file system, the Java language treats them as **two unrelated packages**. A wildcard import `import java.util.*;` brings in the types declared directly in `java.util` only. The justification is in the language definition: a package name is a flat qualified identifier, and there is no special containment relationship between `a.b` and `a.b.c` as far as imports and access rules are concerned. Even package-private access does not extend from one to the other.

```
import java.util.*;
class T {
    List<String> l;              // OK: java.util.List
    // Pattern p;                // error: java.util.regex.Pattern not imported
}
// a separate import is required at the top of the file:  import java.util.regex.*;
```

> **Example.** To use `Pattern` we must write `import java.util.regex.Pattern;` or `import java.util.regex.*;` in addition to `java.util.*`.

Q10. Analyze the accessibility of private, default, protected and public members when the accessing class belongs to the same package and a different package.

5 Marks · CO4

Let `class A` in package `p1` have members `pr` (private), `df` (default), `pt` (protected) and `pb` (public).

Accessing class	pr	df	pt	pb
Class A itself	Yes	Yes	Yes	Yes
Another class, same package p1	No	Yes	Yes	Yes
Subclass of A in another package p2	No	No	Yes (through inheritance)	Yes

Accessing class	pr	df	pt	pb
Non-subclass in another package p2	No	No	No	Yes

Analysis.

- Within the same package, everything except `private` is available, because the default level is defined as “package-wide”.
- In another package, access narrows: a subclass can additionally use `protected` members, but only via itself (`this` or an object of the subclass type), whereas an unrelated class is restricted to `public`.
- The `private` member is never accessible outside the declaring class; it can only be reached through a public method.
- For any of this to apply to a different package, the class `A` itself must also be `public`.

Q11. Given two classes belonging to different packages, analyze which members can be accessed and justify the result based on Java access protection rules.

5 Marks · CO4

Consider:

Given in the question

```
package p1;
public class Account {
    private int pin = 1234;
    int branch = 10;
    protected double balance = 5000;
    public String holder = "Mita";
}
package p2;
import p1.Account;
public class Test {
    public static void main(String[] args) {
        Account a = new Account();
        System.out.println(a.holder);     // (1)
        System.out.println(a.balance);    // (2)
        System.out.println(a.branch);     // (3)
        System.out.println(a.pin);        // (4)
    }
}
```

Result. Only statement (1) compiles.

- (1) `holder` is **public**: accessible from any package.
- (2) `balance` is **protected**: visible in other packages only to subclasses; `Test` does not extend `Account`, so it is an error.
- (3) `branch` is **default**: restricted to package `p1`; `Test` is in `p2`, so it is an error.
- (4) `pin` is **private**: restricted to the class `Account`.

The class `Account` is public, so `Test` can import and instantiate it; if it were not public, even line 1 would fail because the class would be invisible. If `Test extends Account`, line (2) would be legal when accessed through `this.balance`.

Q12. Analyze a Java program using protected members and determine whether access is permitted from a subclass and a non-subclass in another package.

5 Marks · CO4

Given in the question

```
package p1;
public class Vehicle {
    protected int speed = 80;
    protected void show() { System.out.println("Speed " + speed); }
}
package p2;
import p1.Vehicle;
class Car extends Vehicle {                // subclass
    void test() { show(); System.out.println(speed); }       // (A)
}
class Garage {                             // non-subclass
    void test(Vehicle v) { v.show(); System.out.println(v.speed); }   // (B)
}
```

Determination.

- **(A) Permitted.** `Car` is a subclass, so it inherits `speed` and `show()` and may use them through `this` (or a `Car` reference). Protected access extends beyond the package precisely for inheritance.
- **(B) Not permitted.** `Garage` is neither in package `p1` nor a subclass of `Vehicle`, so `v.show()` and `v.speed` fail to compile ("has protected access in Vehicle").

Additional subtlety. Even inside `Car`, `new Vehicle().speed` is not allowed: in a different package the protected member is reachable only through a reference of the subclass's own type (`Car`) or its descendants, not through a plain parent reference.

Q13. Analyze the difference between an in-built package and a user-defined package with respect to organization, reuse and accessibility.

5 Marks · CO4

Aspect	In-built Package	User-Defined Package
Organisation	Defined by the Java platform around general topics (`java.io`, `java.util`)	Defined by the programmer around application features
Reuse	Available to every Java program on every machine with a JDK	Reusable only where the package is on the class path (packaged in a JAR to share)
Accessibility	Public API accessible by import (except `java.lang`, imported automatically); internal implementation classes hidden	Public classes accessible after import; default and protected levels restrict use by package
Stability	Stable, backward compatible, documented	Changes with the application, owned by the developer
Naming	Starts with `java.` or `javax.`; the name `java` is reserved	Normally reverse domain name, such as `com.aot.exam`
Compilation / location	Pre-compiled in the JDK modules	Compiled by `javac -d` into a folder tree

Analysis. Both obey the same mechanisms (package statement, import, access rules). In-built packages provide a common foundation that every developer shares, while user-defined packages organise the application's own logic. Good applications reuse the in-built packages and place their own code in well-named packages.

Q14. Examine the package declaration and import statements in a Java program and determine whether the program will compile successfully. Justify your answer.

5 Marks · CO4

The question bank gives no listing, so the rules are shown by examining three typical programs.

```
// Program 1: compiles
package mypack;
import java.util.ArrayList;
public class A { ArrayList<String> l = new ArrayList<>(); }

// Program 2: does NOT compile
import java.util.*;
package mypack;                  // error: package must be the first statement
public class B { }

// Program 3: does NOT compile
package mypack;
import java.utils.ArrayList;     // error: package java.utils does not exist
public class C { }
```

Determination.

- **Program 1** compiles: `package` first, then imports, then the class; the imported class exists.
- **Program 2** fails because the `package` declaration must be the first statement (comments excepted), before any `import`.
- **Program 3** fails because the imported package or class does not exist (a spelling mistake).

Checklist for any program: (1) at most one `package` statement, first in the file; (2) imports after it and before the class; (3) every imported package and class exists and is public; (4) the file is in a folder that matches the package name; (5) a class used from another package is public, and no ambiguous names arise from two wildcard imports; (6) at most one public class per file, named like the file.

Q15. Analyze the effect of using import package.*; and import package.ClassName; in terms of the classes that become directly accessible in a Java program.

5 Marks · CO4

Assume the package `shapes` contains the public classes `Circle`, `Square`, `Triangle`, and a sub-package `shapes.solid` containing `Cube`.

Import statement	Classes directly accessible by simple name
`import shapes.*;`	Circle, Square, Triangle (all public classes of `shapes`); not Cube
`import shapes.Circle;`	Circle only
`import shapes.Circle;` `import shapes.Square;`	Circle and Square
`import shapes.solid.*;`	Cube only

Import statement	Classes directly accessible by simple name
(no import)	None by simple name; use `shapes.Circle` (full name)

Observations.

- The wildcard makes all public classes of exactly that package accessible, but never those of sub-packages.
- A specific import exposes only the named class, so the rest must be written with their full names.
- If both a specific import and a wildcard supply the same simple name, the specific one wins; if two wildcards supply the same name, using it is ambiguous and is an error.
- Neither form copies code or changes the byte code; each only affects name resolution at compile time.

Q16. Examine a Java program using static import and determine whether the imported static members can be referenced without the class name. Justify your answer.

5 Marks · CO4

Given in the question

```
import static java.lang.Math.sqrt;
import static java.lang.Math.PI;
public class Test {
    public static void main(String[] args) {
        System.out.println(sqrt(49));
        System.out.println(PI);
        System.out.println(Math.abs(-5));
    }
}
```

Determination. Yes, the statically imported members `sqrt` and `PI` can be used without the class name `Math`, because `import static` introduces the **simple names of static members** into the scope of the file. Hence the first two statements compile and print

Output

```
7.0
3.141592653589793
```

The third statement `Math.abs(-5)` also compiles, because `Math` belongs to `java.lang`, which is automatically imported, so the class name `Math` itself is visible; it prints 5.

Conditions and limits.

- Only **static** members (fields and methods, including nested static types) can be imported; an instance member cannot.
- `abs(-5)` alone would fail here, because only `sqrt` and `PI` were imported; `import static java.lang.Math.*;` would import all.
- If a local method has the same name as the imported one, the local method takes precedence.

Q17. Analyze the requirement: a class should be accessible outside its package, but one of its data members should not be directly accessible. Select suitable access protection and justify.

5 Marks · CO4

Requirement 1: the class is accessible outside its package. A top-level class can have only the `public` or default level; default would hide it from other packages, so the class must be declared **`public`**.

Requirement 2: a data member must not be directly accessible. The member should be **`private`**. Private is the strictest level, so neither other packages, nor the same package, nor subclasses can touch it directly.

Controlled access. If other classes need to read or change the value, provide public getters and setters (with validation). If only subclasses are meant to use it, then `protected` would be chosen instead, but that contradicts “not directly accessible”, so private is the correct answer.

```
package bank;
public class Account {                    // public: usable from any package
    private double balance;               // private: hidden from everyone else
    public double getBalance() { return balance; }
    public void deposit(double amt) { if (amt > 0) balance += amt; }
}
```

Justification. Public class plus private data plus public methods is the standard encapsulation pattern: the class’s services are available everywhere while its internal state is protected from misuse and can be changed later without affecting users.

MODULE

8

String, StringBuffer and StringBuilder

CA2: immutable strings, mutable character sequences, comparison and performance

Bloom's Level: Remember (L1)

Q1. What is a String in Java?

2 Marks · CO1

A `String` is an object of the class `java.lang.String` that represents a sequence of characters. String objects are **immutable**: once created, their content cannot be changed. String literals are stored in a special area called the string constant pool.

> **Example.** `String s = "Java";` creates a String holding the characters J, a, v, a.

Q2. What is meant by an immutable String?

2 Marks · CO1

Immutable means that the value of a String object cannot be modified after it is created. Any operation that appears to change it, such as `concat()`, `replace()` or `toUpperCase()`, creates and returns a **new** String object, leaving the original untouched.

> **Example.** `String s = "Hi"; s.concat(" there");` The result is lost and `s` is still "Hi".

Q3. Name the two mutable classes used for handling character sequences in Java.

2 Marks · CO1

`StringBuffer` and `StringBuilder`. Both allow the content to be changed in place (append, insert, delete, reverse) without creating new objects each time; `StringBuffer` is thread-safe and `StringBuilder` is not.

Q4. What is the purpose of the length() method of the String class?

2 Marks · CO1

`length()` returns the number of characters in the String (an `int`), counting every character including spaces. Valid indices are 0 to `length() - 1`.

> **Example.** `"Hello World".length()` returns 11.

Q5. Which method is used to add a string at the end of a StringBuilder object?

2 Marks · CO1

The `append()` method. It adds the given value (String, char, int, etc.) at the end of the StringBuilder and returns the same object, so calls can be chained.

> **Example.** `new StringBuilder("Java").append(" Rocks")` gives "Java Rocks".

Q6. What is StringBuffer?

2 Marks · CO1

`StringBuffer` is a class in `java.lang` for creating **mutable** character sequences. Its methods are **synchronized**, so it is thread-safe, which makes it somewhat slower than `StringBuilder`.

> **Example.** `StringBuffer sb = new StringBuffer("Hello"); sb.append(" World");` modifies `sb` itself.

Q7. What is StringBuilder?

2 Marks · CO1

`StringBuilder` (Java 5) is a mutable character-sequence class with the same methods as `StringBuffer`, but its methods are **not synchronized**. It is therefore faster and is the preferred choice in singlethreaded code.

> **Example.** `StringBuilder sb = new StringBuilder("Hello"); sb.insert(0, "Oh ");` gives "Oh Hello".

Q8. Which method is used to reverse the contents of a StringBuilder?

2 Marks · CO1

The `reverse()` method, which reverses the characters in place and returns the same object.

> **Example.** `new StringBuilder("abc").reverse()` gives "cba".

Bloom's Level: Understand (L2)

Q9. Explain why String objects are considered immutable in Java.

4 Marks · CO2

A String object's character content cannot be altered after creation; every operation that seems to modify it produces a **new** object. In the `String` class, the characters are held in a private final array with no method that changes it, and the class itself is `final`, so nobody can subclass it and add mutating behaviour.

Reasons for the design.

- **String pool and memory saving:** identical literals share one object in the constant pool; this would be unsafe if one reference could change the shared value.
- **Security:** Strings are used for file names, URLs, user names, passwords and class names; immutability prevents them being altered after validation.
- **Thread safety:** immutable objects can be shared by threads without synchronization.
- **Hash code caching:** the hash code is computed once and cached, making Strings efficient keys in `HashMap`.

```
String s = "Java";
String t = s.concat(" Programming");     // new object created
System.out.println(s);                    // Java  (unchanged)
System.out.println(t);                    // Java Programming
```

> **Example.** `s.toUpperCase()` returns "JAVA" as a new String, and `s` remains "Java" unless the result is assigned back.

Q10. Explain the difference between String, StringBuffer, and StringBuilder.

4 Marks · CO2

Basis	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable

Basis	String	StringBuffer	StringBuilder
Thread safety	Safe (immutable)	Safe (synchronized methods)	Not thread-safe
Performance for modifications	Slowest; a new object each time	Slower than StringBuilder	Fastest
Storage	String pool or heap	Heap	Heap
Since	Java 1.0	Java 1.0	Java 1.5
Concatenation	`+` or `concat()` creates new objects	`append()`	`append()`
Use when	Content fixed	Modified by several threads	Modified by one thread

```
String s = "Hi";            s = s + "!";          // new object each time
StringBuffer sf = new StringBuffer("Hi");   sf.append("!");   // same object
StringBuilder sb = new StringBuilder("Hi"); sb.append("!");   // same object
```

Q11. Explain the use of the equals() and equalsIgnoreCase() methods with suitable examples.

4 Marks · CO2

`equals(Object o)` compares the **contents** of two Strings and returns `true` only if they have the same characters in the same order and with the same case. `equalsIgnoreCase(String s)` does the same comparison but treats uppercase and lowercase letters as equal. Both return a `boolean` and, unlike `==`, do not compare references.

```
String a = "Java", b = "java", c = new String("Java");
System.out.println(a.equals(c));              // true  (same content)
System.out.println(a.equals(b));              // false (case differs)
System.out.println(a.equalsIgnoreCase(b));    // true
System.out.println(a == c);                   // false (different objects)
```

Output

```
true
false
true
false
```

> **Example.** `equalsIgnoreCase` is useful for user input such as checking whether the answer “YES”, “Yes” or “yes” was typed.

Q12. Explain the purpose of the charAt(), substring() and indexOf() methods of the String class.

4 Marks · CO2

- **`charAt(int index)`** returns the character at the given zero-based position; an invalid index throws `StringIndexOutOfBoundsException`.

- **`substring(int begin)` and `substring(int begin, int end)`** return the part of the String from `begin` up to, but excluding, `end` (or to the end of the string).
- **`indexOf(String/char)` returns the position of the first occurrence of the character or substring, or** -1** if it is absent; `lastIndexOf` searches from the end.

```
String s = "Object Oriented";
System.out.println(s.charAt(0));          // O
System.out.println(s.substring(7));       // Oriented
System.out.println(s.substring(0, 6));    // Object
System.out.println(s.indexOf("Or"));      // 7
System.out.println(s.indexOf('z'));       // -1
```

Output

```
O
Oriented
Object
7
-1
```

Q13. Explain the use of the append() and insert() methods of StringBuilder.

4 Marks · CO2

- **`append(x)`** adds the value `x` (String, char, int, double, boolean, object) at the **end** of the sequence. It returns the same StringBuilder, so calls can be chained.
- **`insert(int offset, x)`** adds the value `x` at the given **position**, shifting the existing characters to the right.

Both modify the object itself, so no new object is created.

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming").append(2026);      // Java Programming2026
sb.insert(4, " Language");                   // Java Language Programming2026
System.out.println(sb);
```

Output

```
Java Language Programming2026
```

Q14. Explain why StringBuilder is generally faster than StringBuffer when thread synchronization is not required.

4 Marks · CO2

Every public method of `StringBuffer` is `synchronized`: before the method runs, the thread must acquire the object's lock, and it releases it afterwards. This guarantees that two threads do not corrupt the buffer, but acquiring and releasing the lock costs time even when only one thread is using the object, and may cause other threads to wait. `StringBuilder` has the same methods but **without synchronization**, so there is no locking overhead. When the object is used by a single thread (a local variable, for example) the protection is unnecessary, and `StringBuilder` runs faster, especially in loops with many `append()` operations. The price is that it must not be shared among threads without external synchronization.

> **Example.** For a loop that appends 1,000,000 characters, `StringBuilder` typically takes noticeably less time than `StringBuffer`, because 1,000,000 lock operations are avoided.

Q15. Explain the difference between == and equals() when comparing String objects.

4 Marks · CO2

== compares **references**: it is true only when both variables point to the very same object in memory. `equals()` compares the **contents** of the Strings character by character.

String literals with the same text are stored once in the string pool, so == is true for two literals, but a String created with `new` is a separate heap object.

```
String a = "Java";
String b = "Java";
String c = new String("Java");
System.out.println(a == b);          // true  (same pooled object)
System.out.println(a == c);          // false (different objects)
System.out.println(a.equals(c));     // true  (same content)
System.out.println(a == c.intern()); // true  (intern returns the pooled one)
```

Output

```
true
false
true
true
```

Conclusion: use `equals()` (or `equalsIgnoreCase()`) to compare String values; use == only to test whether two references are identical.

Bloom's Level: Apply (L3)

Q16. Write a Java program to create a String and demonstrate the use of length(), charAt(), and substring() methods.

5 Marks · CO3

```
public class StringMethods {
    public static void main(String[] args) {
        String s = "Object Oriented Programming";
        System.out.println("String      : " + s);
        System.out.println("Length      : " + s.length());
        System.out.println("charAt(0)   : " + s.charAt(0));
        System.out.println("charAt(7)   : " + s.charAt(7));
        System.out.println("substring(7): " + s.substring(7));
        System.out.println("substring(0,6): " + s.substring(0, 6));
        // last character using length()
        System.out.println("Last char   : " + s.charAt(s.length() - 1));
    }
}
```

Output

```
String      : Object Oriented Programming
Length      : 27
charAt(0)   : O
charAt(7)   : O
substring(7): Oriented Programming
substring(0,6): Object
Last char   : g
```

Example. Indexing starts from 0, so the last valid index is `length() - 1` = 26. `charAt(27)` would throw `StringIndexOutOfBoundsException`.

Q17. Write a Java program using StringBuilder to append two strings and display the resulting string.

5 Marks · CO3

```
public class AppendDemo {
    public static void main(String[] args) {
        String first = "Object";
        String second = " Oriented";

        StringBuilder sb = new StringBuilder(first);
        sb.append(second);                    // appends the second string
        System.out.println("Result : " + sb);
        System.out.println("Length : " + sb.length());

        sb.append(" Programming").append('!'); // chained appends
        System.out.println(sb.toString());
    }
}
```

Output

```
Result : Object Oriented
Length : 15
Object Oriented Programming!
```

Example. `append()` modifies the same StringBuilder object and returns it, so the first and second calls act on one object; no new objects are created for the intermediate results.

Q18. Write a Java program using StringBuffer to demonstrate the append(), insert() and delete() methods.

5 Marks · CO3

```
public class BufferDemo {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World");                  // Hello World
        System.out.println("After append : " + sb);

        sb.insert(5, ",");                    // Hello, World
        System.out.println("After insert : " + sb);

        sb.delete(5, 6);                      // removes index 5 (the comma)
        System.out.println("After delete : " + sb);

        sb.deleteCharAt(0);                   // removes 'H'
        System.out.println("deleteCharAt : " + sb);
        sb.replace(0, 4, "Jelly");            // replaces "ello"
        System.out.println("After replace: " + sb);
    }
}
```

Output

```
After append : Hello World
After insert : Hello, World
After delete : Hello World
deleteCharAt : ello World
After replace: Jelly World
```

> **Example.** `delete(start, end)` removes characters from `start` up to but not including `end`.

Q19. Write a Java program to reverse a given string using the StringBuilder class.

5 Marks · CO3

```
import java.util.Scanner;

public class ReverseString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();

        String rev = new StringBuilder(s).reverse().toString();
        System.out.println("Reversed: " + rev);

        if (s.equalsIgnoreCase(rev))
            System.out.println("It is a palindrome");
        else
            System.out.println("It is not a palindrome");
    }
}
```

Output

```
Enter a string: Madam
Reversed: madaM
It is a palindrome
```

> **Example.** `reverse()` works in place on the builder; `toString()` converts the result back to a String. For “Java” the output would be “avaJ”.

Q20. Write a Java program to count the number of vowels in a given String.

5 Marks · CO3

```
public class VowelCount {
    public static void main(String[] args) {
        String s = "Object Oriented Programming";
        int count = 0;
        String lower = s.toLowerCase();
        for (int i = 0; i < lower.length(); i++) {
            char ch = lower.charAt(i);
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                count++;
        }
        System.out.println("String : " + s);
        System.out.println("Vowels : " + count);
    }
}
```

Output
```
String : Object Oriented Programming
Vowels : 9
```

Example. Counting the vowels word by word: Object has O and e (2); Oriented has O, i, e, e (4); Programming has o, a, i (3). Total = 2 + 4 + 3 = 9. The program converts the string to lower case first so that capital vowels such as 'O' are counted too.

Q21. Determine the output of: String s = "Java"; s.concat(" Programming"); System.out.println(s);

5 Marks · CO3

Output:

Output
```
Java
```

Explanation. A String is immutable. The method `concat()` does not change `s`; it returns a **new** String "Java Programming". Since the returned value is not assigned to any variable, it is discarded (it becomes eligible for garbage collection), and `s` still refers to the original object "Java". To keep the result the program must write `s = s.concat(" Programming");`, which makes `s` refer to the new String.

```
String s = "Java";
s.concat(" Programming");          // result ignored
System.out.println(s);             // Java
s = s.concat(" Programming");      // result stored
System.out.println(s);             // Java Programming
```

Example. Memory view: after line 2 two String objects exist ("Java" referred to by `s` and "Java Programming" with no reference); `s` has not moved.

Q22. Determine the output of: StringBuilder sb = new StringBuilder("Java"); sb.append(" Programming"); System.out.println(sb);

5 Marks · CO3

Output:

Output
```
Java Programming
```

Explanation. `StringBuilder` is mutable. The `append()` method adds " Programming" to the end of the **same** object that `sb` refers to, changing its content. When `println(sb)` is called, the object's `toString()` gives the modified content "Java Programming". Unlike the `String.concat()` case, the result need not be assigned: the change is made to the object itself. No new object holds the result; only the internal character buffer grows if its capacity is insufficient.

Code	String s	StringBuilder sb
Operation	s.concat(" Programming")	sb.append(" Programming")
Object changed?	No, a new String returned	Yes, the same object

Code	String s	StringBuilder sb
Printed value after statement	Java	Java Programming

Bloom's Level: Analyze (L4)

Q23. Analyze the differences among String, StringBuffer and StringBuilder with respect to mutability, synchronization and performance.

5 Marks · CO4

Mutability. `String` is immutable: each change creates a new object. `StringBuffer` and `StringBuilder` are mutable: methods such as `append`, `insert`, `delete` and `reverse` change the same object using an internal resizable character array.

Synchronization. `String` needs none because it cannot change, so it is naturally thread-safe. `StringBuffer` methods are `synchronized`, so concurrent threads cannot corrupt the content. `StringBuilder` has no synchronization and is unsafe if shared by threads without external locking.

Performance. Repeated modification of a `String` is slowest: n concatenations copy characters again and again (time about proportional to n squared) and create many temporary objects. `StringBuffer` avoids the object creation but pays a locking cost. `StringBuilder` is fastest, since it has neither object creation nor locking.

Aspect	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Thread safety	Yes, inherently	Yes, synchronized	No
Speed in repeated modification	Slowest	Medium	Fastest
Memory churn	High	Low	Low
Best for	Constants, keys, fixed text	Shared, concurrently modified text	Local, single-threaded text building

Q24. Analyze the code: String s1 = "ABC"; s1.concat("D"); StringBuilder s2 = new StringBuilder("ABC"); s2.append("D"); and explain why the values of s1 and s2 behave differently.

5 Marks · CO4

Given in the question

```
String s1 = "ABC";
s1.concat("D");
StringBuilder s2 = new StringBuilder("ABC");
s2.append("D");
```

Result. After these statements `s1` is **"ABC"** and `s2` is **"ABCD"**.

Explanation.

- `String` is immutable. `s1.concat("D")` builds a new String "ABCD" and returns it; since the result is not stored, it is lost. The object that `s1` refers to is untouched.

- `StringBuilder` is mutable. `s2.append("D")` modifies the character buffer of the very object `s2` refers to, so `s2` now contains “ABCD”. The returned reference is the same object, and ignoring it is harmless.

Therefore the same-looking operation has a different effect: an **assignment is needed for String** (`s1 = s1.concat("D")`), but not for StringBuilder.

Q25. A program performs thousands of string concatenation operations in a single-threaded environment. Analyze whether String, StringBuffer or StringBuilder is most appropriate and justify.

5 Marks · CO4

Most appropriate: `StringBuilder`.

- **Not `String`:** every `+` or `concat()` creates a new String and copies all earlier characters. For n concatenations about n(n+1)/2 characters are copied and n temporary objects are produced, burdening the garbage collector. Thousands of operations would be slow and memory hungry.
- **Not `StringBuffer`:** it avoids the repeated copying, but each `append()` acquires a lock. In a single-threaded program this synchronization is pure overhead.
- **`StringBuilder`:** it appends into one growing buffer (amortised constant time per append) with no lock, giving the best speed.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) sb.append(i).append(',');
String result = sb.toString();
```

> **Example.** Tip: if the final length is roughly known, `new StringBuilder(capacity)` avoids buffer re-allocations.

Q26. A multithreaded application frequently modifies a character sequence. Analyze the suitability of String, StringBuffer and StringBuilder for this application.

5 Marks · CO4

- **`String` is unsuitable for frequent modification.** It is thread-safe, but every change allocates a new object, so a shared variable would be reassigned by threads (`s = s + x`) in a non-atomic manner, and updates can be lost; it is also inefficient.
- **`StringBuilder` is unsuitable if the object is shared.** Without synchronization two threads calling `append()` simultaneously can corrupt the internal array or lose characters, giving wrong output or exceptions.
- **`StringBuffer` is suitable.** Its synchronized methods make each operation atomic, so that concurrent calls on one object are executed one at a time and the content stays consistent.

Conclusion. Use `StringBuffer` for a character sequence **shared** by several threads. A further caution: although each method is atomic, a sequence of calls (check then modify) is not; such compound actions need extra synchronization with `synchronized` blocks. If each thread has its own private builder, `StringBuilder` can still be used.

```
StringBuffer log = new StringBuffer();      // shared by threads
// Thread 1: log.append("T1 ");   Thread 2: log.append("T2 ");   -> safe
```

Q27. Analyze the statement and explain its effect on String objects: String s = “Java”; s = s + “ Programming”;

5 Marks · CO4

Step by step.

1. `String s = "Java";` creates (or reuses from the pool) the String object **A** = “Java”, and `s` refers to A.
2. The expression `s + " Programming"` creates a **new** String object **B** = “Java Programming”. (The literal “ Programming” is also a pooled object **C**.) The compiler translates the `+` into a `StringBuilder` append and `toString()`.
3. The assignment `s = ...` makes `s` refer to B.

Effect. Object A is **not modified**; it remains “Java” in the pool, merely no longer referred to by `s`. There are now three objects: A (“Java”), C (“ Programming”) and B (“Java Programming”). Only the **reference** `s` changed, not any String. This is what immutability means: the variable changes, the object does not. In a loop, this pattern creates many garbage Strings, which is why `StringBuilder` is preferred there.

Diagram

```
Before:  s ---> "Java"
After :  s ---> "Java Programming"     "Java" (still in pool, unreferenced by s)
```

Q28. Compare the use of StringBuffer and StringBuilder in terms of synchronization, thread safety and performance.

5 Marks · CO4

Aspect	StringBuffer	StringBuilder
Synchronization	All main methods are `synchronized`	None
Thread safety	Safe for concurrent use of a shared object	Not safe if shared
Performance	Slower due to lock acquisition and release	Faster; no locking overhead
API	Same methods (append, insert, delete, reverse, replace)	Identical methods
Introduced	Java 1.0	Java 5, as a drop-in unsynchronized replacement
Preferred when	A buffer is modified by several threads	The buffer is local to one thread (the common case)

Discussion. The two classes have the same API, so one can be replaced by the other by changing only the class name. Because most string building takes place inside one method, `StringBuilder` is the default choice. `StringBuffer`‘s thread-safety applies to individual calls only; sequences of calls still need external locking, which further reduces its advantage. A common micro-benchmark shows `StringBuilder` is roughly 10 to 30 percent faster in heavy appending loops (results vary with the JVM).

Q29. A developer uses String for repeated modifications of a large text. Analyze the possible impact on object creation and performance, and suggest a suitable alternative.

5 Marks · CO4

Impact.

- **Object creation:** each modification (`+=`, `concat`, `replace`, `insert` simulated by substring) creates a new String object. For k modifications of a text of n characters, k new large objects are created, most of them garbage immediately.
- **Time:** every operation copies the whole existing text into the new object, so total time grows roughly as k x n, or quadratically in loops that append one piece at a time.
- **Memory and GC:** large temporary arrays increase heap usage and the frequency of garbage collection, which can cause pauses.

```
String text = "";
for (int i = 0; i < 100000; i++) text += "line " + i + "\n";    // very slow
```

Alternative. Use **`StringBuilder`** (or `StringBuffer` if shared by threads). It keeps a single resizable array, modifies it in place and copies characters only when the capacity must grow (doubling), so the total cost is linear.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100000; i++) sb.append("line ").append(i).append('\n');
String text = sb.toString();            // one conversion at the end
```

> **Example.** Typical outcome: the String loop above may take seconds and create 100,000 big temporary objects, whereas the StringBuilder loop finishes in milliseconds.

Q30. Analyze an application that requires frequent insertion, deletion and reversal of characters. Justify whether StringBuffer or StringBuilder should be selected based on the application requirements.

5 Marks · CO4

`insert()`, `delete()` and `reverse()` are provided by both `StringBuffer` and `StringBuilder` and work in place; a `String` cannot do them without creating new objects. The choice between the two depends on **thread usage**:

Application condition	Choice	Reason
Text is edited by one thread (a text editor buffer on the UI thread, a parser, a local utility method)	**StringBuilder**	No lock needed; faster for the many frequent operations
Text object is shared and edited by several threads at once (shared log, collaborative document buffer)	**StringBuffer**	Synchronized methods keep each operation atomic and the content consistent

Justification. The operations named in the question are frequent, so the cost of every call matters: for the usual single-threaded application, `StringBuilder` gives the best performance. If threads genu-

inely share one buffer, correctness outweighs speed and `StringBuffer` is selected (compound operations still need additional synchronization).

```
StringBuilder sb = new StringBuilder("Hello World");
sb.insert(5, ",");        // Hello, World
sb.delete(5, 6);          // Hello World
sb.reverse();             // dlroW olleH
```